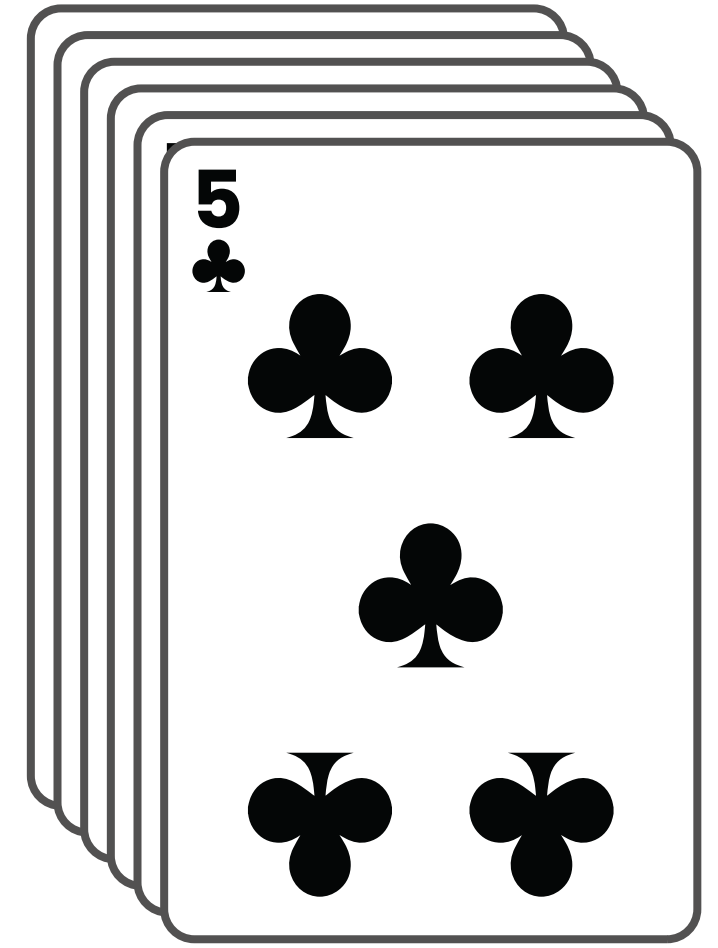


Ordenació

(II) Ordenació per Selecció

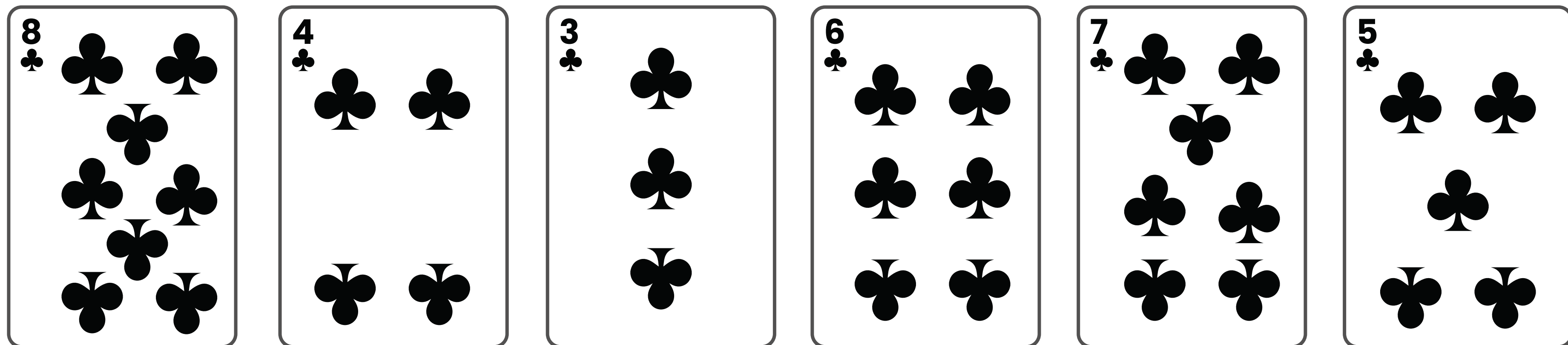
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

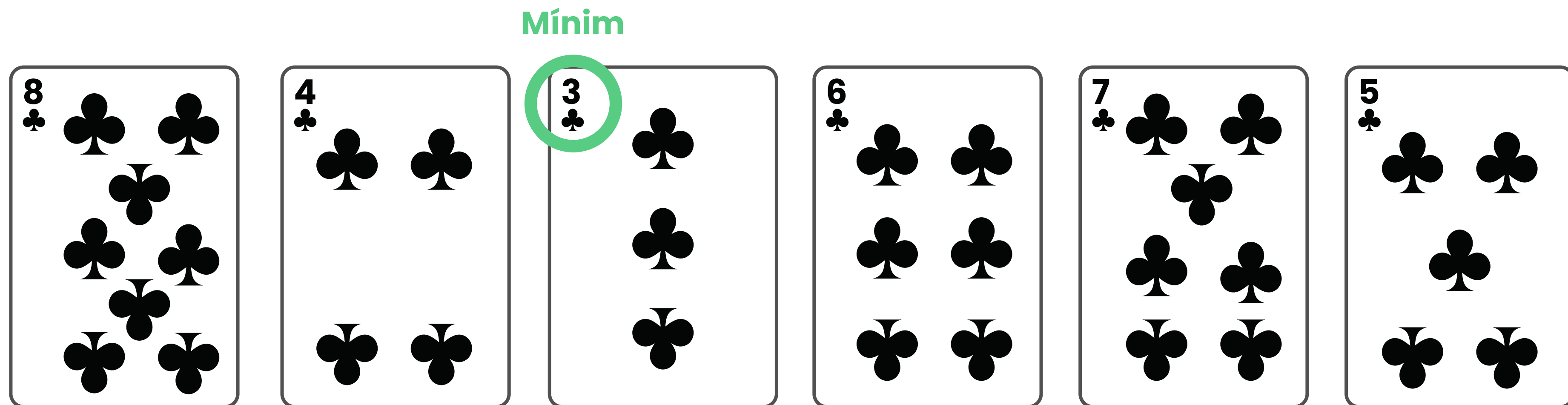
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

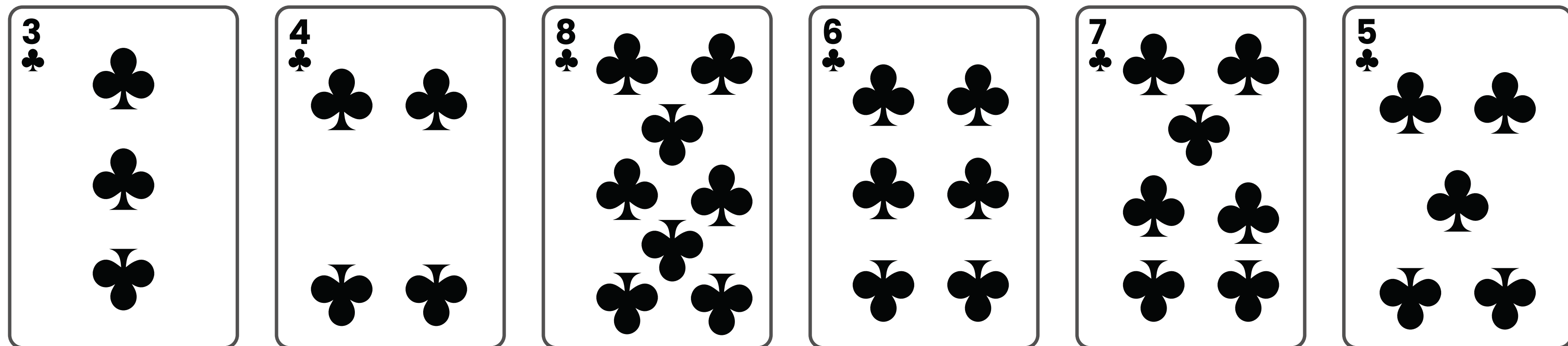
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

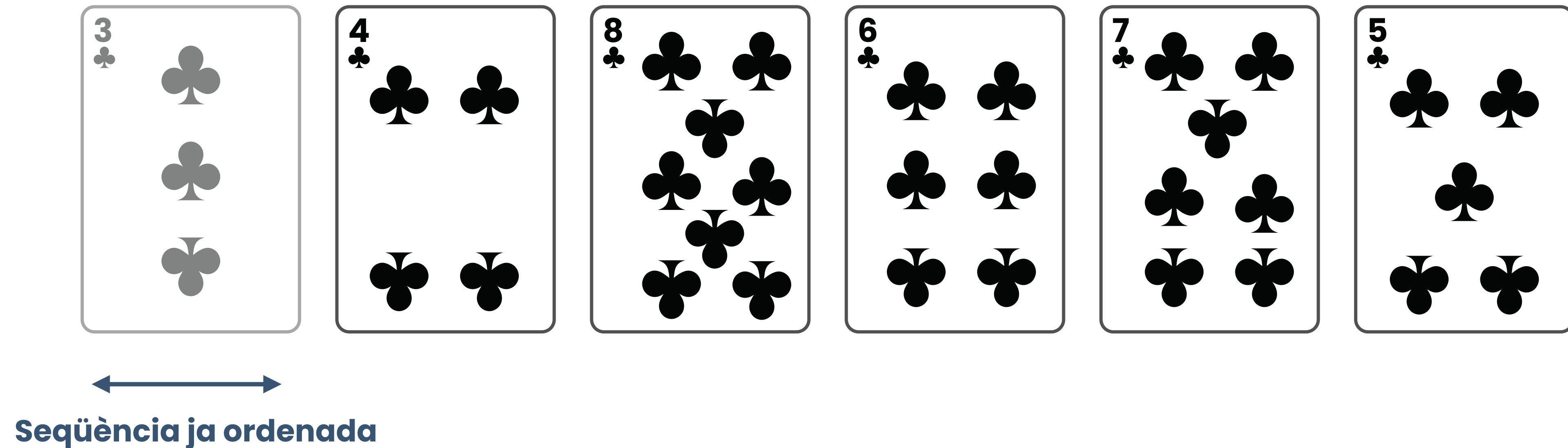
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

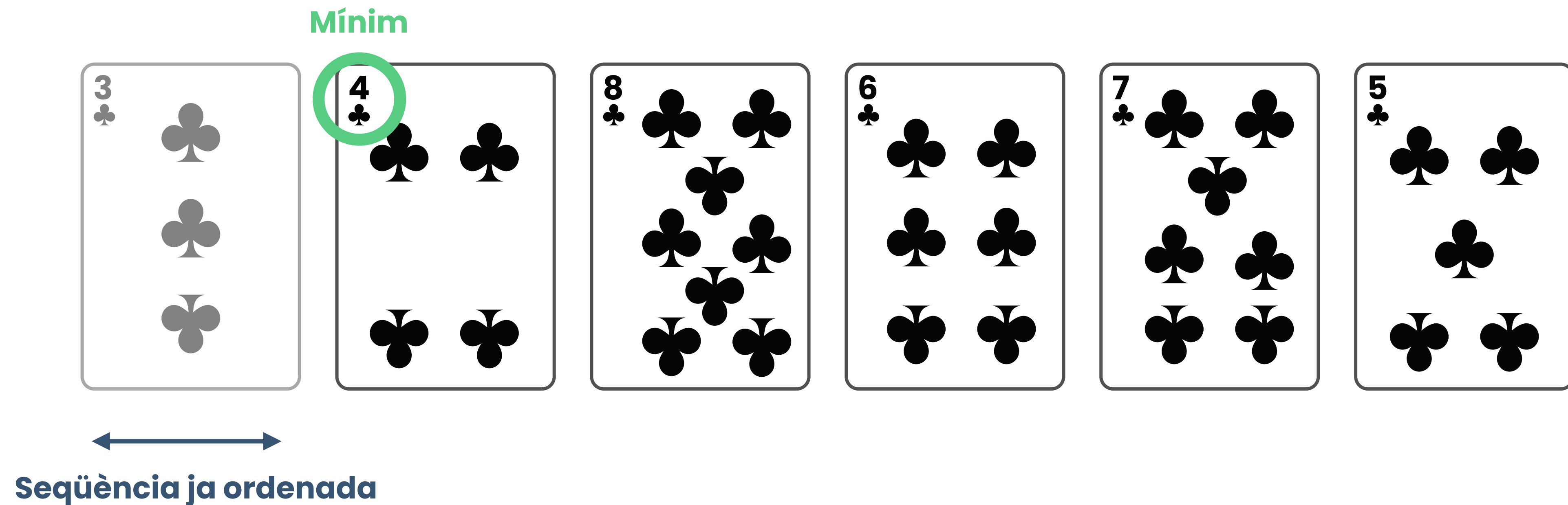
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

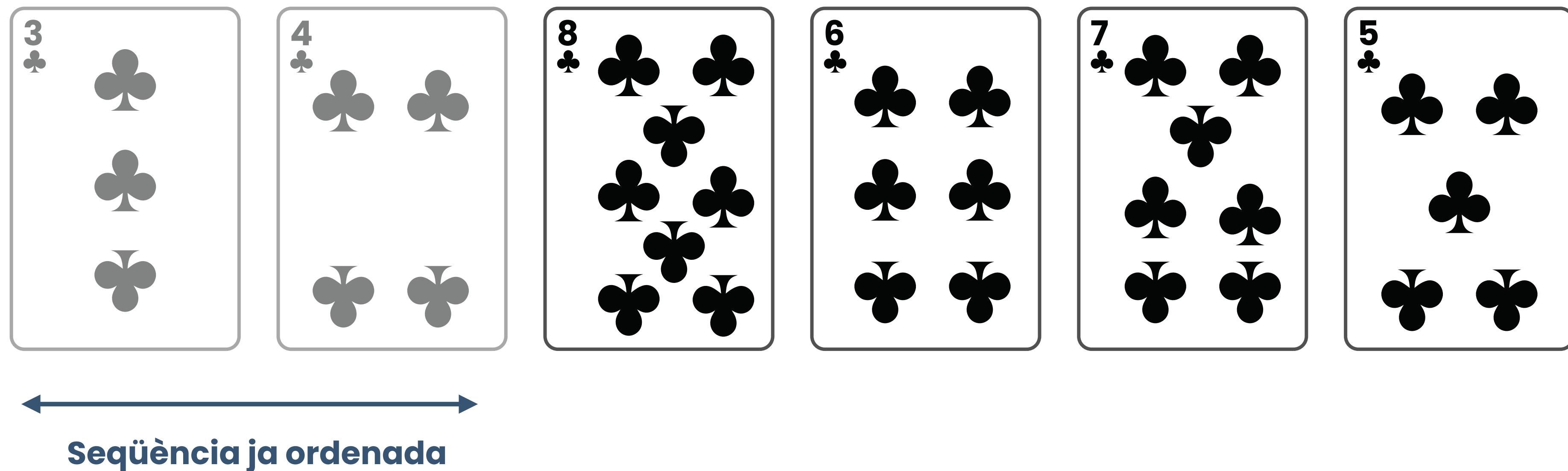
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

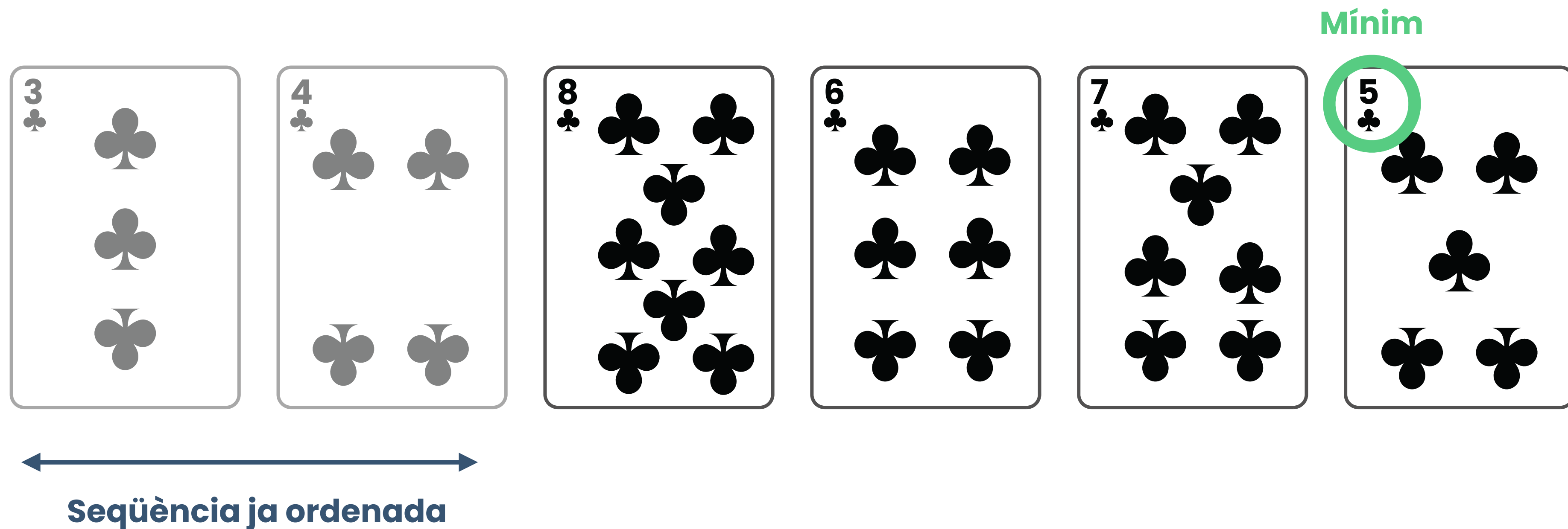
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

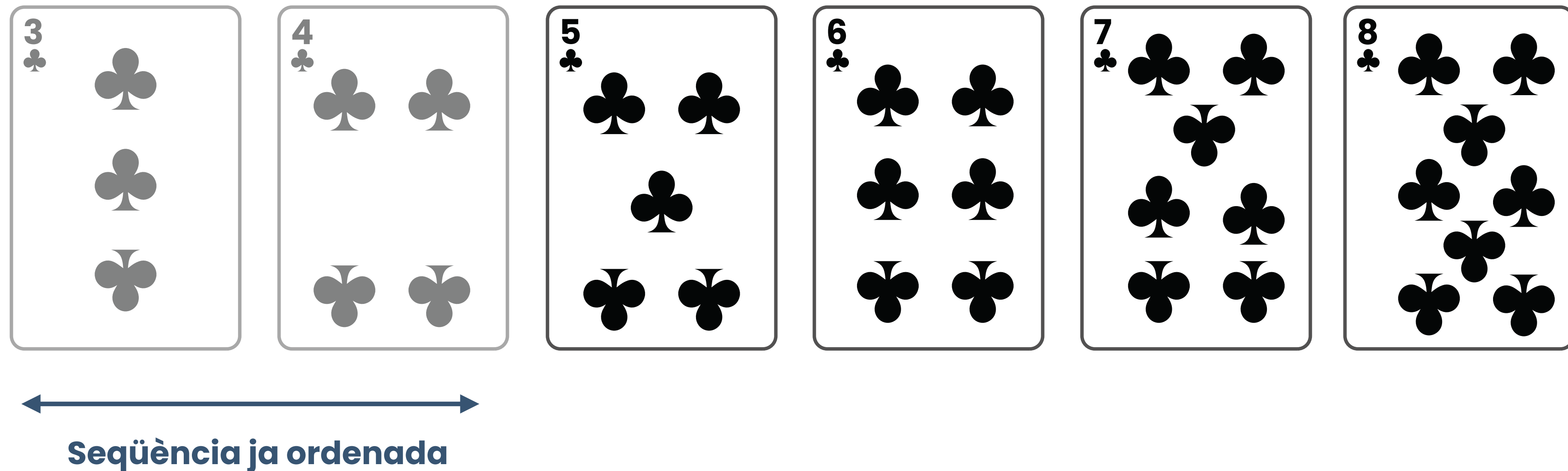
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

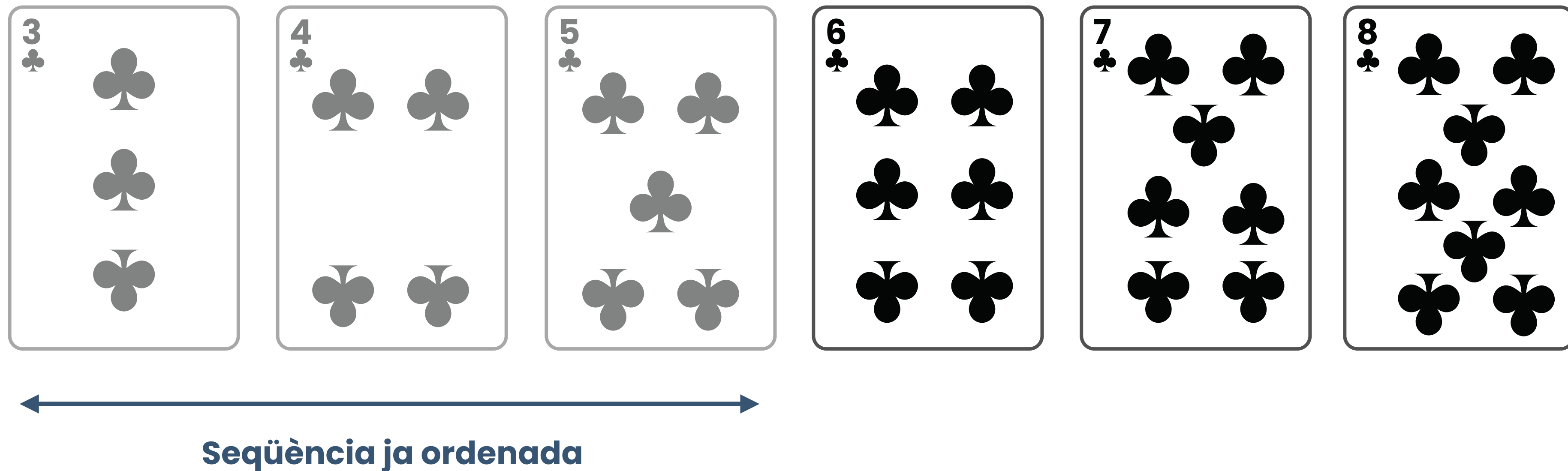
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

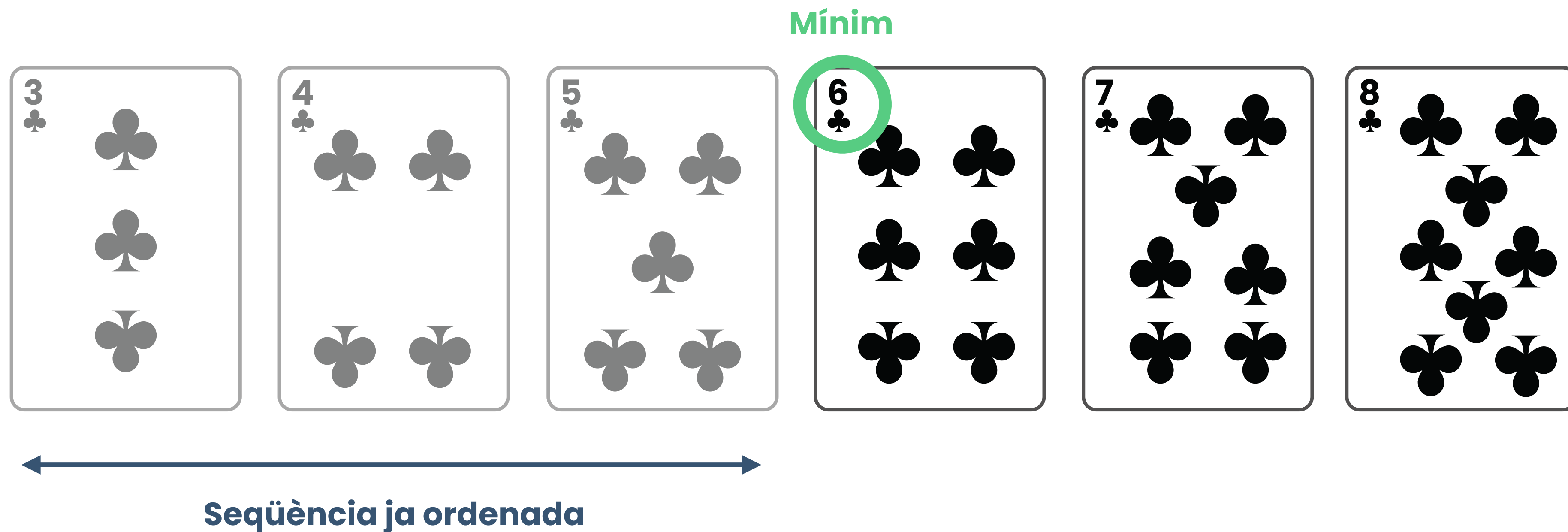
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

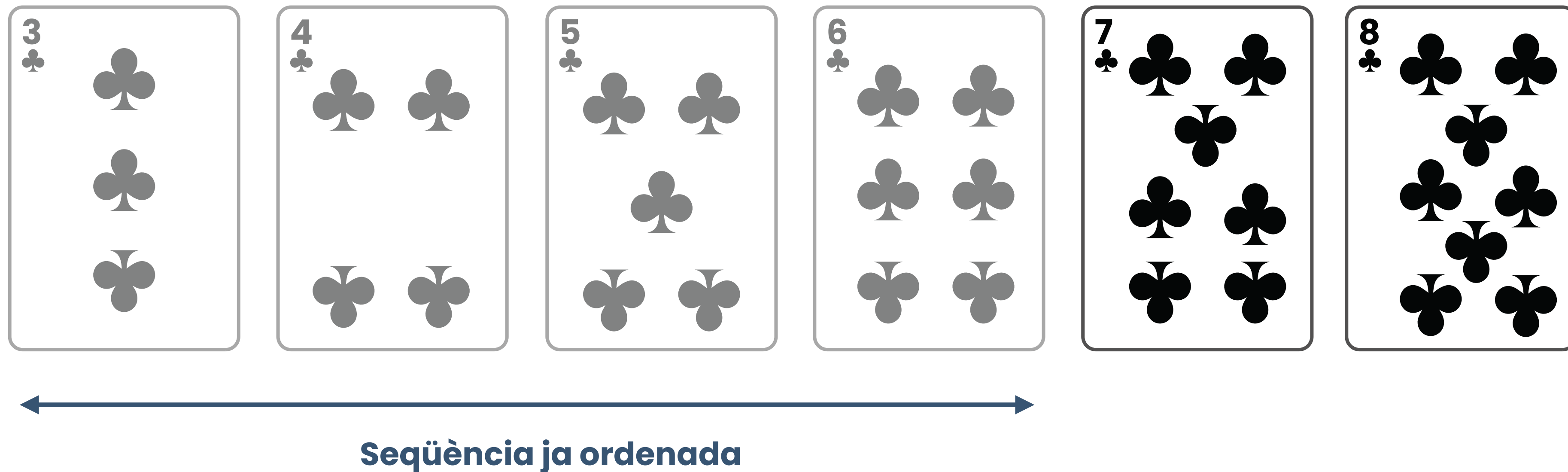
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

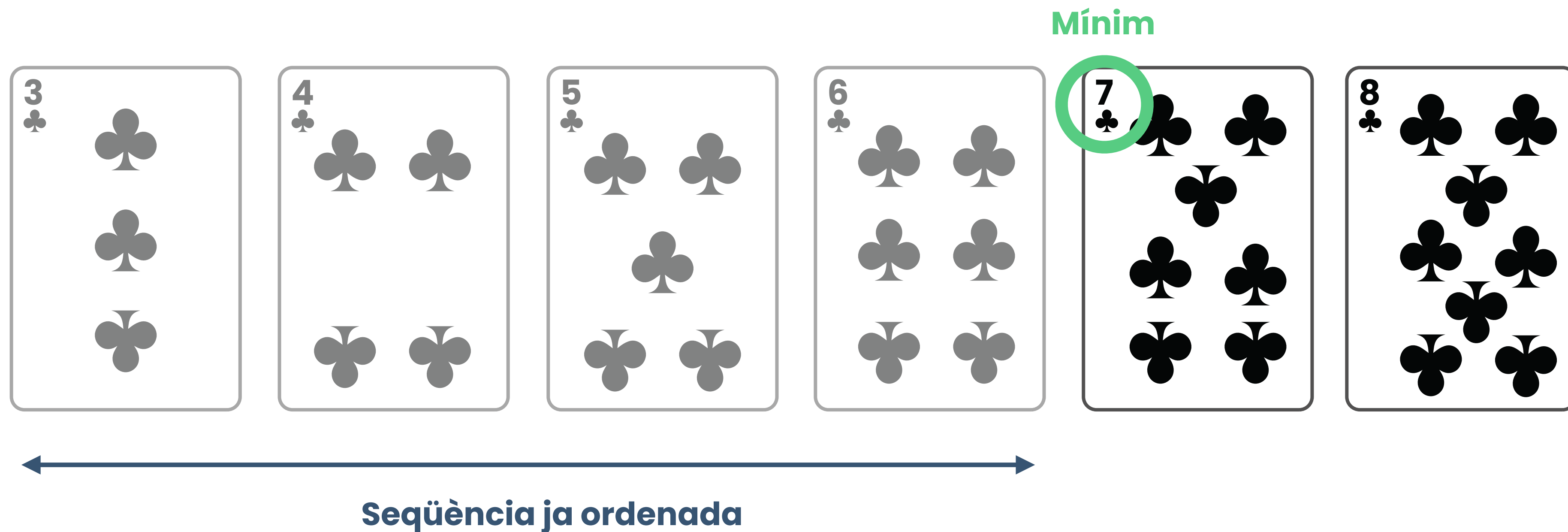
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

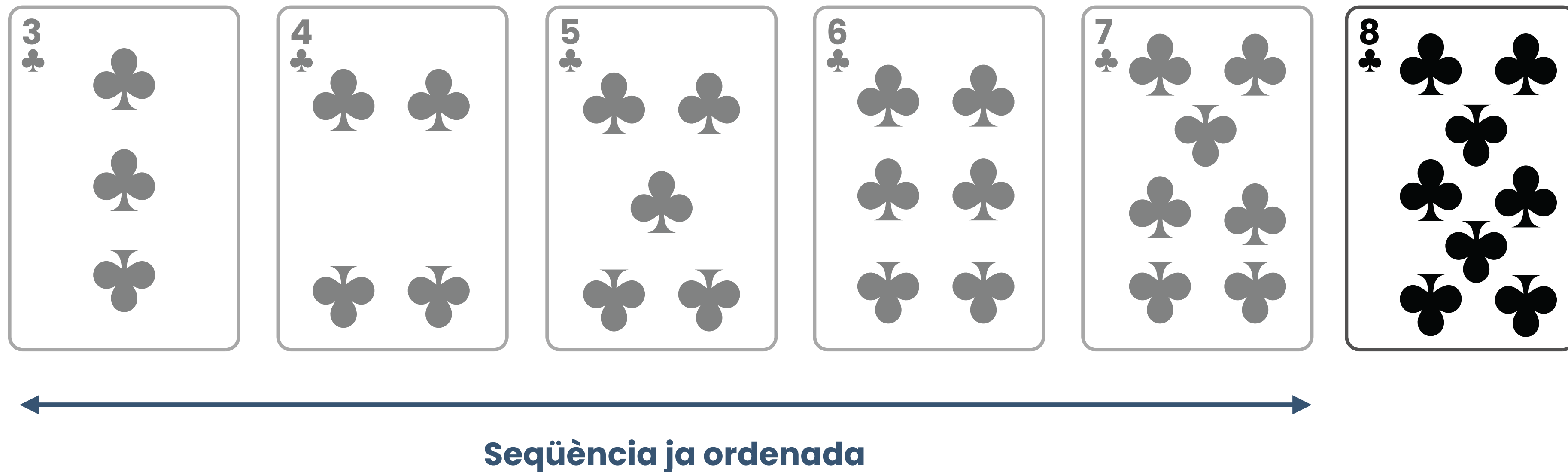
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

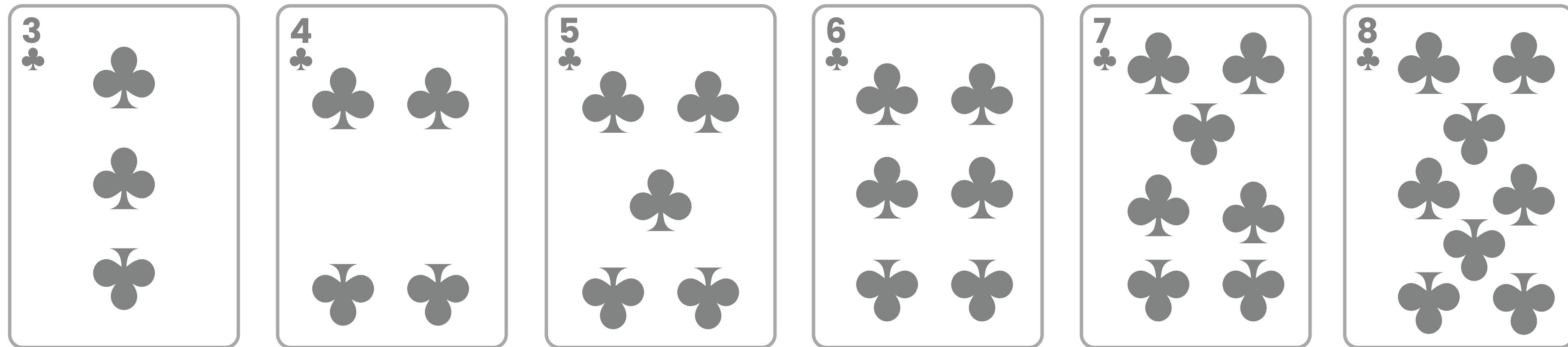
- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Ordenació

(II) Ordenació per Selecció

- Aquest algorisme es basa en:
 - Donat un conjunt de registres, seleccionar el registre amb clau mínima
 - Intercanviar-lo amb el registre del conjunt que ocupa la primera posició.
 - Repetir aquests passos sense tenir en compte l'element que acabem de tractar (com si el traguéssim del conjunt)



Seqüència ja ordenada HEM ACABAT!

Ordenació

(II) Ordenació per Selecció

Algorisme

```
$ Suposem ordenació creixent
acció ordenacio_seleccio (v: taula[] d'enter, n_elems: enter) és
var
  i, j, pos, minim : enter;
fvar
inici
  per (i=0; i < n_elems-1; i:=i+1) fer
    minim := v[i];
    pos := i;
    $ Cercar el minim (no de tots, només del conjunt "actiu")
    per (j:=i+1; j < n_elems; j:=j+1) fer
      si (v[j] < minim)
        minim := v[j];
        pos := j;
      fsi
    fper
    $ Fem l'intercanvi
    v[pos] := v[i];
    v[i] := minim;
  fper
facció
```

Ordenació

(II) Ordenació per Selecció

Anàlisi de costos:

- Cada "passada" consisteix en trobar el mínim ($O(n)$) i intercanviar-lo amb el primer dels no-ordenats.

Ordenació

(II) Ordenació per Selecció

Anàlisi de costos:

- Cada "passada" consisteix en trobar el mínim ($O(n)$) i intercanviar-lo amb el primer dels no-ordenats.
- En el **millor cas**, el vector ja està ordenat: $\{ 1, 2, 3, 4, 5 \}$
- No haurà de fer cap intercanvi però haurà de buscar tots els mínims.

Best case = $O(n^2)$ comparacions, $O(1)$ intercanvis COST = $O(n^2)$

Ordenació

(II) Ordenació per Selecció

Anàlisi de costos:

- Cada "passada" consisteix en trobar el mínim ($O(n)$) i intercanviar-lo amb el primer dels no-ordenats.
- En el **millor cas**, el vector ja està ordenat: $\{ 1, 2, 3, 4, 5 \}$
- No haurà de fer cap intercanvi però haurà de buscar tots els mínims.

Best case = $O(n^2)$ comparacions, $O(1)$ intercanvis COST = $O(n^2)$

- En el **pitjor cas**, el vector està inversament ordenat: $\{ 5, 4, 3, 2, 1 \}$
- Farà el mateix que abans però fent els intercanvis.

Worst case = $O(n^2)$ comparacions, $O(n)$ intercanvis COST = $O(n^2)$

Ordenació

(II) Ordenació per Selecció

Anàlisi de costos:

- Cada "passada" consisteix en trobar el mínim ($O(n)$) i intercanviar-lo amb el primer dels no-ordenats.
- En el **millor cas**, el vector ja està ordenat: $\{ 1, 2, 3, 4, 5 \}$
- No haurà de fer cap intercanvi però haurà de buscar tots els mínims.

Best case = $O(n^2)$ comparacions, $O(1)$ intercanvis COST = $O(n^2)$

- En el **pitjor cas**, el vector està inversament ordenat: $\{ 5, 4, 3, 2, 1 \}$
- Farà el mateix que abans però fent els intercanvis.

Worst case = $O(n^2)$ comparacions, $O(n)$ intercanvis COST = $O(n^2)$

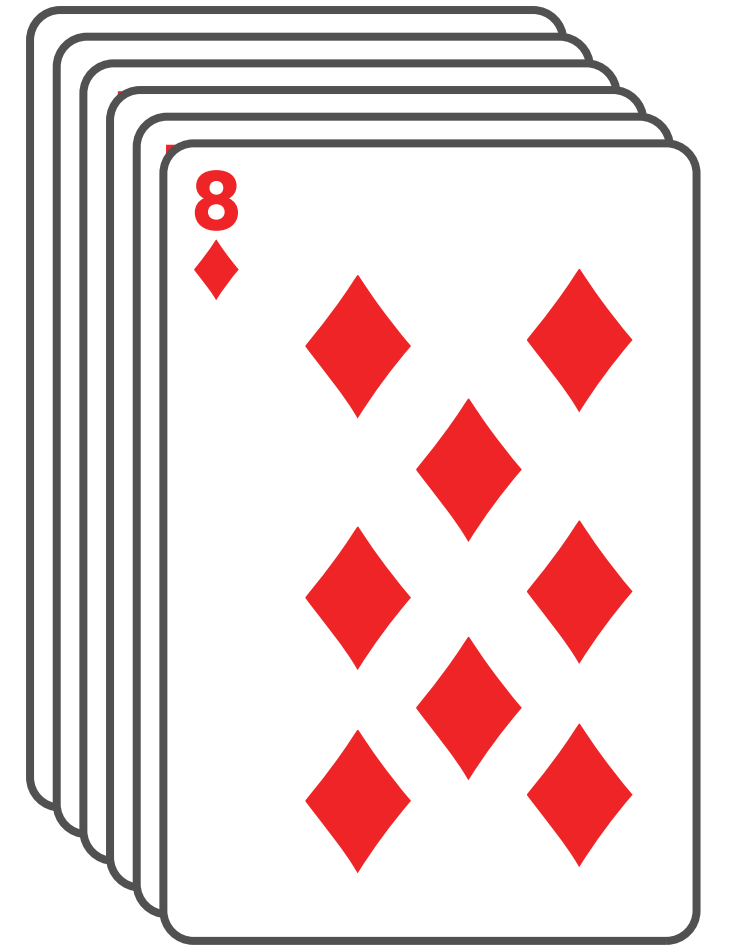
- En el **cas mig**:

Average case = $O(n^2)$ comparacions, $O(n)$ intercanvis COST = $O(n^2)$

Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

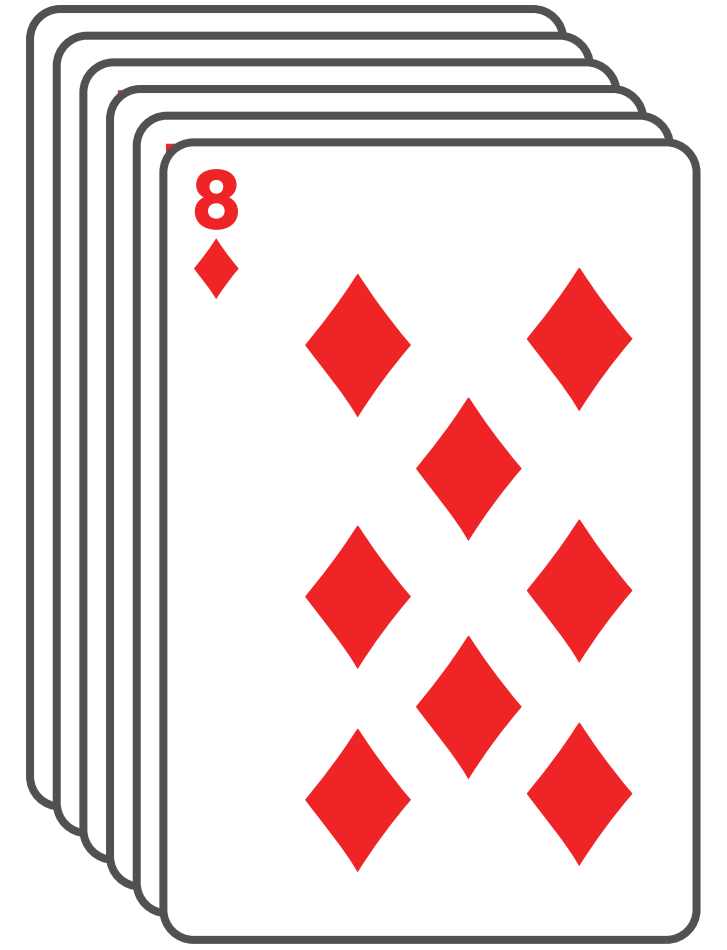


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Versió 2

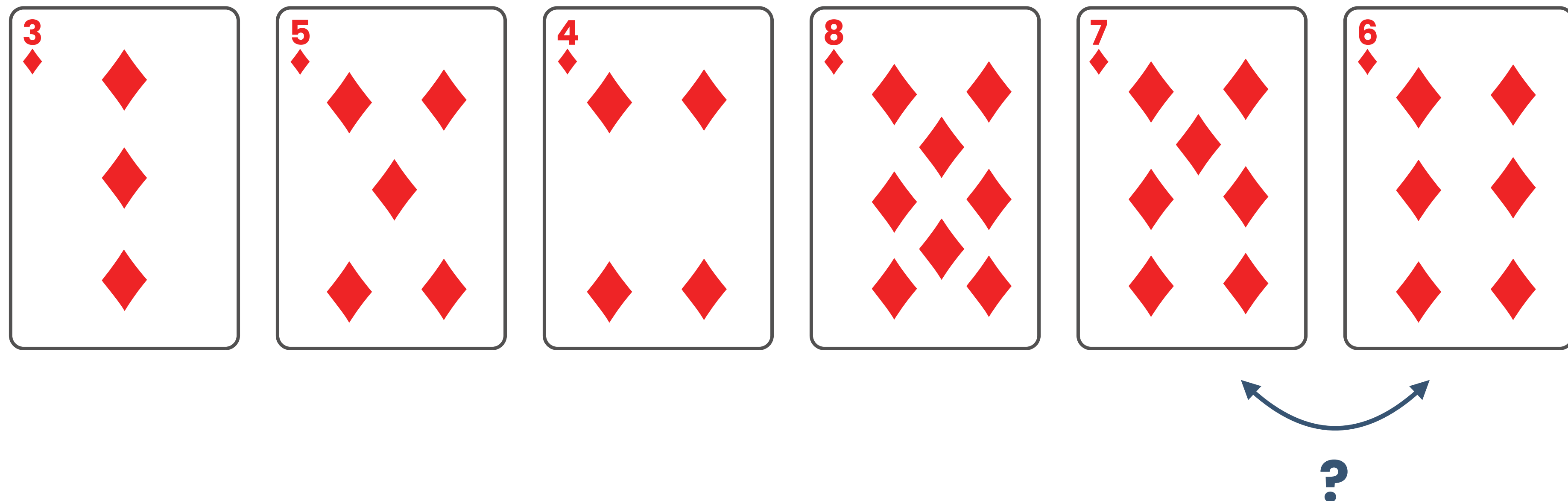


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

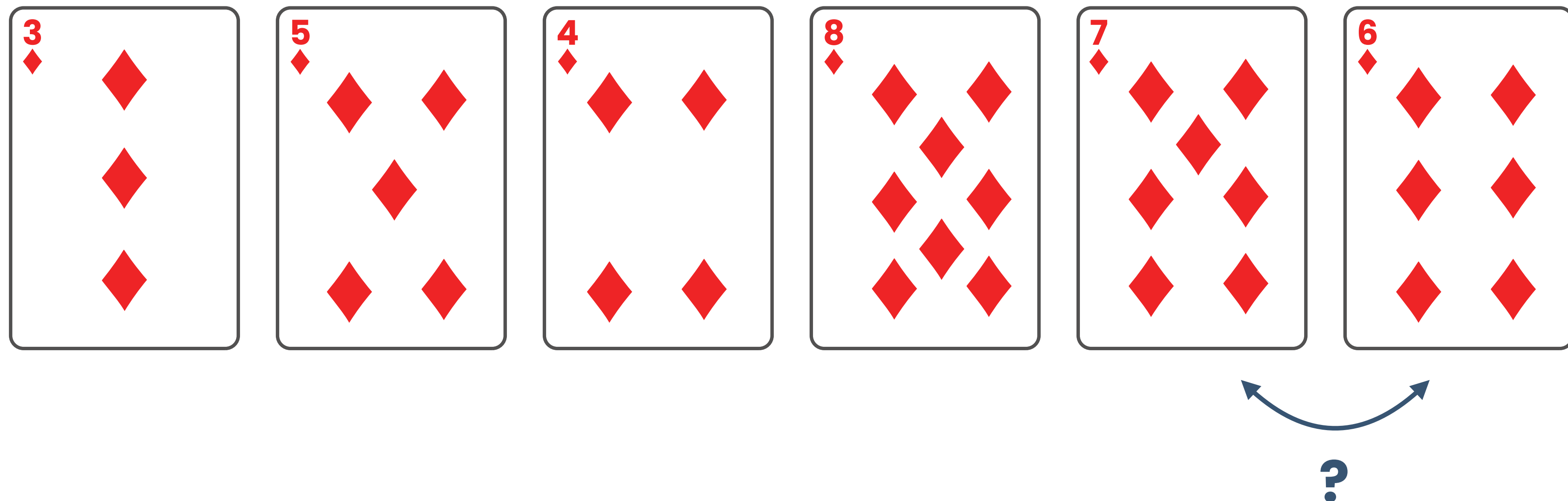


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

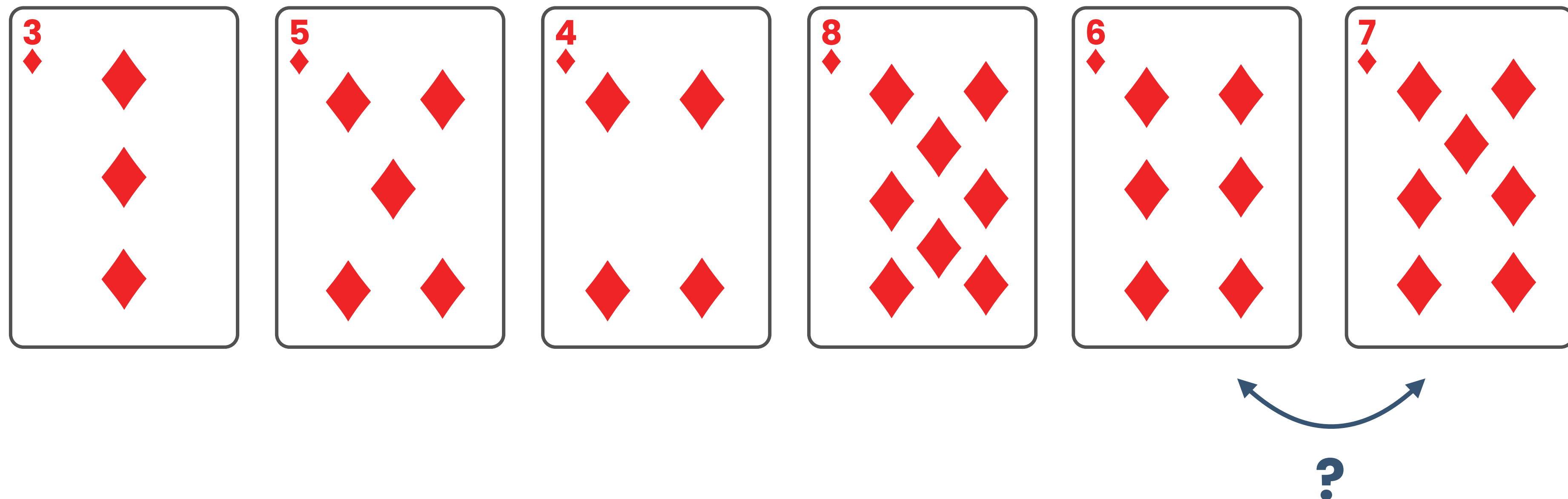


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

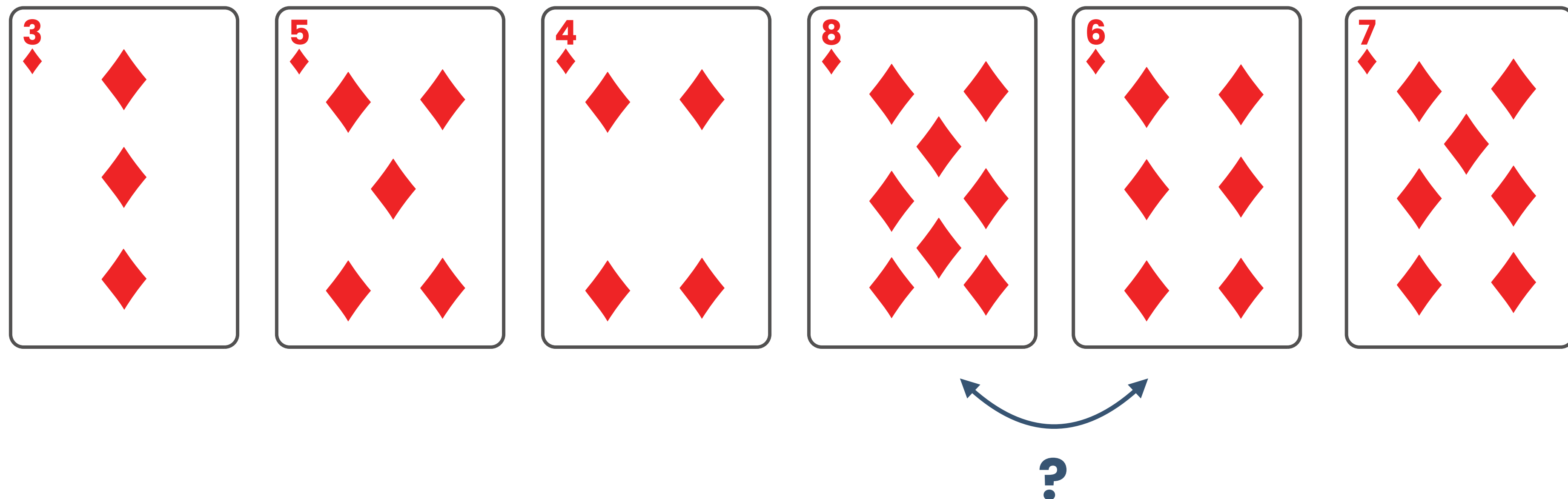


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

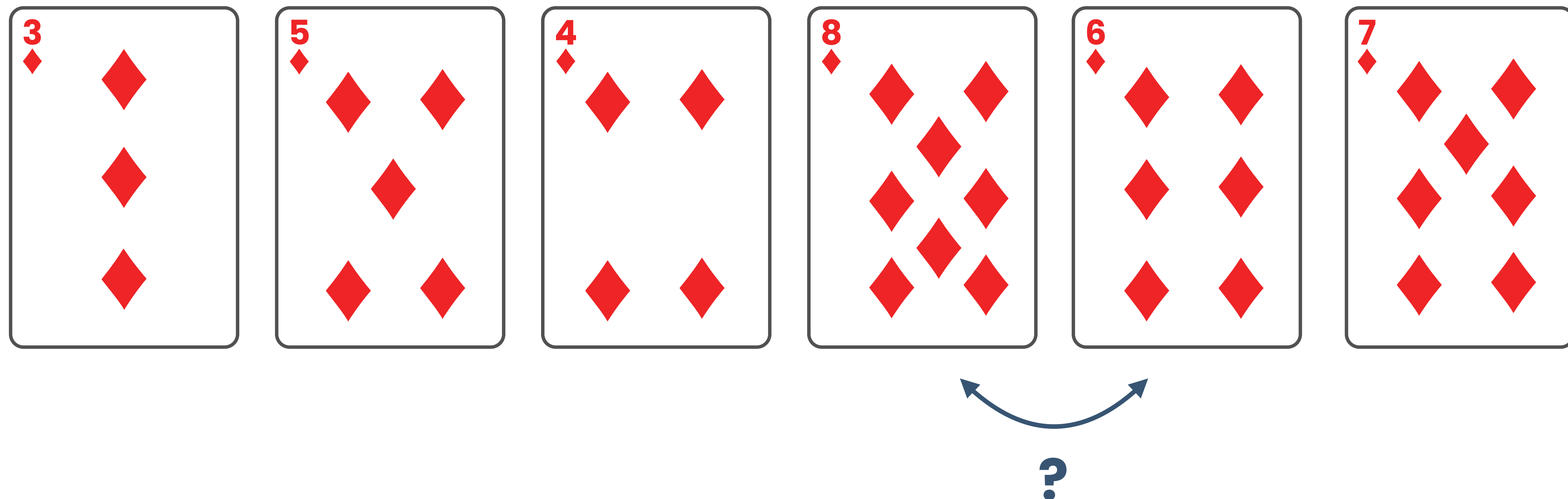


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

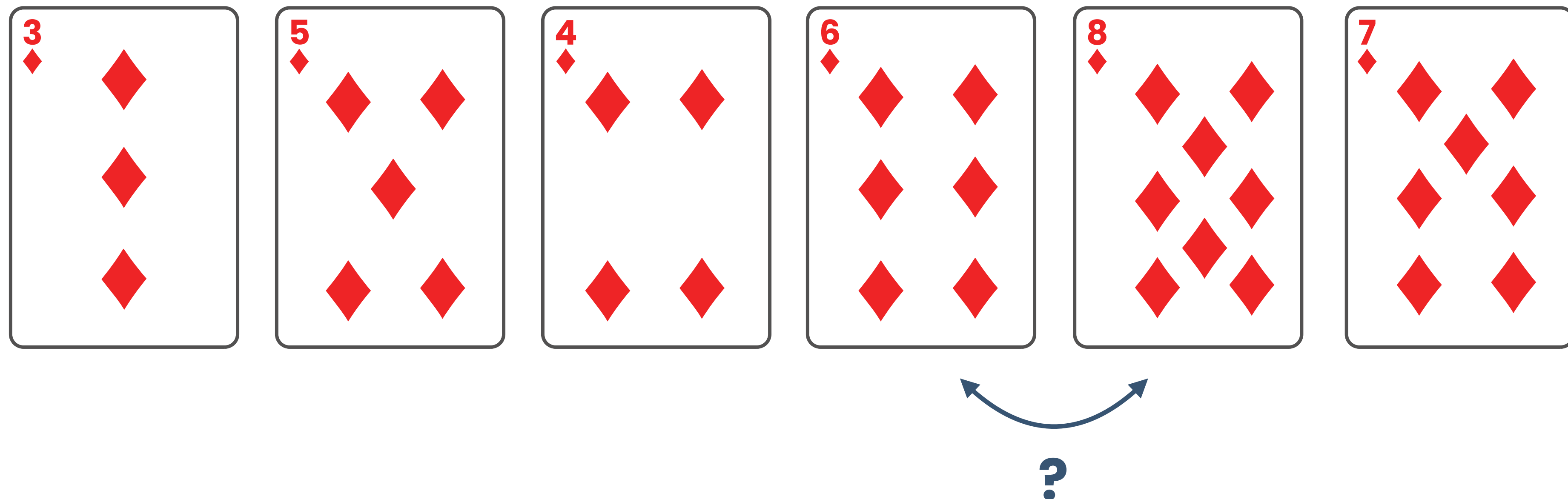


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

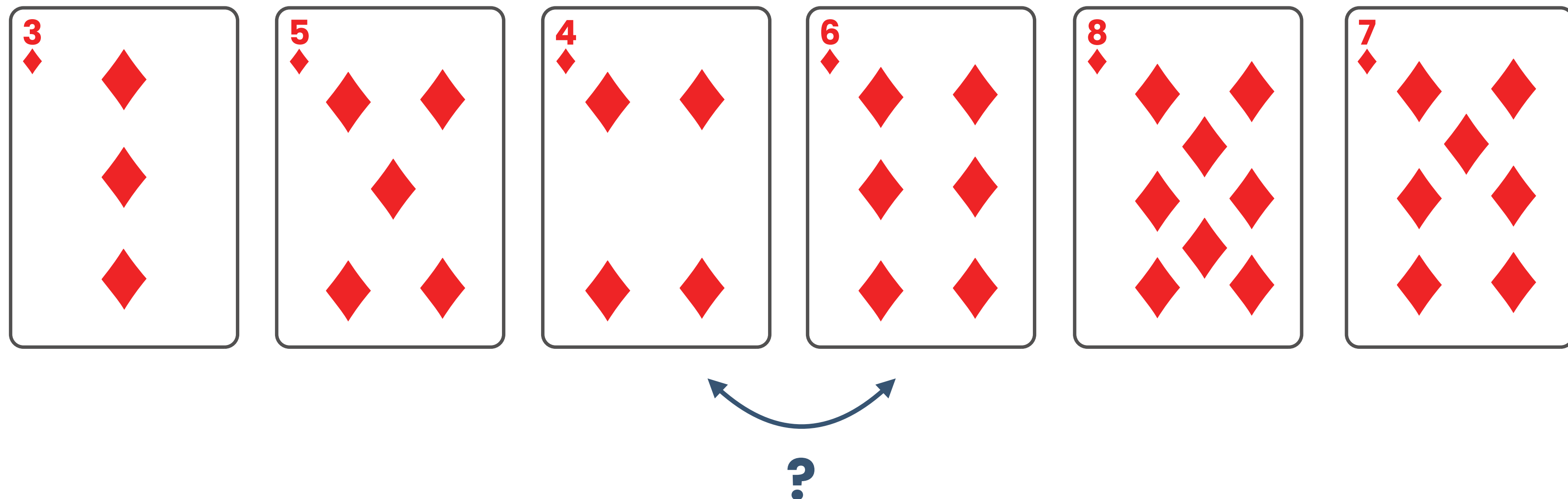


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

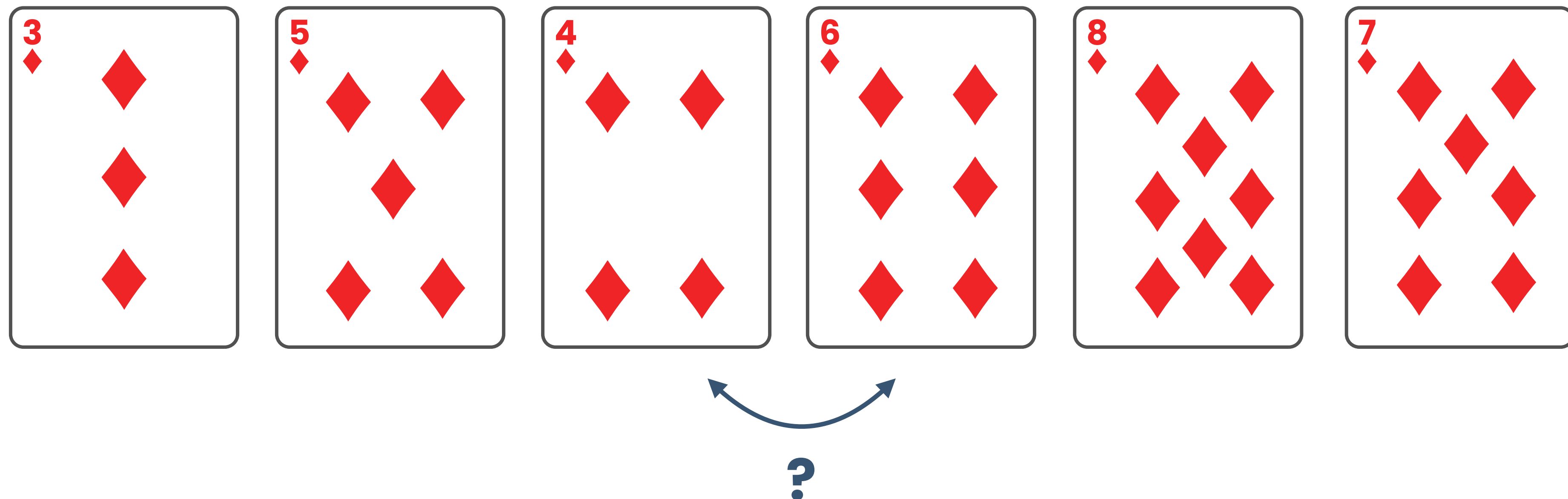


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

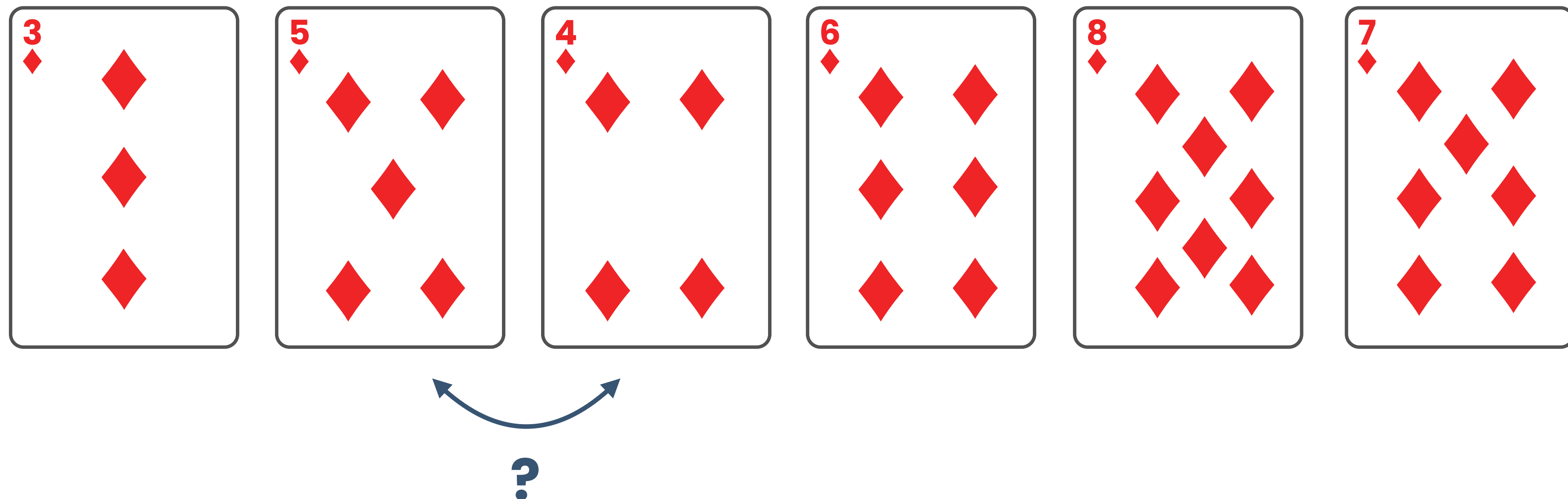


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

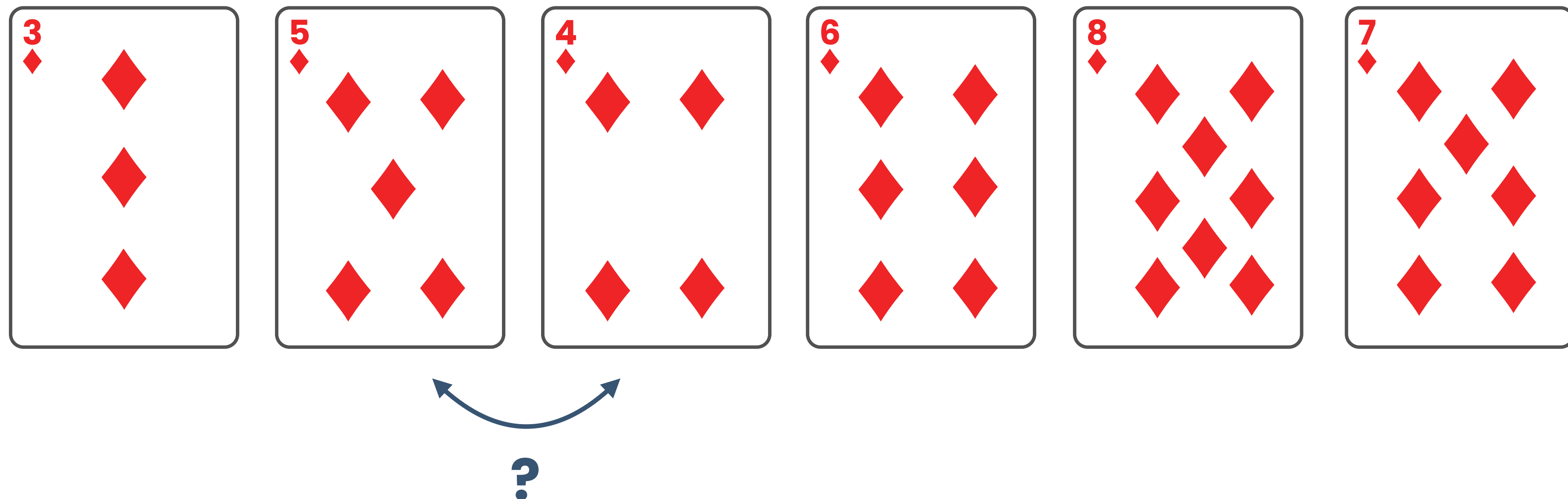


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

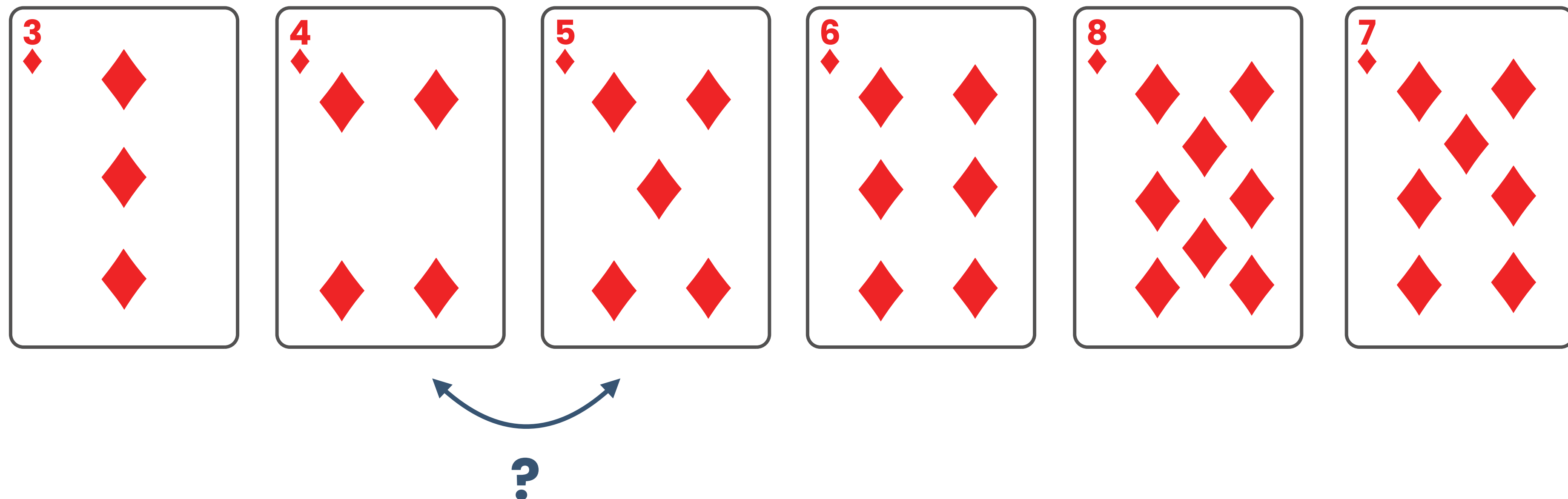


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

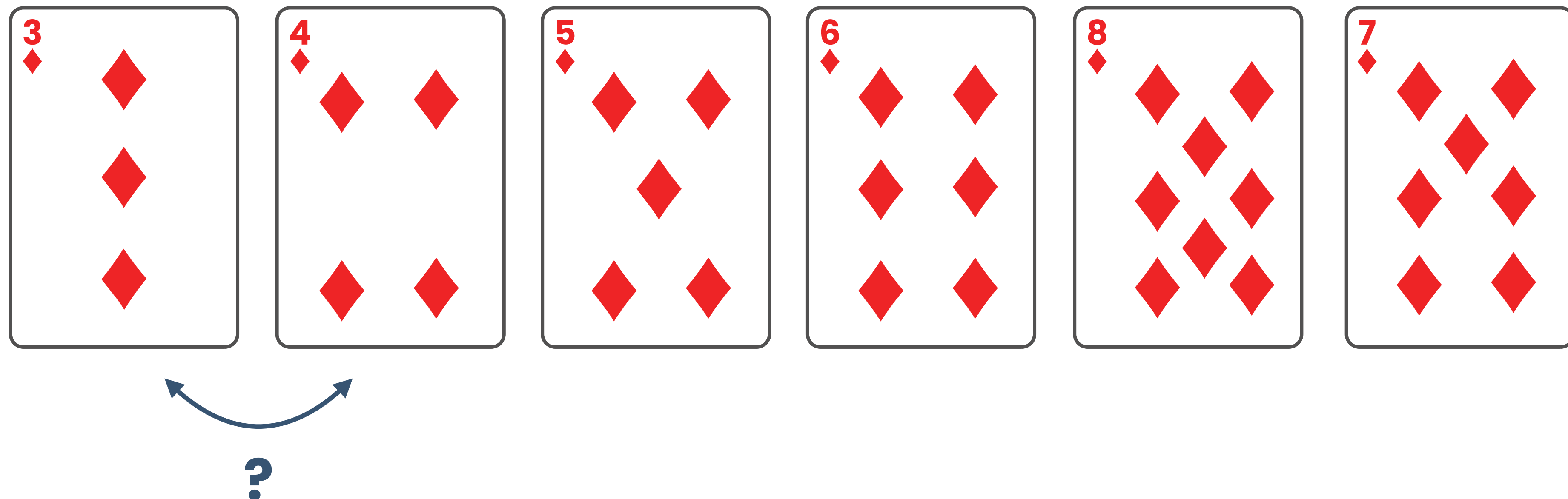


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

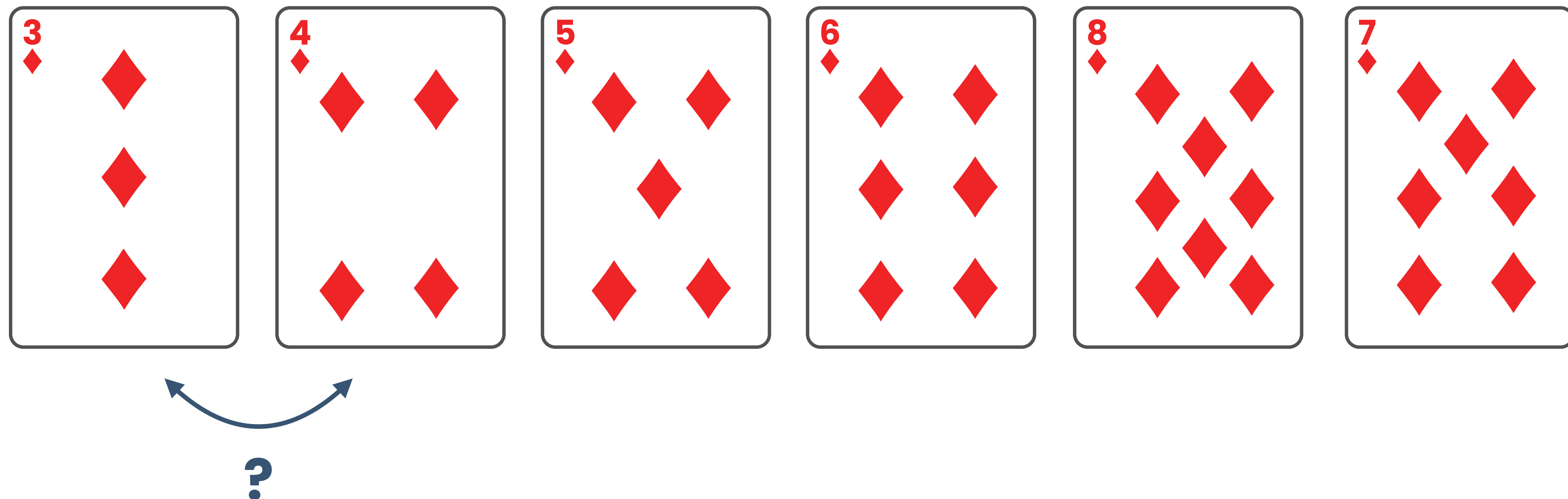


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

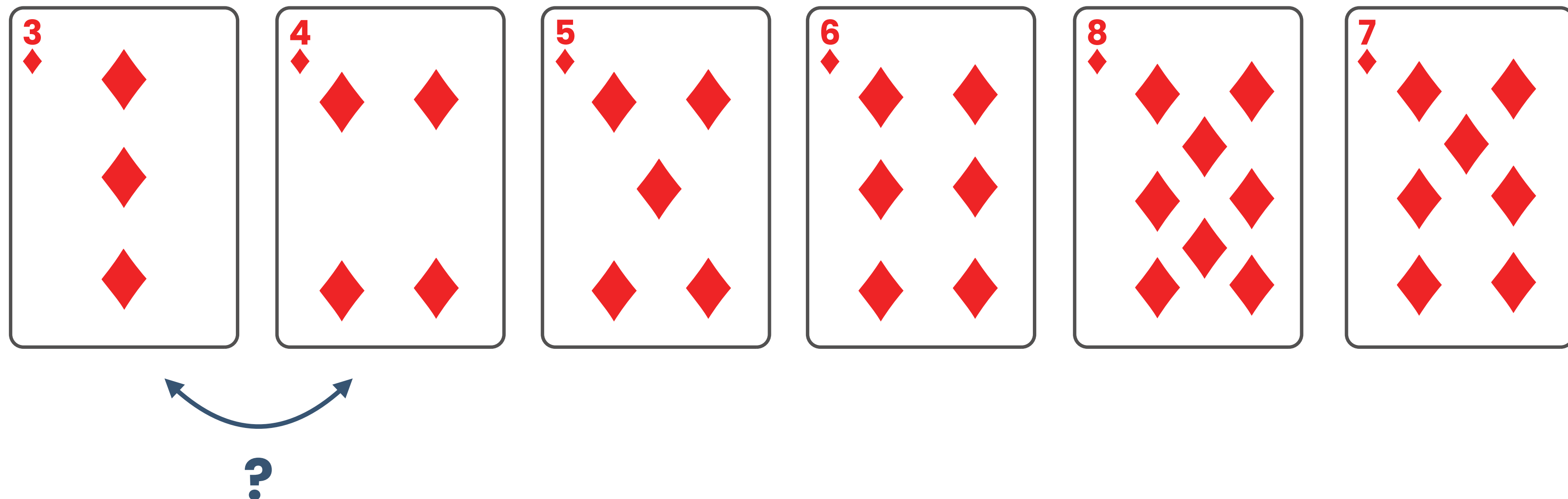


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 1

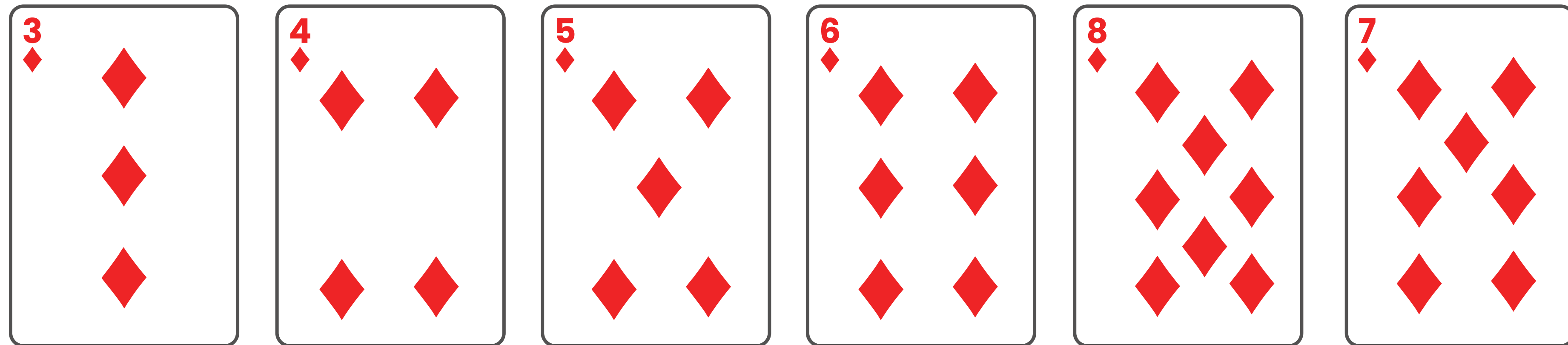


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 2

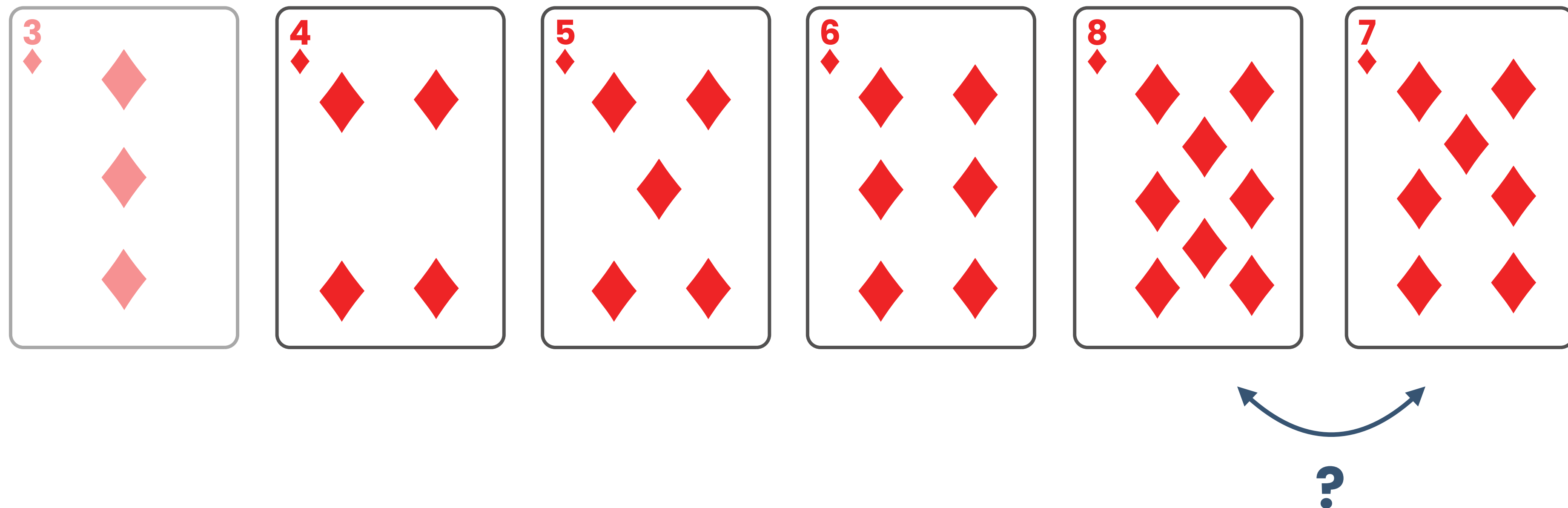


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 2

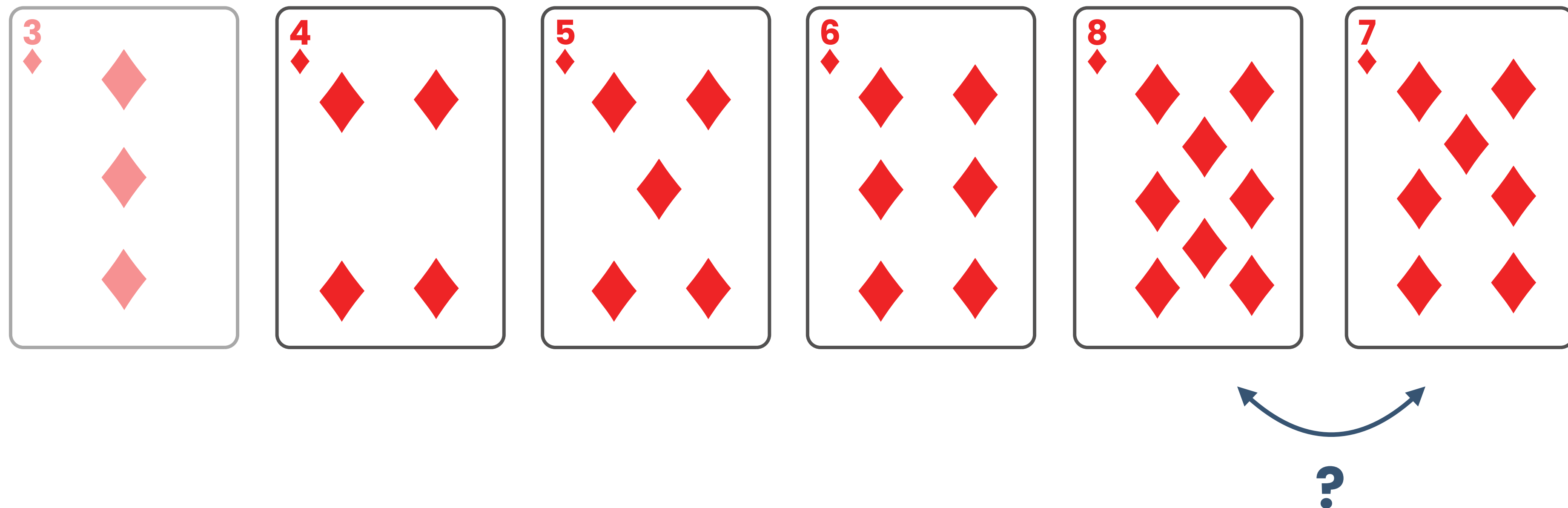


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 2

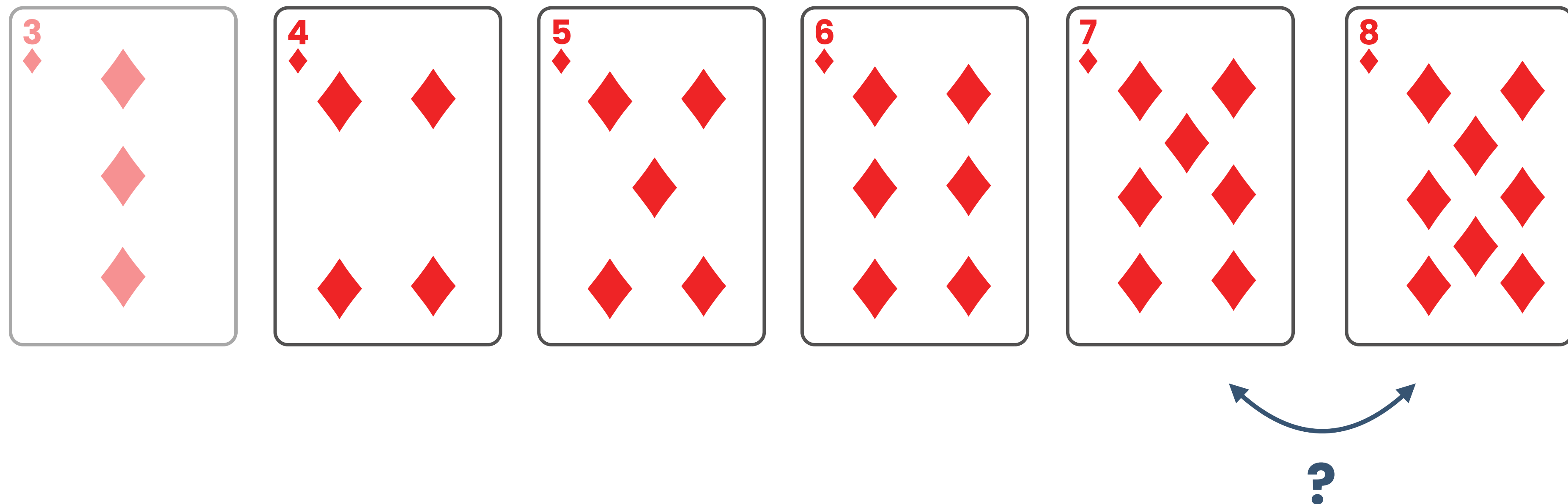


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 2

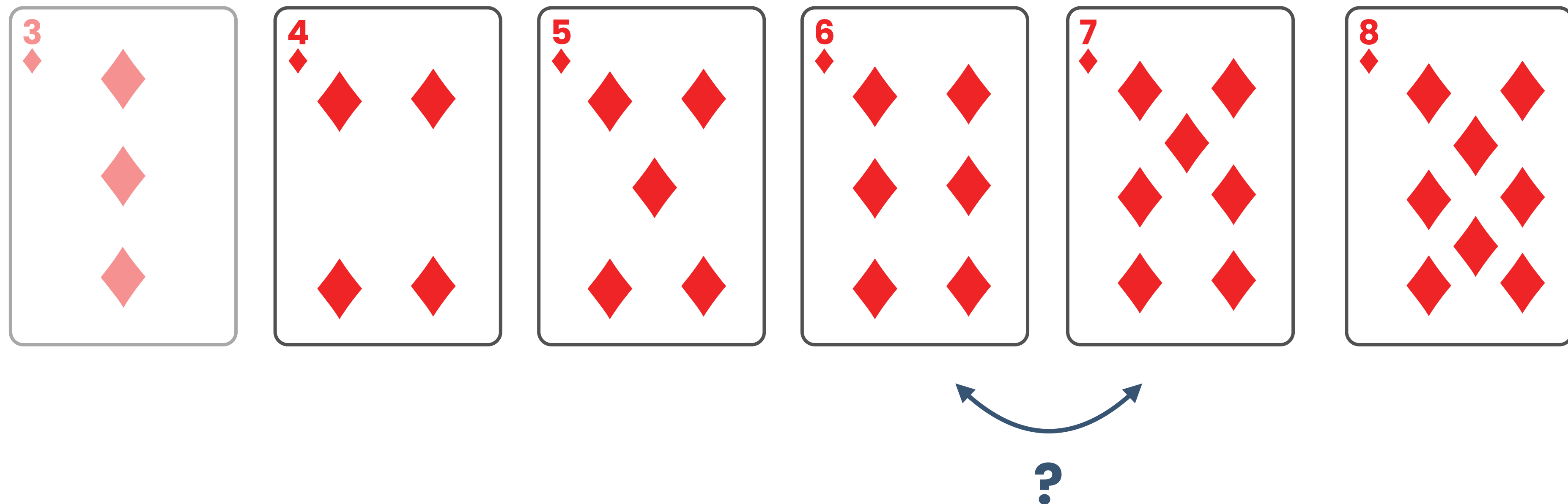


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 2

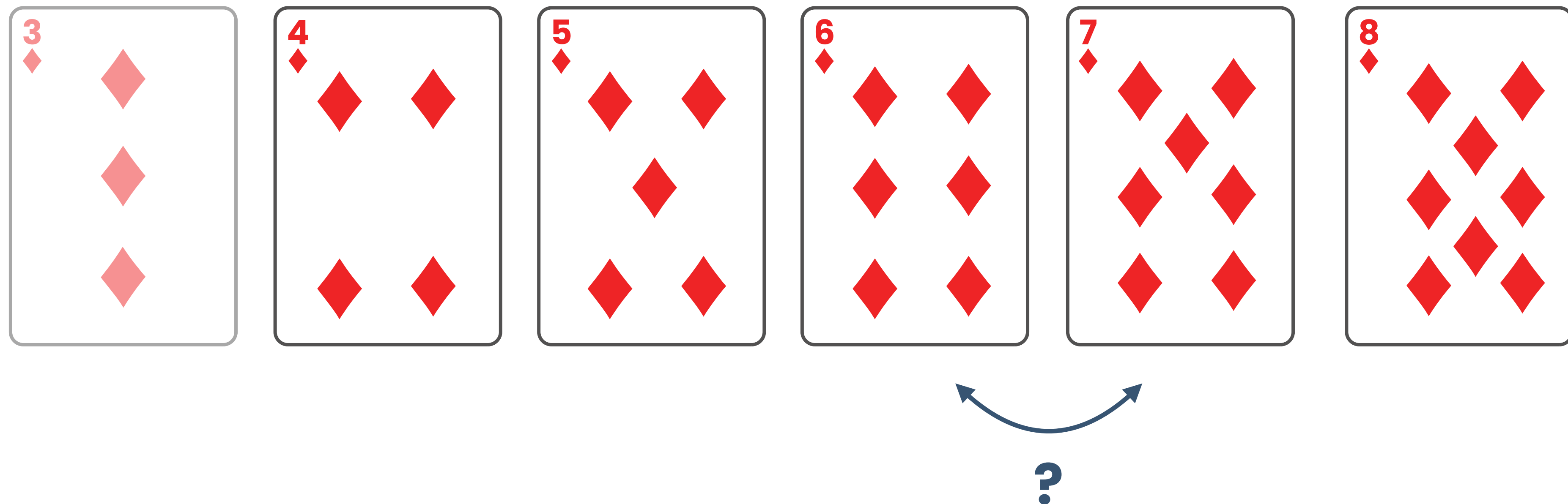


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 2

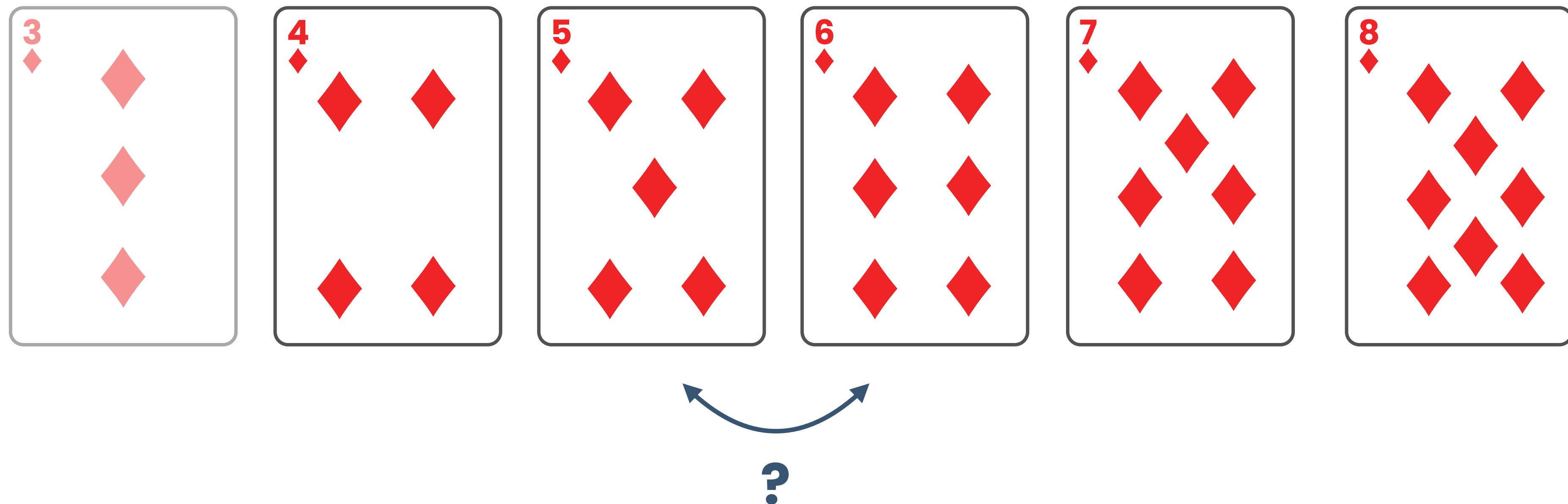


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 2

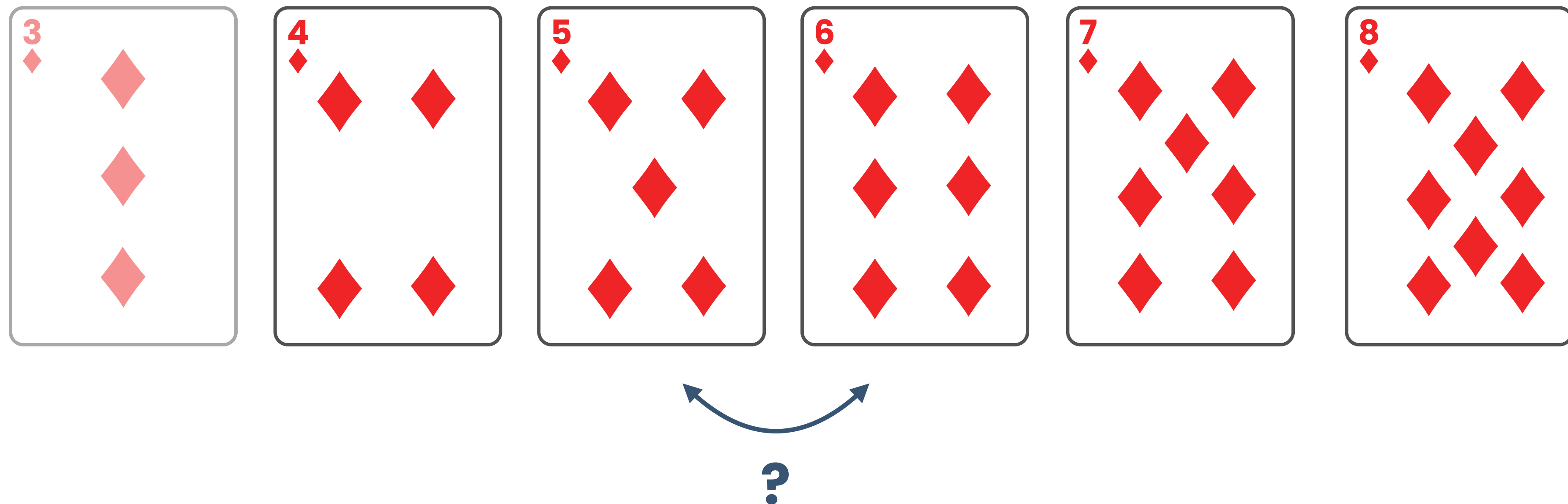


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 2

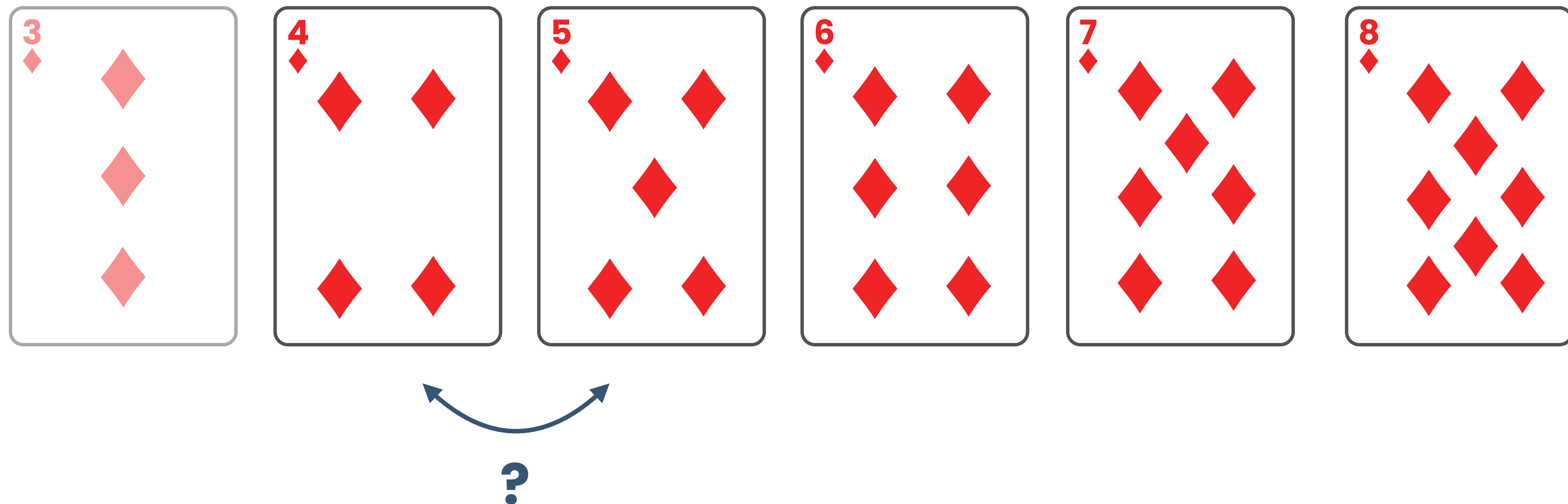


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 2

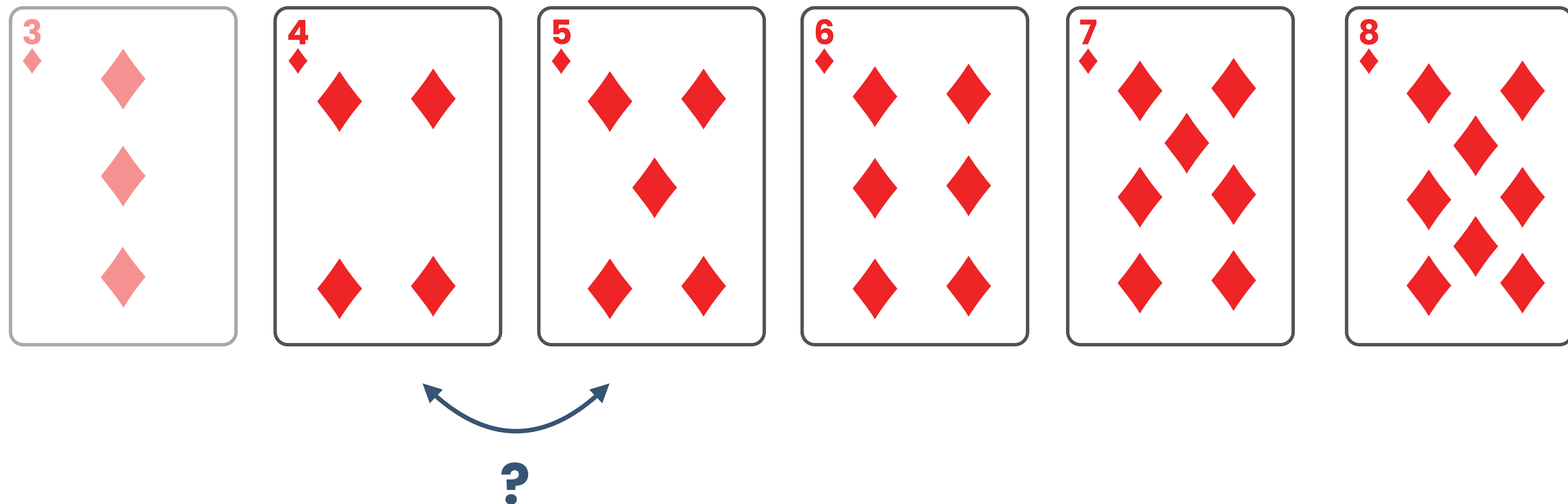


Ordenació

(III) Ordenació per intercanvi (Bombolla)

- Principi: comparar i intercanviar registres adjacents fins que tots estiguin ordenats
- Començant pel final, comparar els registres adjacents per parelles. Si el predecessor és més gran que el successor, intercanviar-los entre ells. Si no, deixar-los com estan.
- Seguir comparant parella a parella fins que arribem al principi de la seqüència.
- Si el vector encara no està ordenat, fer una altra passada igual que l'anterior.

Passada: 2



Ordenació

(III) Ordenació per intercanvi (Bombolla)

```
$ Algorisme del Wirth (p. 71). Versió 1 (la "dolenta")
acció ordenacio_bombolla (v: taula[] d'enter, n_elems:
enter) és
var
  i, j, aux : enter;
fvar
inici
  per (i:=1; i < n_elems; i:=i+1) fer
    per (j:=n_elems-1; j >= i; j:=j-1) fer
      si (v[j-1] > v[j]) llavors
        $ Fem l'intercanvi
        aux := v[j-1];
        v[j-1] := v[j];
        v[j] := aux;
      fsi
    fper
  fper
facció
```


Ordenació

(III) Ordenació per intercanvi (Bombolla)

```
$ Algorisme del Wirth (p. 71). Versió 1 (la "dolenta")
acció ordenacio_bombolla (v: taula[] d'enter, n_elems:
enter) és
var
  i, j, aux : enter;
fvar
inici
  per (i:=1; i < n_elems; i:=i+1) fer
    per (j:=n_elems-1; j >= i; j:=j-1) fer
      si (v[j-1] > v[j]) llavors
        $ Fem l'intercanvi
        aux := v[j-1];
        v[j-1] := v[j];
        v[j] := aux;
      fsi
    fper
  fper
facció
```



Problema greu d'eficiència:

Passada=1	[	3	5	4	8	6	7	]	
Passada=1	[	3	5	4	6	8	7	]	
Passada=1	[	3	5	4	6	8	7	]	
Passada=1	[	3	4	5	6	8	7	]	
Passada=1	[	3	4	5	6	8	7	]	
Passada=2	[	3	4	5	6	7	8	]	*
Passada=2	[	3	4	5	6	7	8	]	
Passada=2	[	3	4	5	6	7	8	]	
Passada=2	[	3	4	5	6	7	8	]	
Passada=3	[	3	4	5	6	7	8	]	
Passada=3	[	3	4	5	6	7	8	]	
Passada=3	[	3	4	5	6	7	8	]	
Passada=4	[	3	4	5	6	7	8	]	
Passada=4	[	3	4	5	6	7	8	]	
Passada=5	[	3	4	5	6	7	8	]	

En aquest cas particular, el vector ja està ordenat amb la segona passada.

La 3, 4, i 5 no calen!

Ordenació

(III) Ordenació per intercanvi (Bombolla)

Versió 2:

Una técnica obvia para mejorar este algoritmo es controlar si se ha producido algun cambio en una pasada. Es necesaria, por tanto, una última pasada sin operaciones de intercambio para determinar que el algoritmo puede acabar. - Wirth

```
$ Algorisme del Wirth (p. 71) Adaptat per versió "bona"
acció ordenacio_bombolla_v2 (v: taula[] d'enter, n_elems: enter) és
var
  i, j, aux : enter;
  ja_ordenat : booleà;
fvar
inici
  i := 1;
  ja_ordenat := fals;
mentre (i < n_elems i no(ja_ordenat)) fer
  ja_ordenat := cert;
  per (j=n_elems-1; j >= i; j:=j-1) fer
    si (v[j-1] > v[j]) llavors
      $ Fem l'intercanvi
      aux := v[j-1];
      v[j-1] := v[j];
      v[j] := aux;
      ja_ordenat := fals;
    fsi
  fper
  i:=i+1;
fmentre
facció
```

Ordenació

(III) Ordenació per intercanvi (Bombolla)

Anàlisi de costos:

- Aquests costos corresponen a la versió "bona", la que ja comprova si està ordenat

Ordenació

(III) Ordenació per intercanvi (Bombolla)

Anàlisi de costos:

- Aquests costos corresponen a la versió "bona", la que ja comprova si està ordenat
- En el **millor cas**, el vector ja està ordenat: { 1, 2, 3, 4, 5 }
- Haurà de fer una passada per comprovar que està ordenat

Best case = $O(n)$

Ordenació

(III) Ordenació per intercanvi (Bombolla)

Anàlisi de costos:

- Aquests costos corresponen a la versió "bona", la que ja comprova si està ordenat
- En el **millor cas**, el vector ja està ordenat: { 1, 2, 3, 4, 5 }
- Haurà de fer una passada per comprovar que està ordenat

Best case = $O(n)$

- En el **pitjor cas**, el vector està inversament ordenat: { 5, 4, 3, 2, 1 }
- Farà N passades, i cada passada comprova N elements

Worst case = $O(n^2)$

Ordenació

(III) Ordenació per intercanvi (Bombolla)

Anàlisi de costos:

- Aquests costos corresponen a la versió "bona", la que ja comprova si està ordenat
- En el **millor cas**, el vector ja està ordenat: { 1, 2, 3, 4, 5 }
- Haurà de fer una passada per comprovar que està ordenat

Best case = $O(n)$

- En el **pitjor cas**, el vector està inversament ordenat: { 5, 4, 3, 2, 1 }
- Farà N passades, i cada passada comprova N elements

Worst case = $O(n^2)$

- En el **cas mig**:

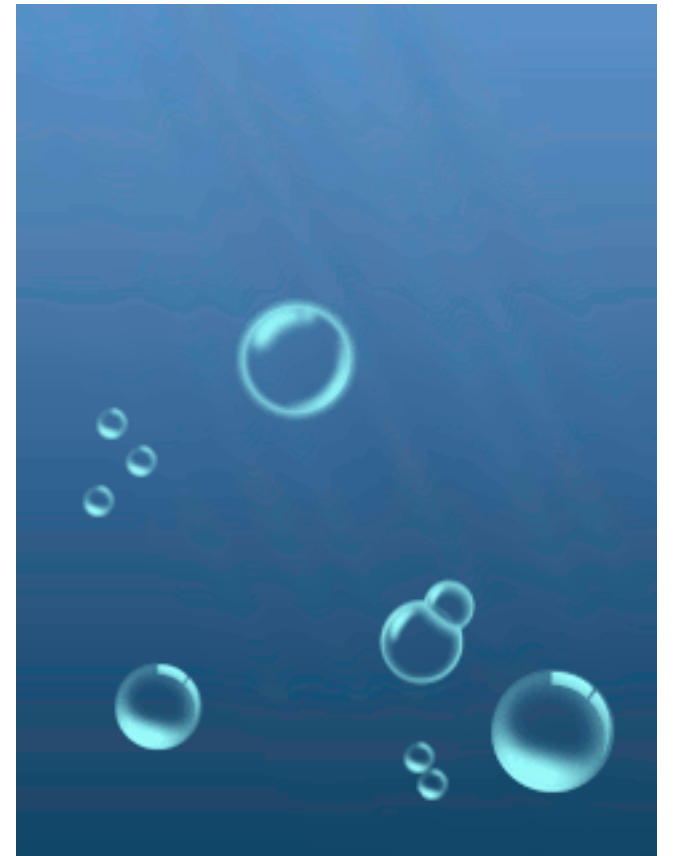
Average case = $O(n^2)$

Ordenació

(III) Ordenació per intercanvi (Bombolla)



- Nombres petits situats al final arriben al seu lloc en una sola passada, mentre que nombres grans situats al principi els hi costa diverses passades arribar al final (es mouen una posició amunt per cada passada).

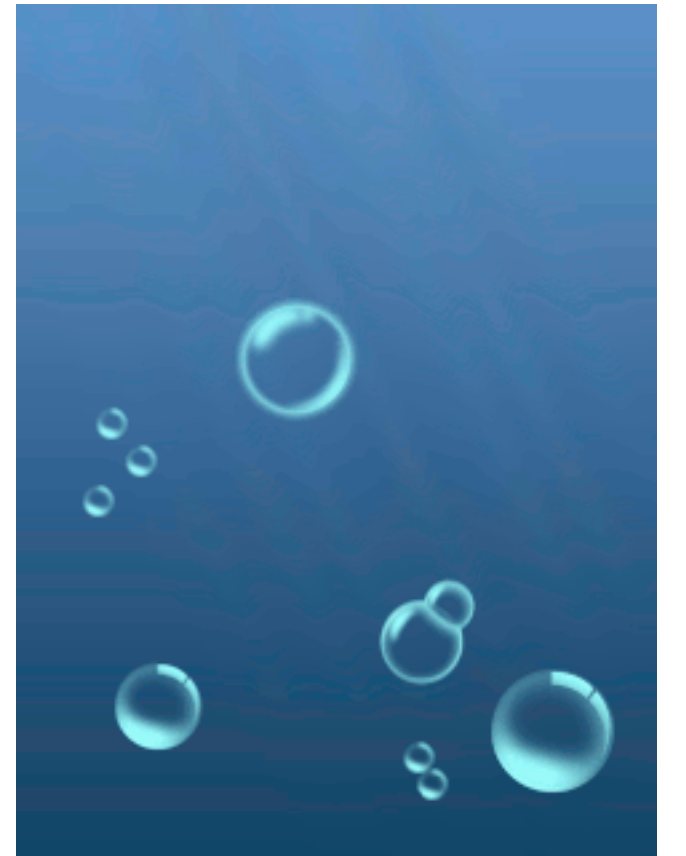
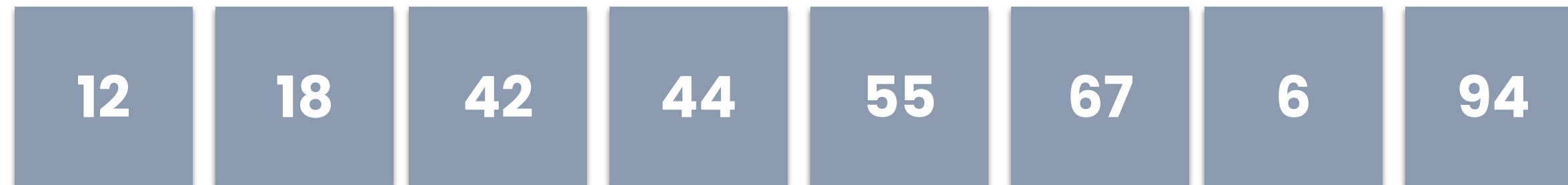


Ordenació

(III) Ordenació per intercanvi (Bombolla)



- Nombres petits situats al final arriben al seu lloc en una sola passada, mentre que nombres grans situats al principi els hi costa diverses passades arribar al final (es mouen una posició amunt per cada passada).
- Cas "nombre lleuger a la dreta" (el 6)



Ordenació

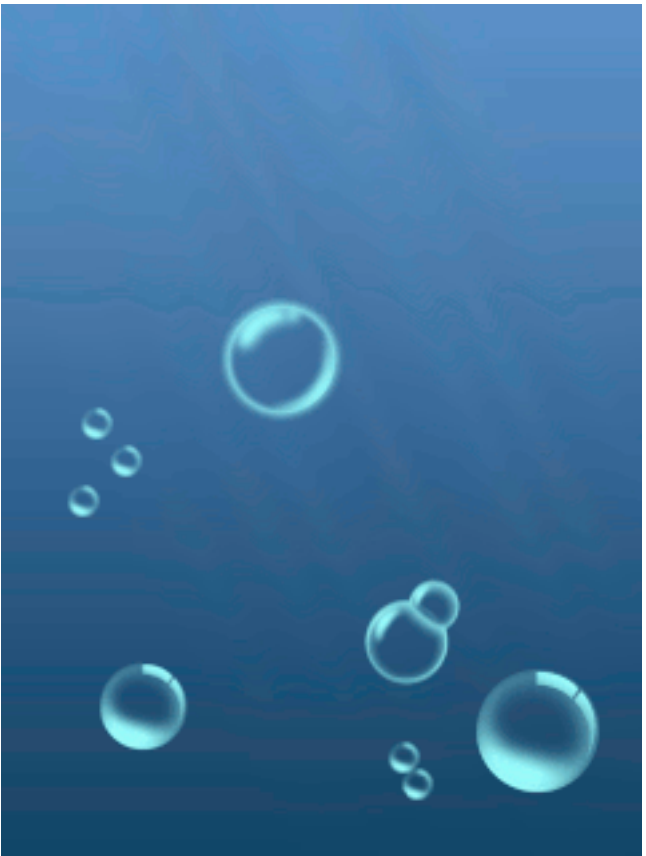
(III) Ordenació per intercanvi (Bombolla)



- Nombres petits situats al final arriben al seu lloc en una sola passada, mentre que nombres grans situats al principi els hi costa diverses passades arribar al final (es mouen una posició amunt per cada passada).
- Cas "nombre lleuger a la dreta" (el 6)

12	18	42	44	55	67	6	94
----	----	----	----	----	----	---	----

Passada=1	[	12	18	42	44	55	67	6	94	]
Passada=1	[	12	18	42	44	55	6	67	94	]
Passada=1	[	12	18	42	44	6	55	67	94	]
Passada=1	[	12	18	42	6	44	55	67	94	]
Passada=1	[	12	18	6	42	44	55	67	94	]
Passada=1	[	12	6	18	42	44	55	67	94	]
Passada=1	[	6	12	18	42	44	55	67	94	]
*										



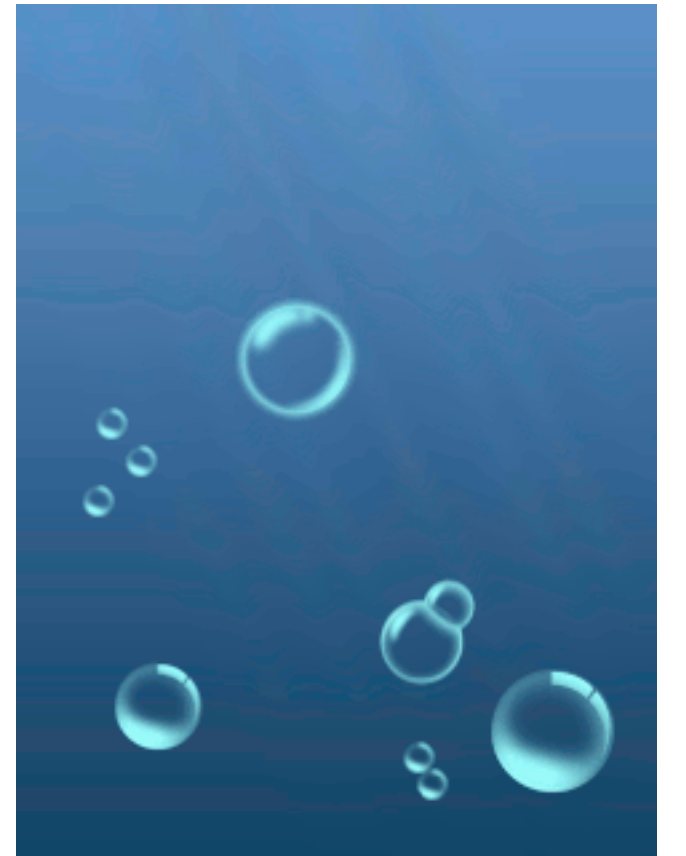
Ordenació

(III) Ordenació per intercanvi (Bombolla)



- Nombres petits situats al final arriben al seu lloc en una sola passada, mentre que nombres grans situats al principi els hi costa diverses passades arribar al final (es mouen una posició amunt per cada passada).
- Cas "nombre pesat a l'esquerra" (el 94)

94	6	12	18	42	44	55	67
----	---	----	----	----	----	----	----



Ordenació

(III) Ordenació per intercanvi (Bombolla)

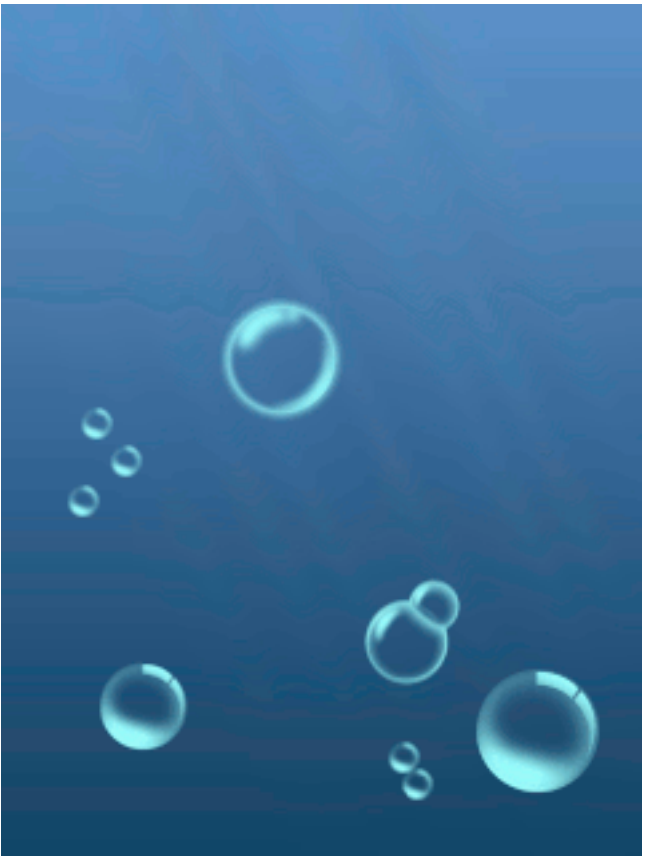


- Nombres petits situats al final arriben al seu lloc en una sola passada, mentre que nombres grans situats al principi els hi costa diverses passades arribar al final (es mouen una posició amunt per cada passada).

- Cas "nombre pesat a l'esquerra" (el 94)

94	6	12	18	42	44	55	67
----	---	----	----	----	----	----	----

Passada=1	[	94	6	12	18	42	44	55	67	]		Passada=3	[	6	12	94	18	42	44	55	67	]	
Passada=1	[	94	6	12	18	42	44	55	67	]		Passada=3	[	6	12	94	18	42	44	55	67	]	
Passada=1	[	94	6	12	18	42	44	55	67	]		Passada=3	[	6	12	94	18	42	44	55	67	]	
Passada=1	[	94	6	12	18	42	44	55	67	]		Passada=3	[	6	12	94	18	42	44	55	67	]	
Passada=1	[	94	6	12	18	42	44	55	67	]		Passada=3	[	6	12	18	94	42	44	55	67	]	
Passada=1	[	94	6	12	18	42	44	55	67	]		Passada=4	[	6	12	18	94	42	44	55	67	]	
Passada=1	[	6	94	12	18	42	44	55	67	]		Passada=4	[	6	12	18	94	42	44	55	67	]	
Passada=2	[	6	94	12	18	42	44	55	67	]		Passada=4	[	6	12	18	94	42	44	55	67	]	
Passada=2	[	6	94	12	18	42	44	55	67	]		Passada=4	[	6	12	18	42	94	44	55	67	]	
Passada=2	[	6	94	12	18	42	44	55	67	]		Passada=5	[	6	12	18	42	94	44	55	67	]	
Passada=2	[	6	94	12	18	42	44	55	67	]		Passada=5	[	6	12	18	42	94	44	55	67	]	
Passada=2	[	6	94	12	18	42	44	55	67	]		Passada=5	[	6	12	18	42	44	94	55	67	]	
Passada=2	[	6	12	94	18	42	44	55	67	]		Passada=6	[	6	12	18	42	44	94	55	67	]	
												Passada=6	[	6	12	18	42	44	55	94	67	]	
												Passada=7	[	6	12	18	42	44	55	67	94	]	*



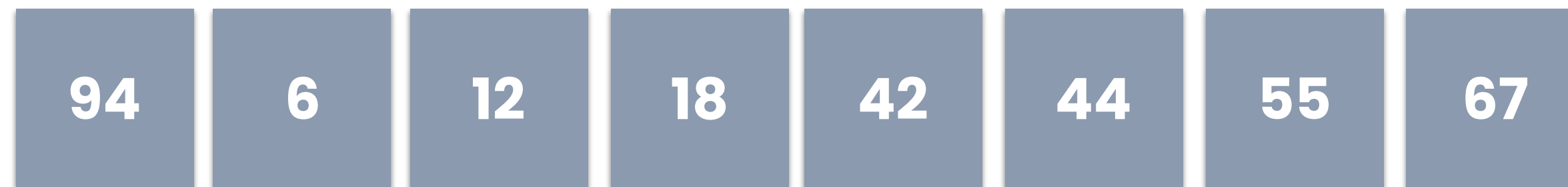
Ordenació

(III) Ordenació per intercanvi (Bombolla)

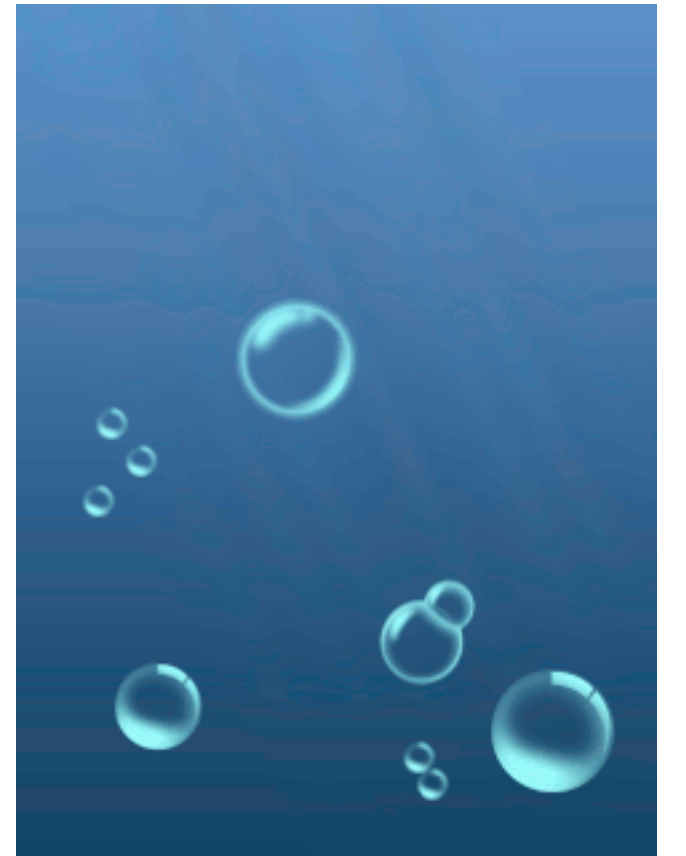


- Nombres petits situats al final arriben al seu lloc en una sola passada, mentre que nombres grans situats al principi els hi costa diverses passades arribar al final (es mouen una posició amunt per cada passada).

- Cas "nombre pesat a l'esquerra" (el 94)



- Variacions:
 - Mètode que alterna la direcció de les comparacions (una passada es fa de dreta a esquerra, la següent d'esquerra a dreta... etc)





TEORIA

PROGRAMACIÓ CIENTÍFICA

T10-Apèndix
Quicksort

Altres algorismes d'ordenació

Per saber-ne més

- Els anteriors algorismes no són els més eficients, però són la base de moltes variacions posteriors.
- Altres categories d'algorismes d'ordenació: algorismes recursius (que es criden a sí mateixos), basats en el principi de "divideix i venceràs": dividir el problema en subproblemes que es resolen de la mateixa manera. (Similar a com ho féiem a l'algorisme de cerca binària). Aquests tenen els costos més baixos possibles: $O(n \log n)$
 - MergeSort: https://en.wikipedia.org/wiki/Merge_sort
 - QuickSort: <https://en.wikipedia.org/wiki/Quicksort>

6 5 3 1 8 7 2 4

Altres algorismes d'ordenació

Per saber-ne més

- Els anteriors algorismes no són els més eficients, però són la base de moltes variacions posteriors.
- Altres categories d'algorismes d'ordenació: algorismes recursius (que es criden a sí mateixos), basats en el principi de "divideix i venceràs": dividir el problema en subproblemes que es resolen de la mateixa manera. (Similar a com ho féiem a l'algorisme de cerca binària). Aquests tenen els costos més baixos possibles: $O(n \log n)$
 - MergeSort: https://en.wikipedia.org/wiki/Merge_sort
 - QuickSort: <https://en.wikipedia.org/wiki/Quicksort>
- Per veure simulacions d'alguns algorismes d'ordenació:
 - <https://www.toptal.com/developers/sorting-algorithms>
 - <https://visualgo.net/en/sorting>



6 5 3 1 8 7 2 4

Quicksort

- Quicksort és un algorisme d'ordenació creat per Tony Hoare al 1959 i publicat el 1961 que, a dia d'avui, és un dels algorismes d'ordenació més utilitzats.
- És un algorisme de tipus “divideix i venceràs” (aquesta filosofia es basa en dividir el problema en trossos més petits que es puguin solucionar) i és un algorisme **recursiu**.

Quicksort

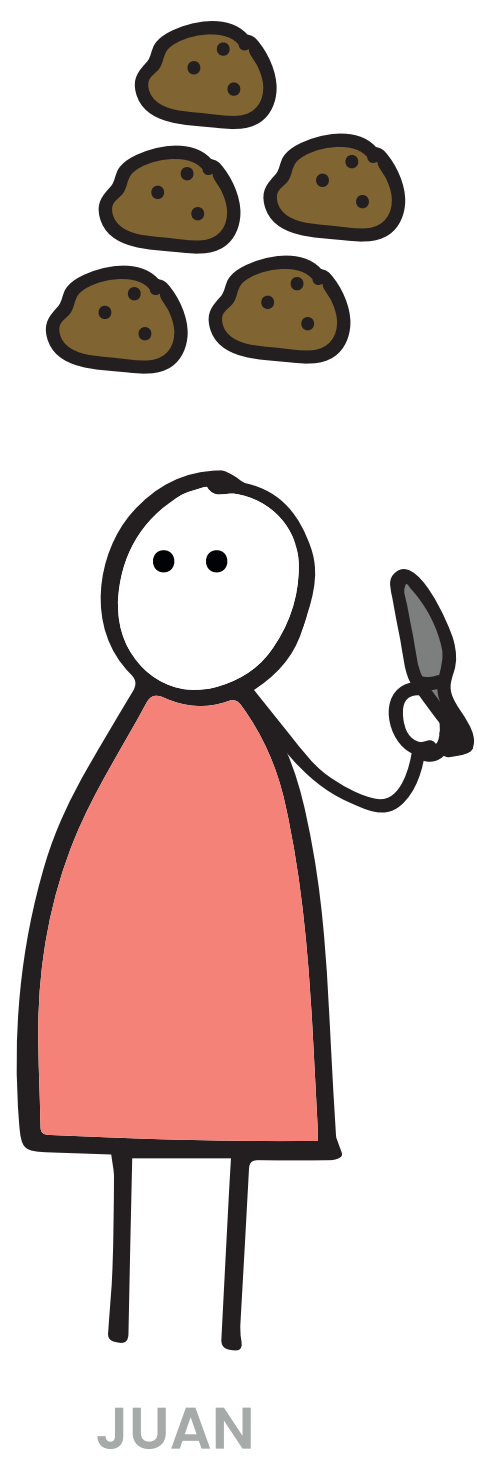
- Quicksort és un algorisme d'ordenació creat per Tony Hoare al 1959 i publicat el 1961 que, a dia d'avui, és un dels algorismes d'ordenació més utilitzats.
- És un algorisme de tipus “divideix i venceràs” (aquesta filosofia es basa en dividir el problema en trossos més petits que es puguin solucionar) i és un algorisme **recursiu**.

Algorismes recursius

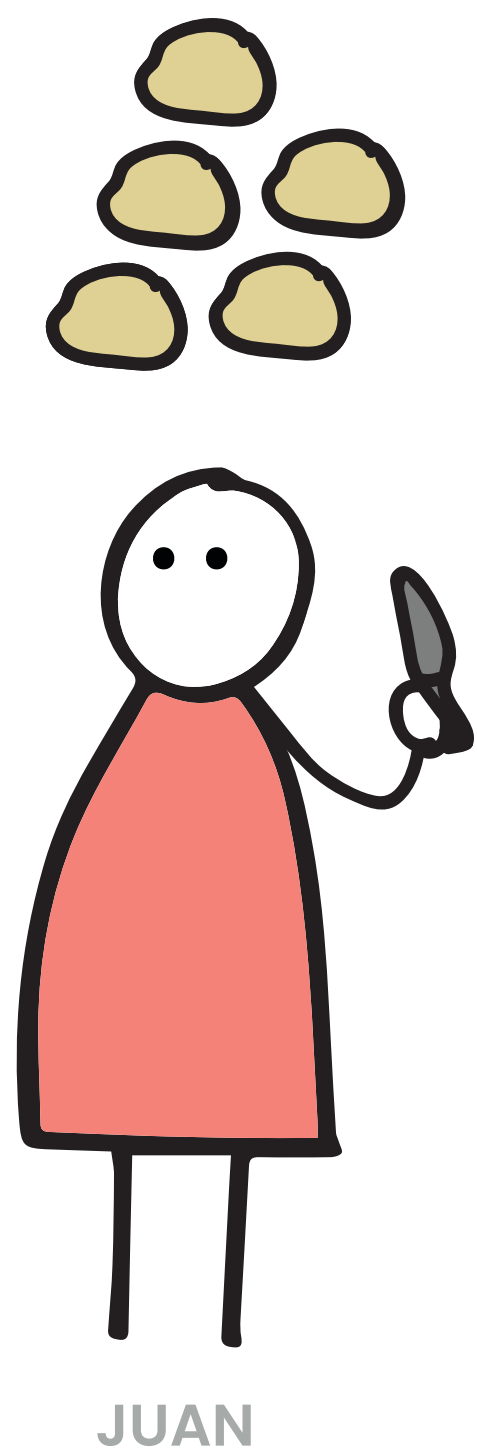
- La recursió és una manera de solucionar un problema on la solució depèn de solucions d'instàncies més petites del mateix problema que se solucionen igual.
- Aquest tipus de problemes a vegades també es poden solucionar iterativament, però les solucions recursives acostumen a ser més senzilles.
- La manera d'implementar solucions recursives és fer servir funcions que es criden a si mateixes dins la pròpia funció.



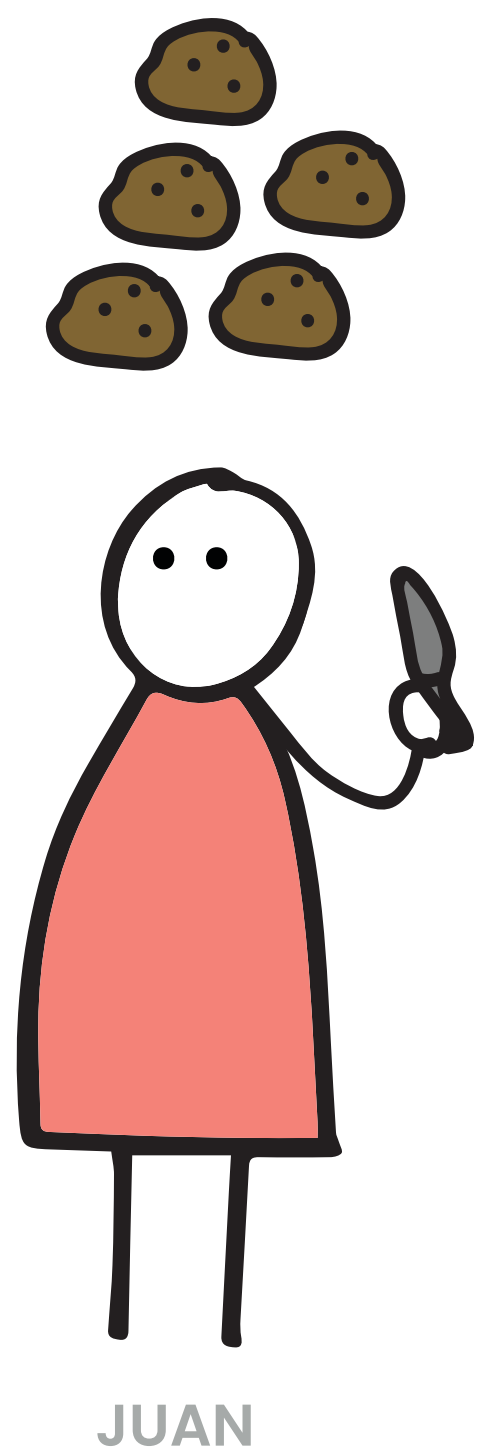
Versió iterativa



Versió iterativa



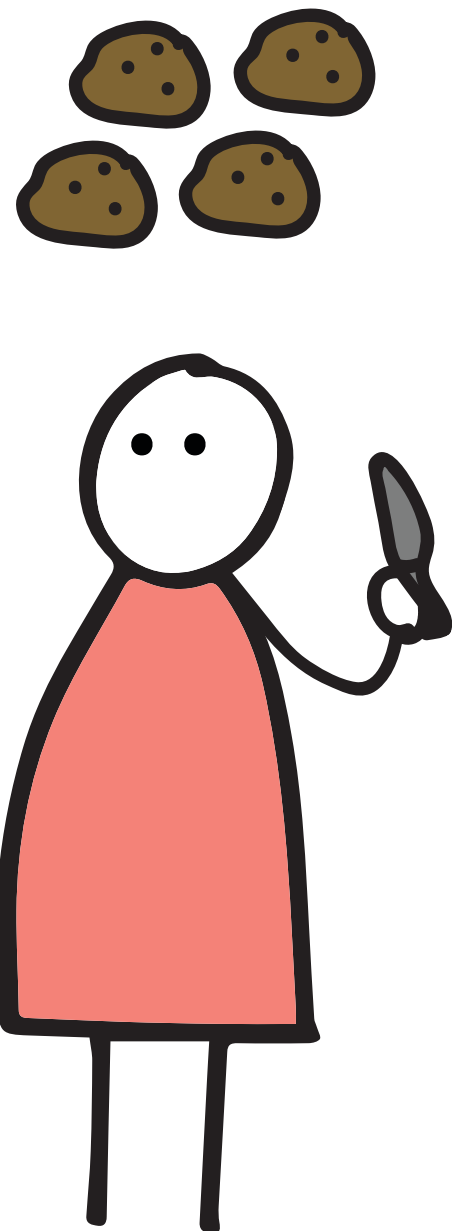
Versió recursiva



Versió recursiva



JUAN

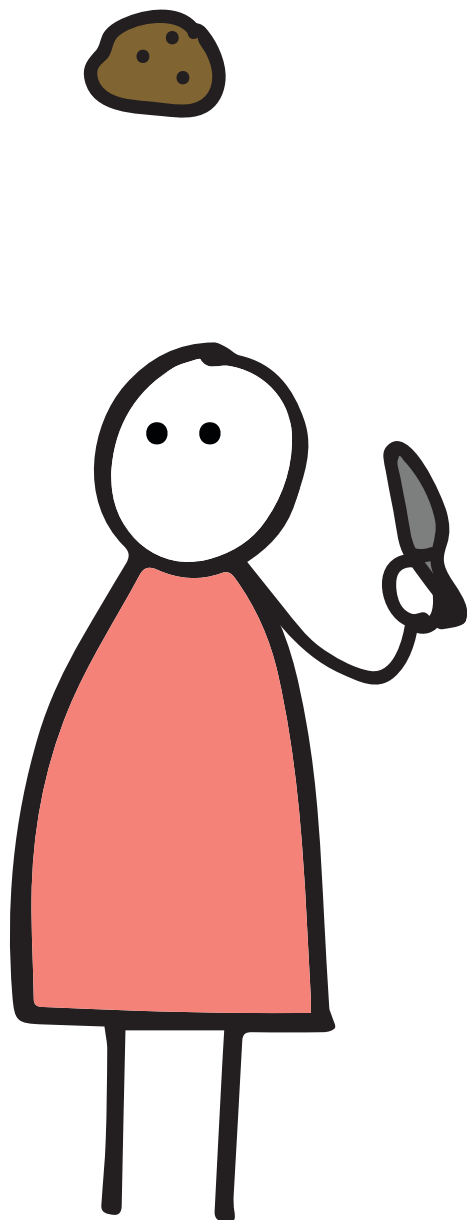


POL

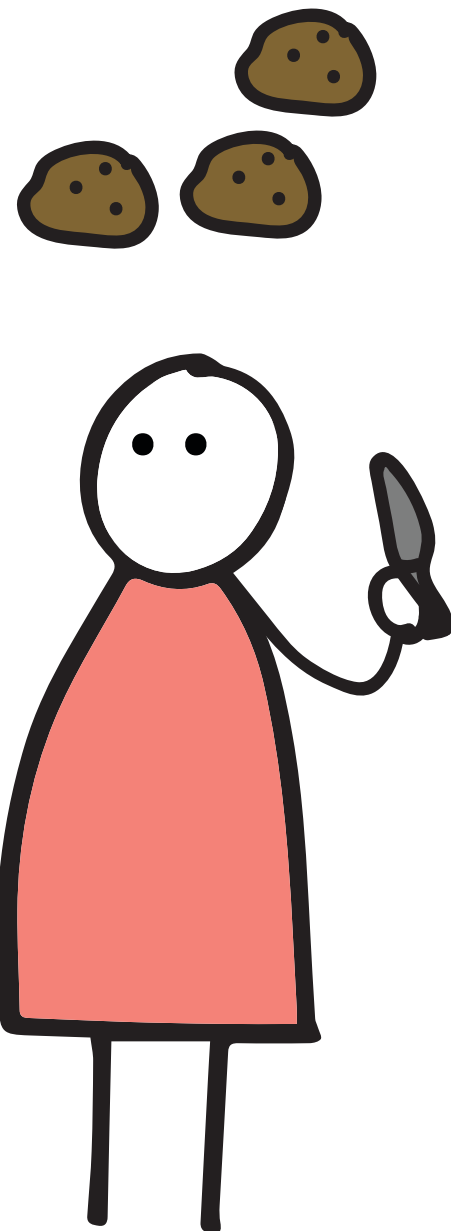
Versió recursiva



JUAN

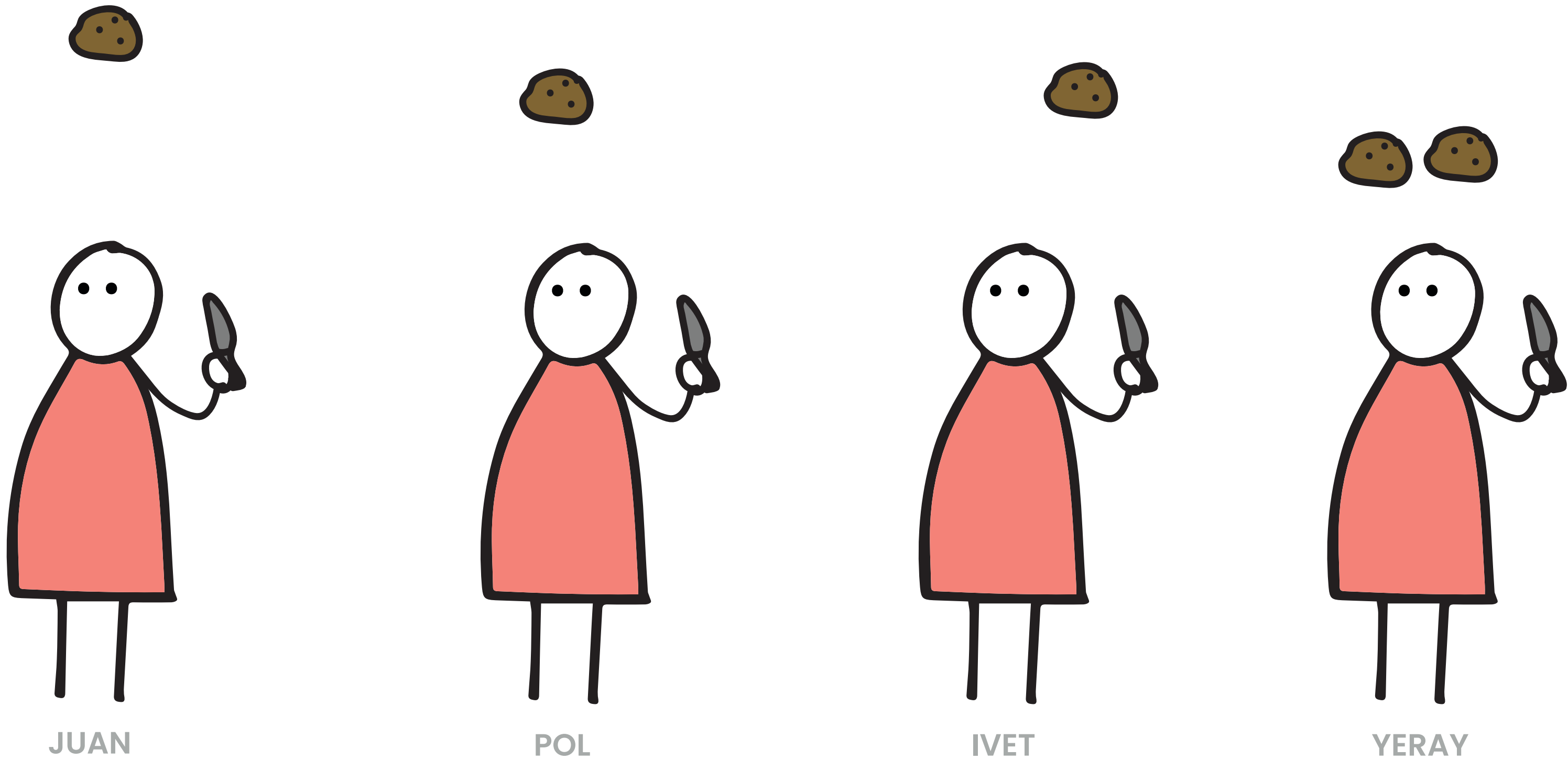


POL



IVET

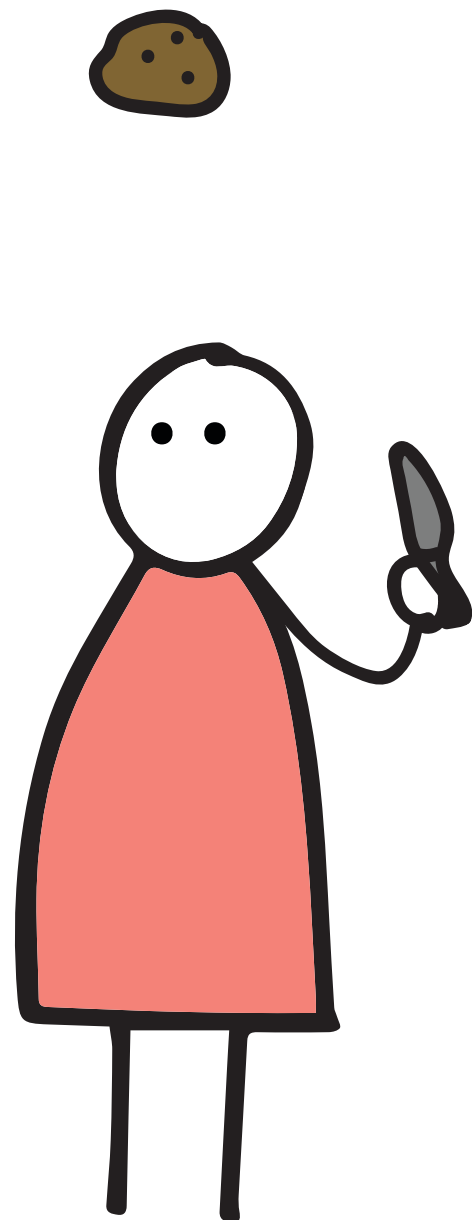
Versió recursiva



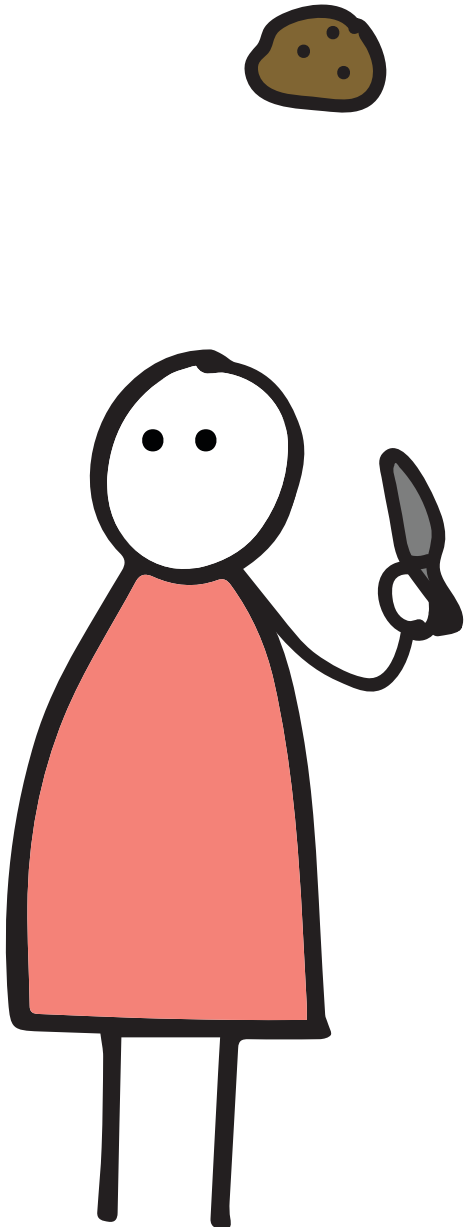
Versió recursiva



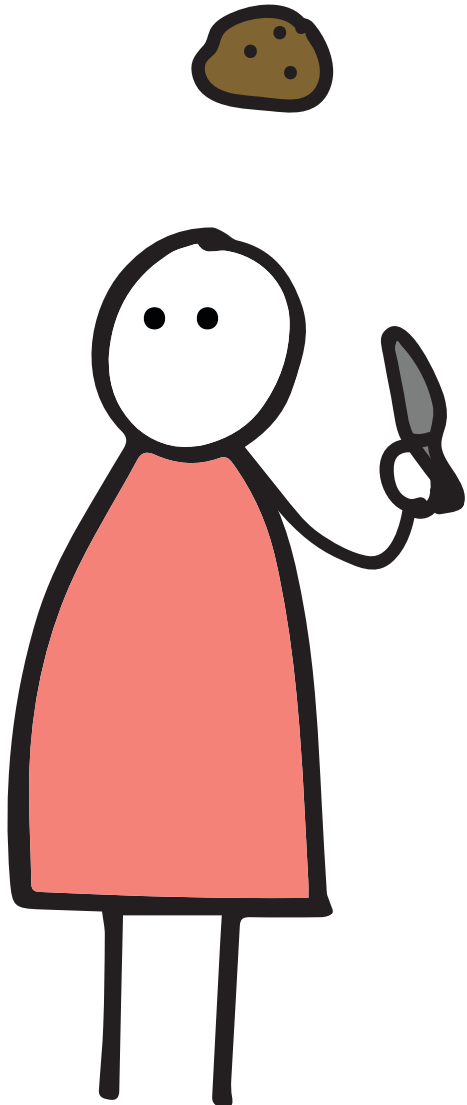
JUAN



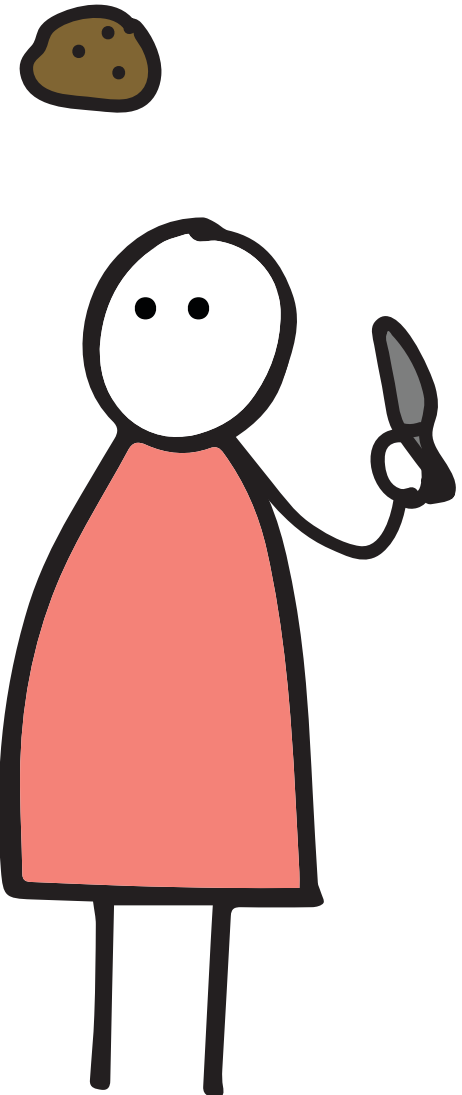
POL



IVET



YERAY

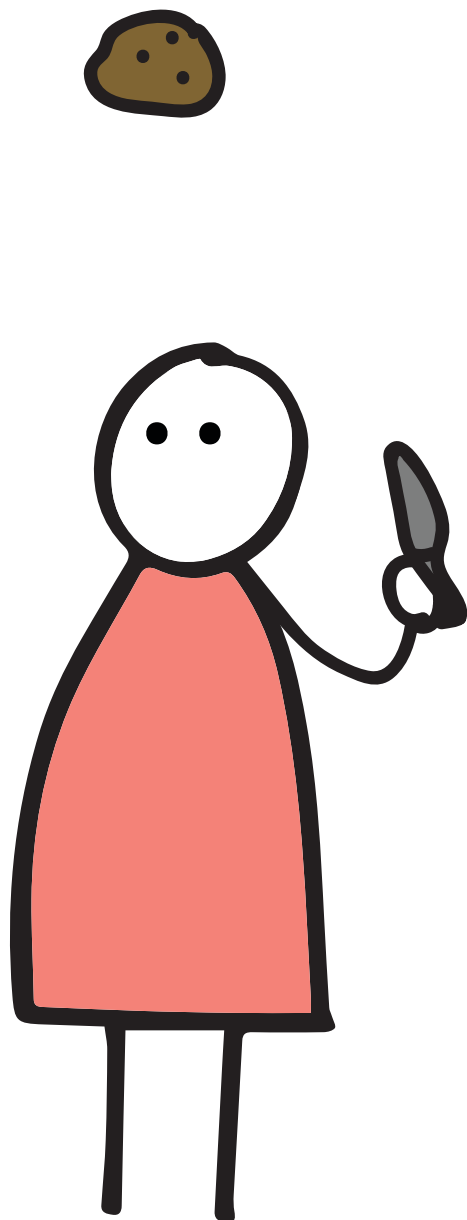


MARCEL

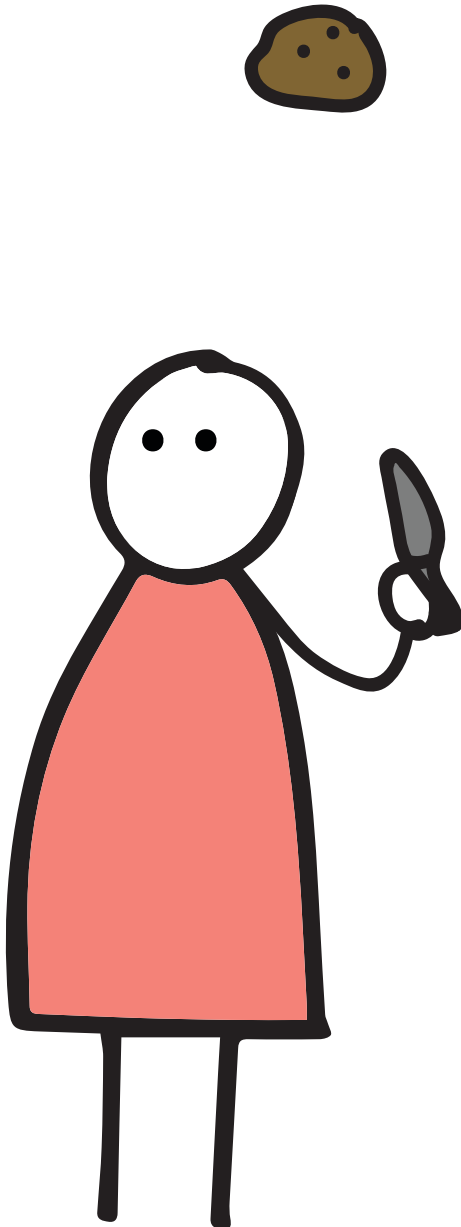
Versió recursiva



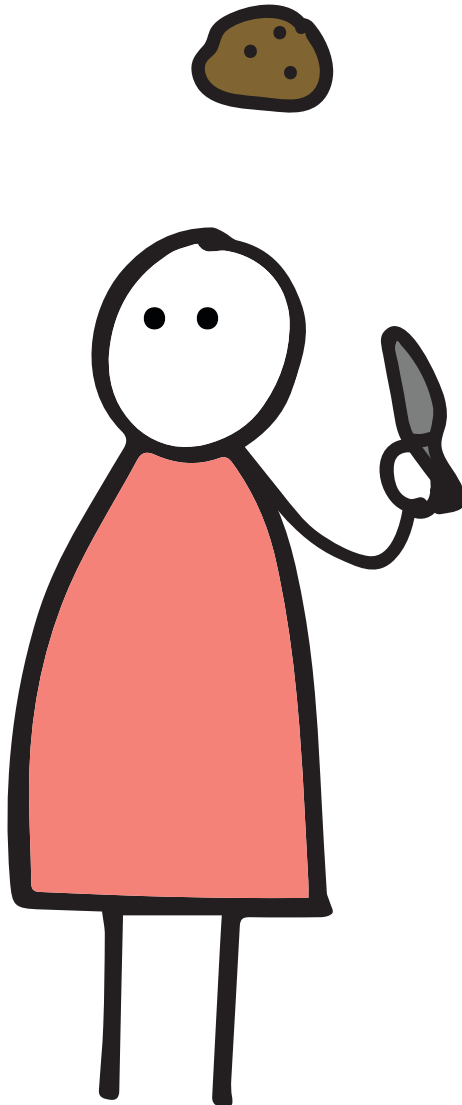
JUAN



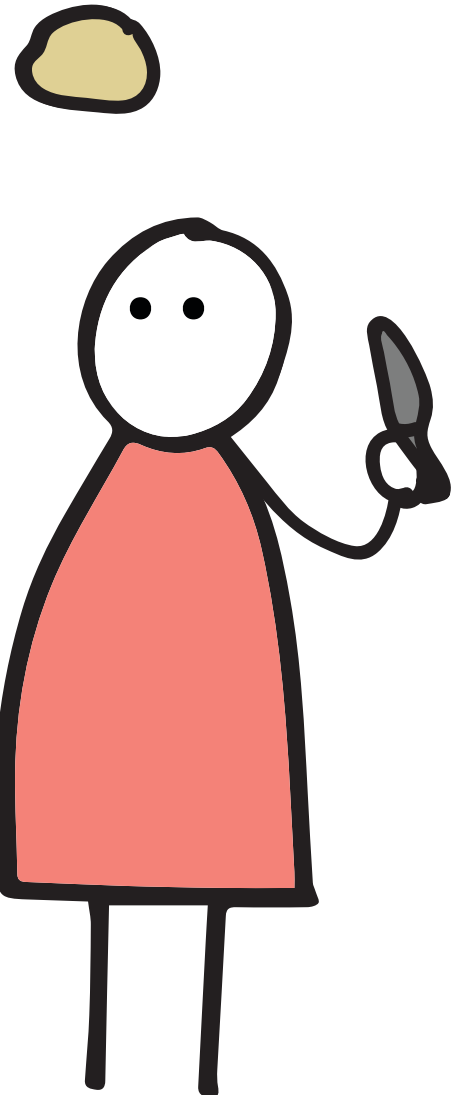
POL



IVET



YERAY

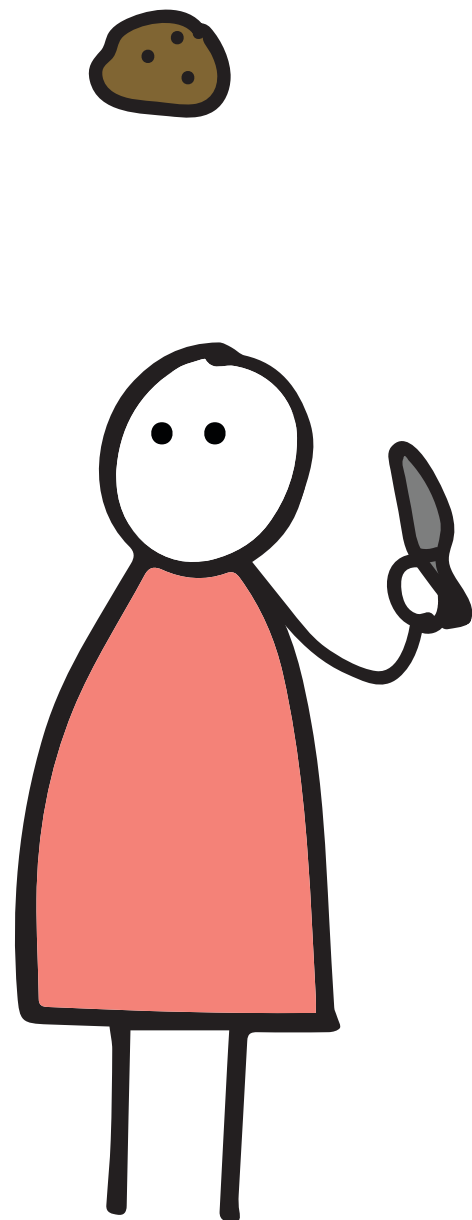


MARCEL

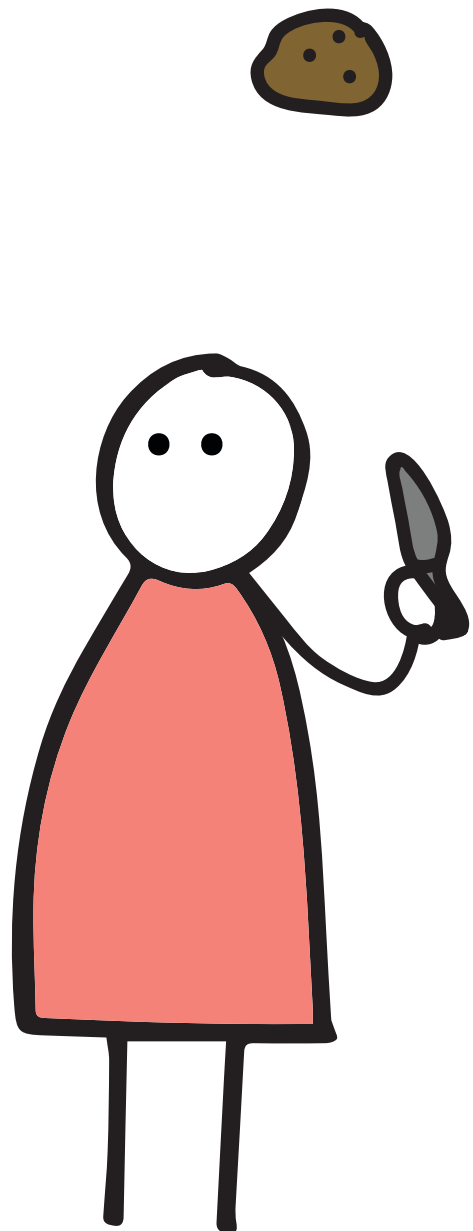
Versió recursiva



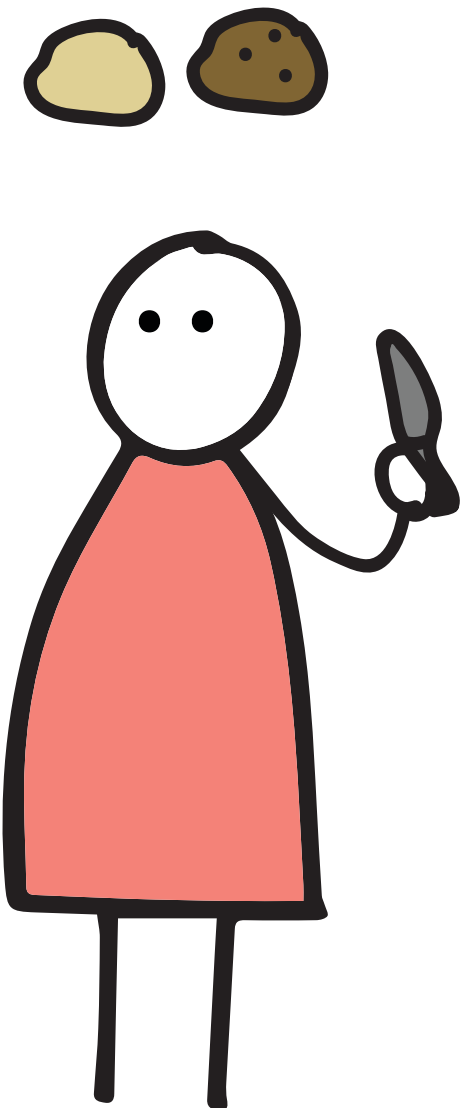
JUAN



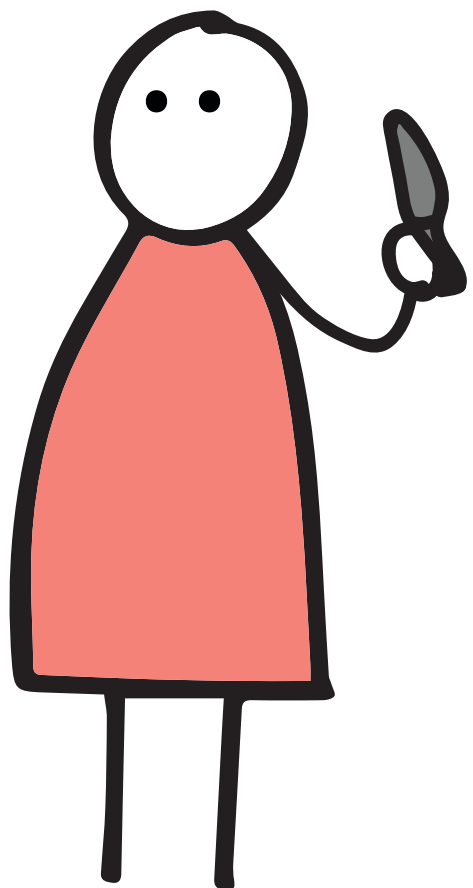
POL



IVET



YERAY

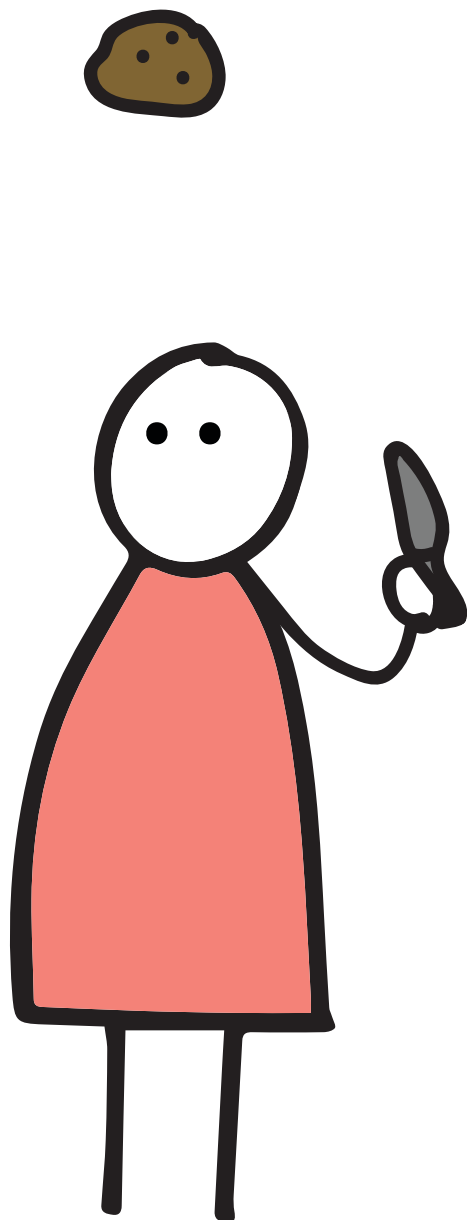


MARCEL

Versió recursiva



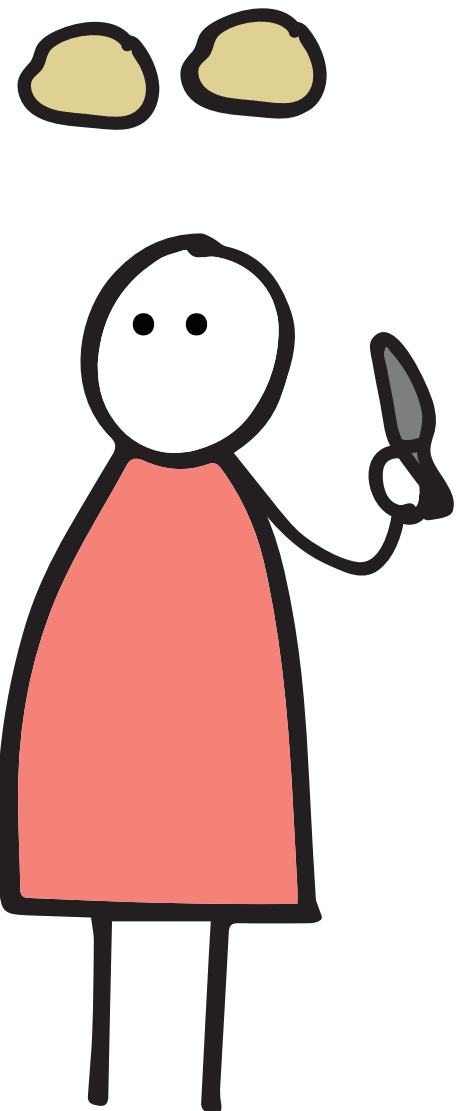
JUAN



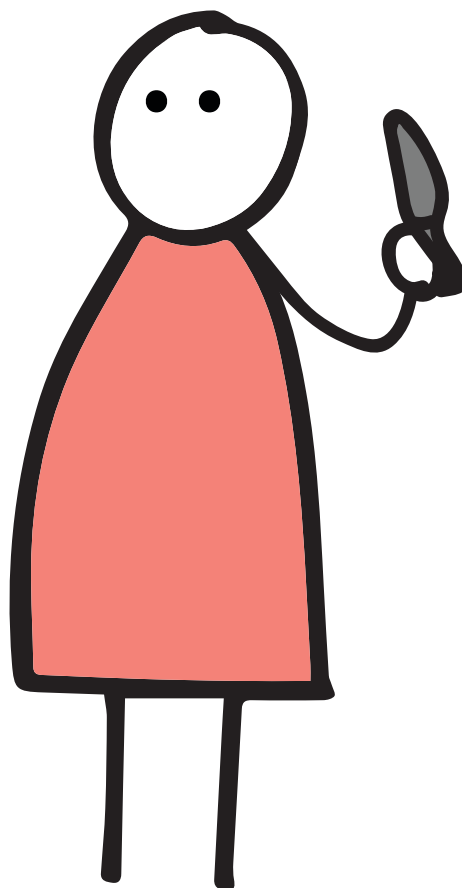
POL



IVET



YERAY

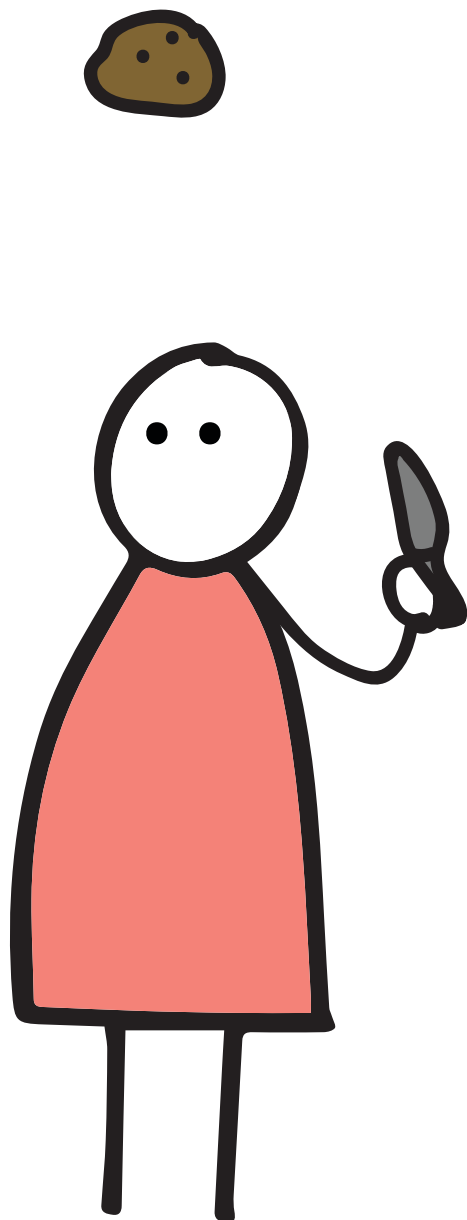


MARCEL

Versió recursiva



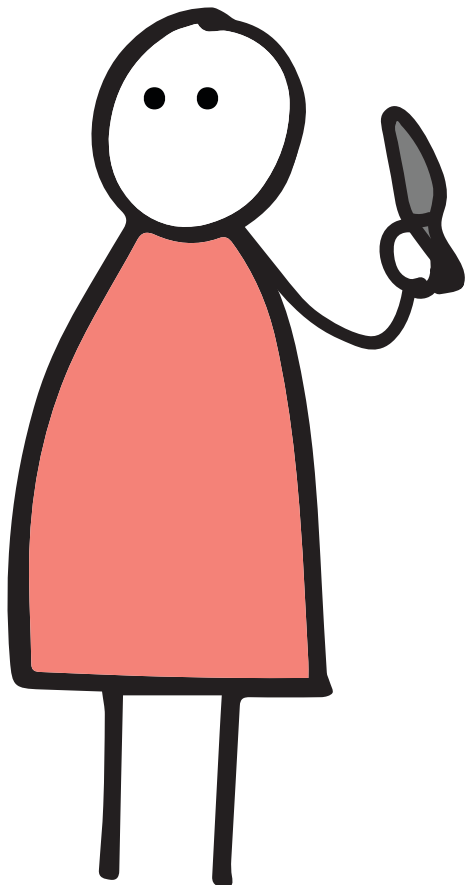
JUAN



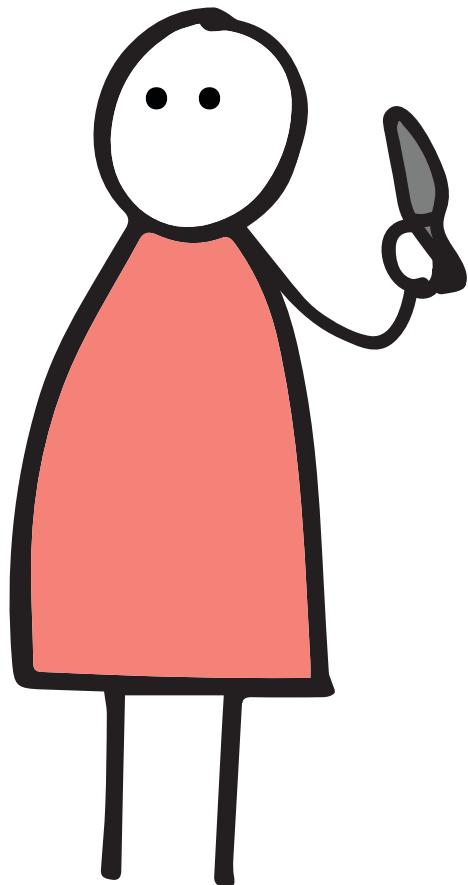
POL



IVET



YERAY

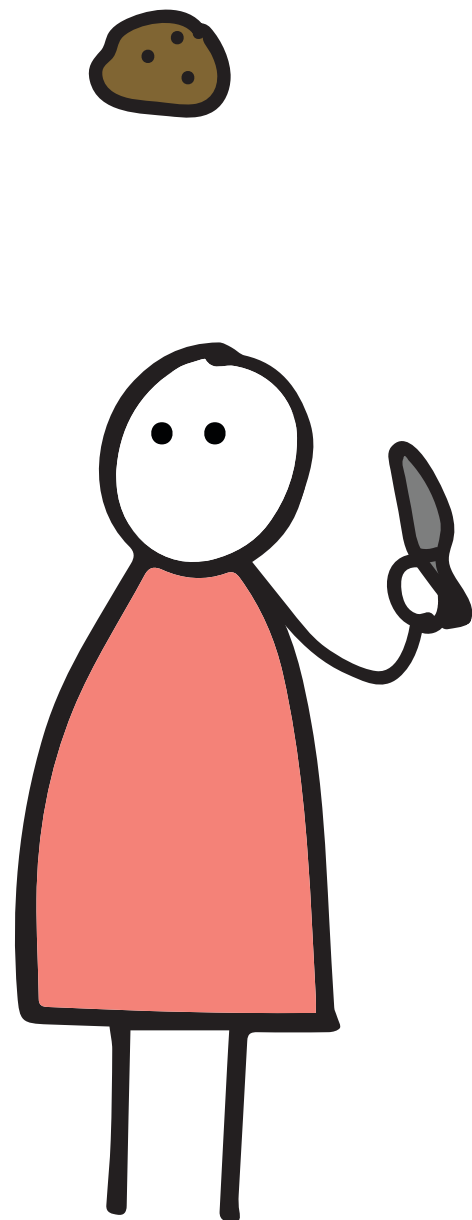


MARCEL

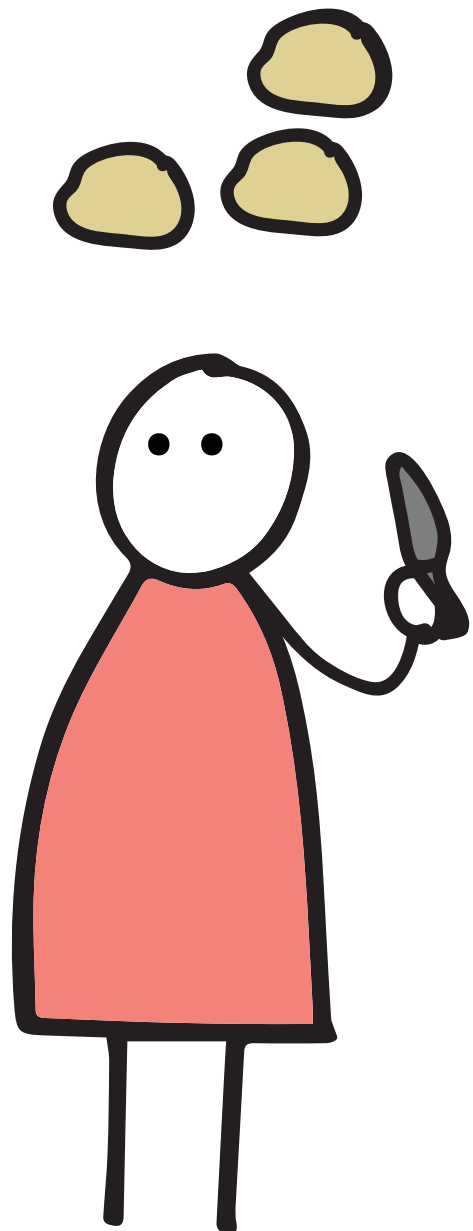
Versió recursiva



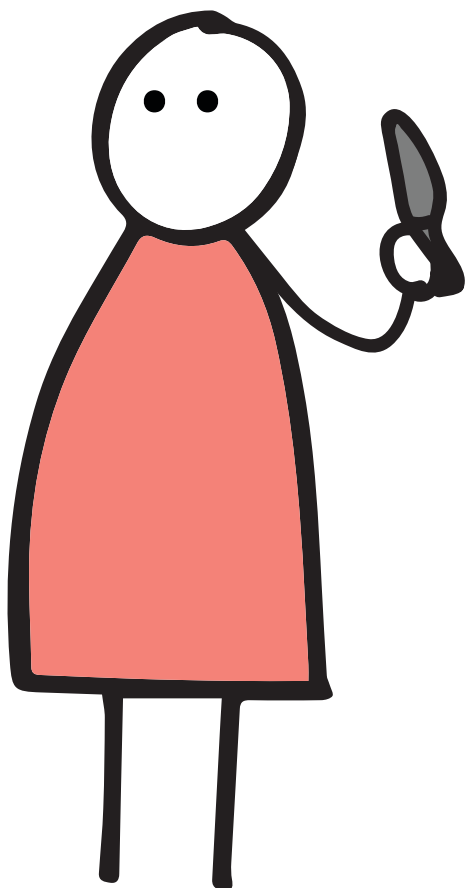
JUAN



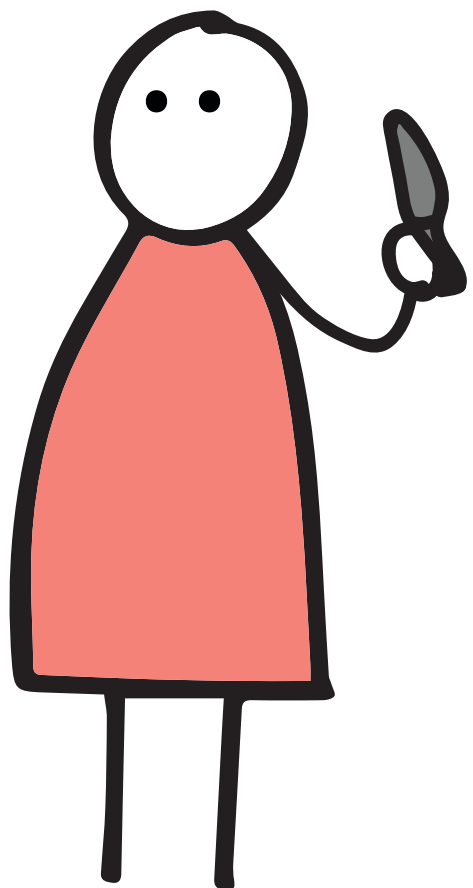
POL



IVET

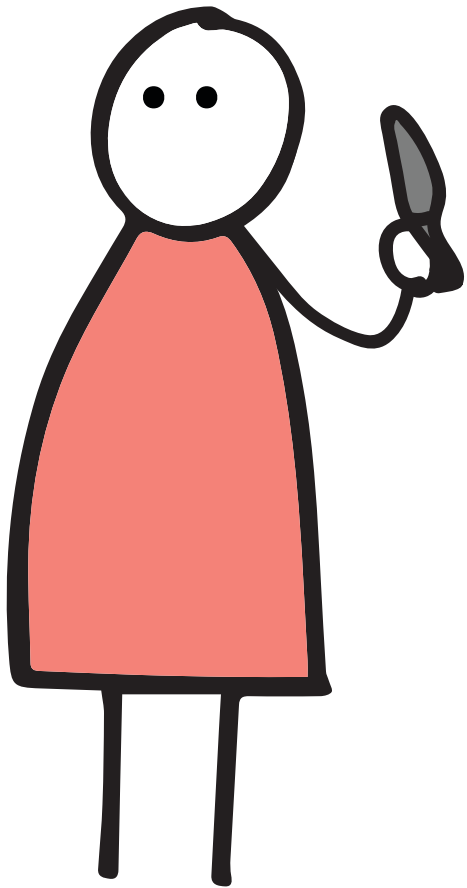


YERAY

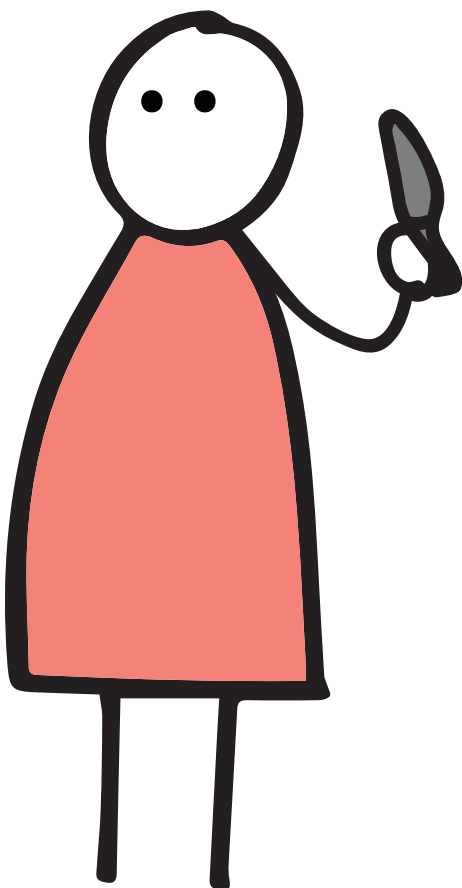
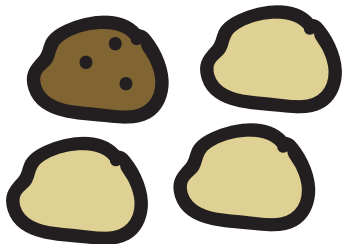


MARCEL

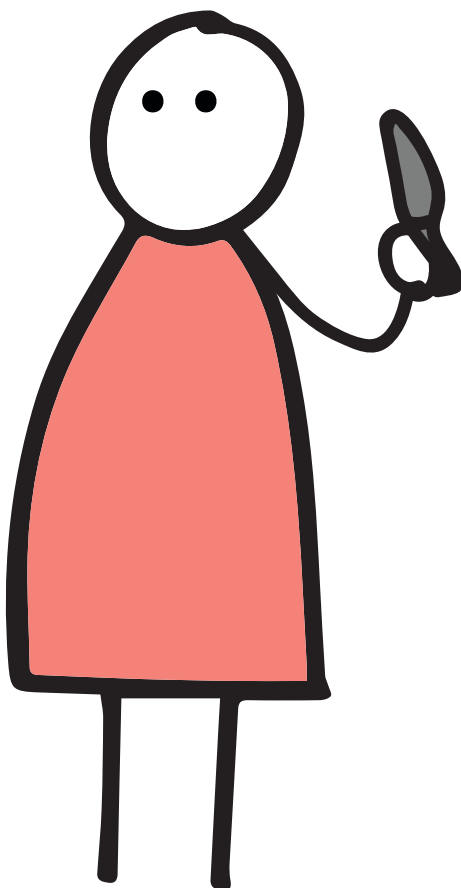
Versió recursiva



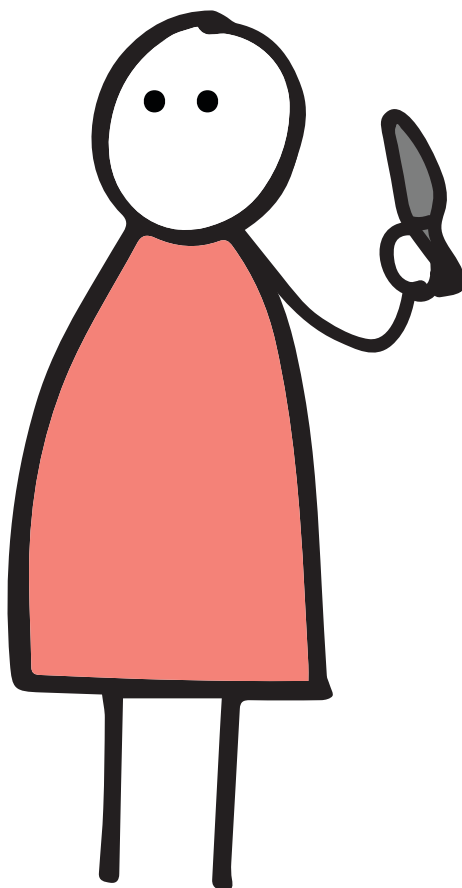
JUAN



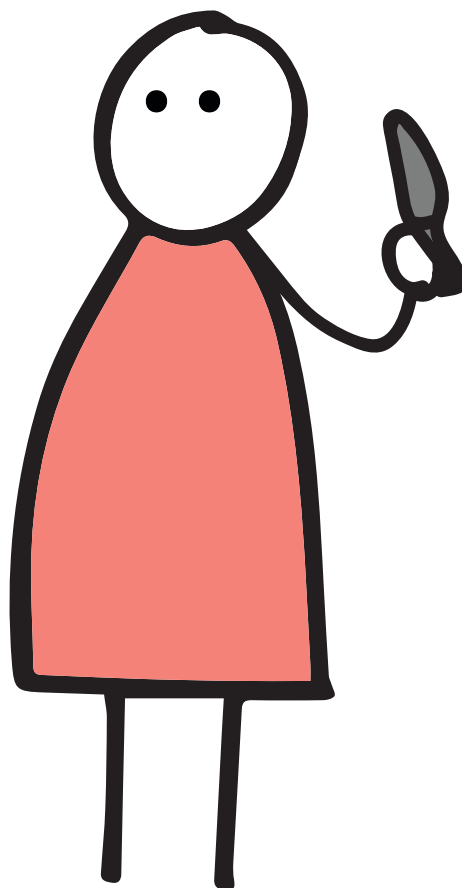
POL



IVET

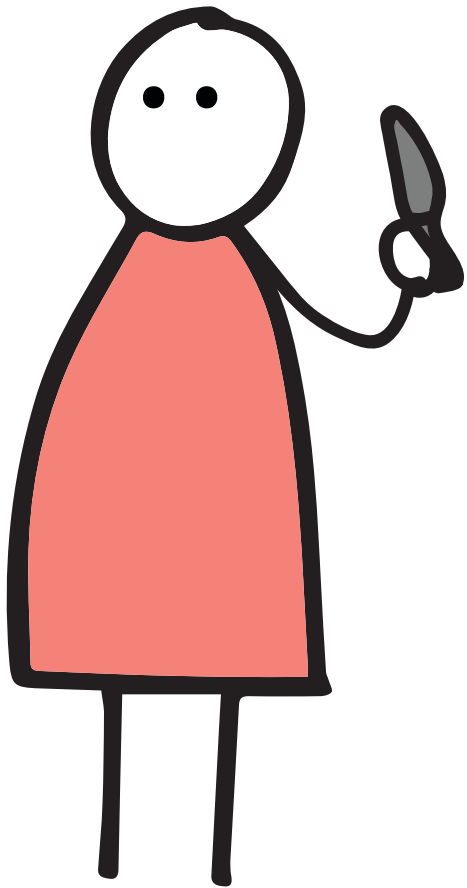
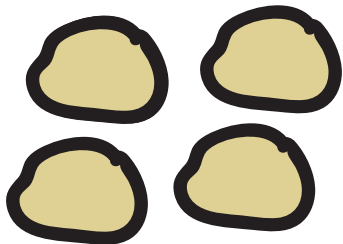


YERAY

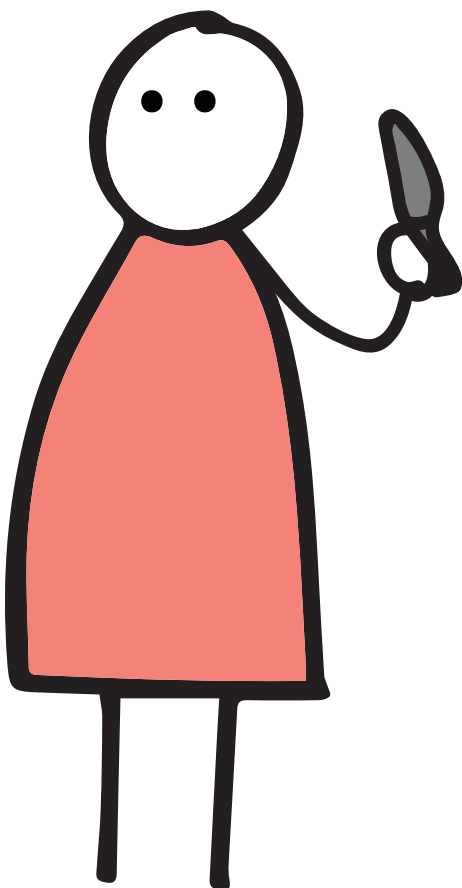


MARCEL

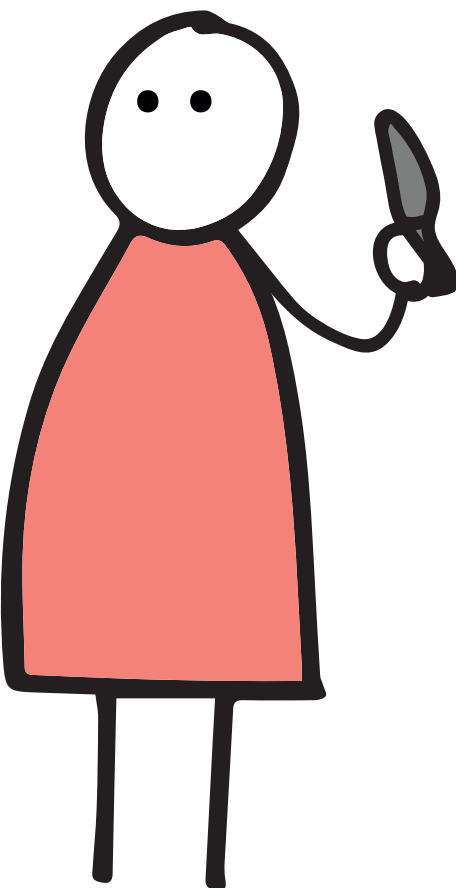
Versió recursiva



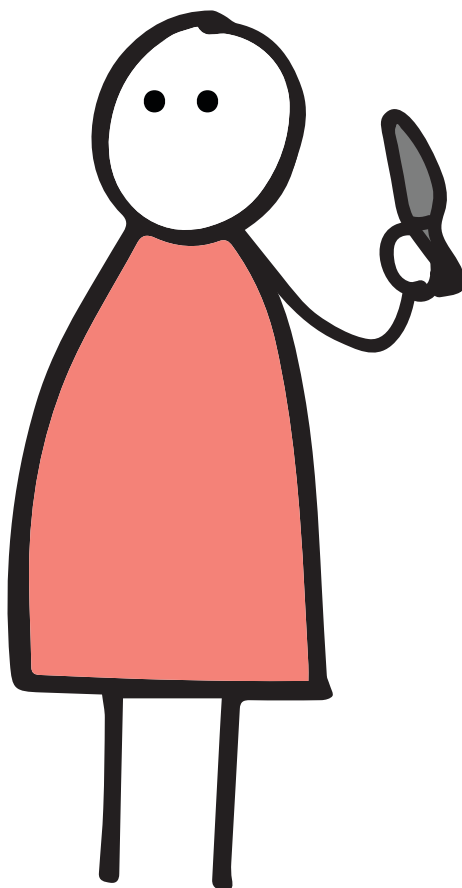
JUAN



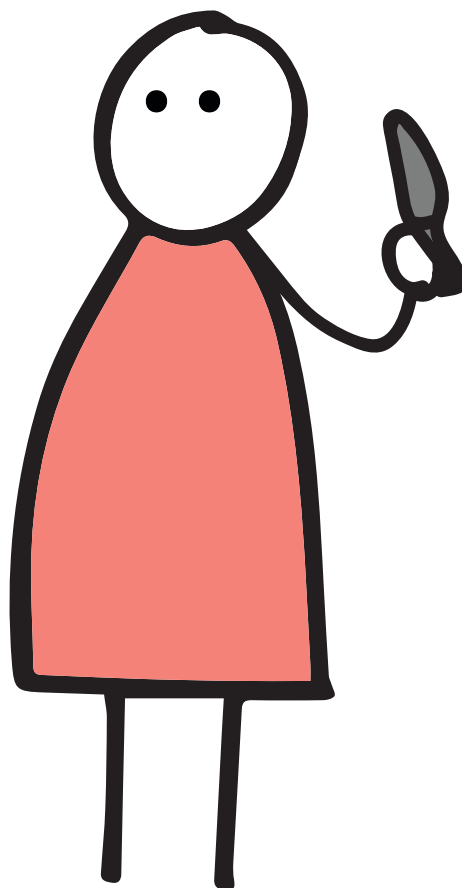
POL



IVET

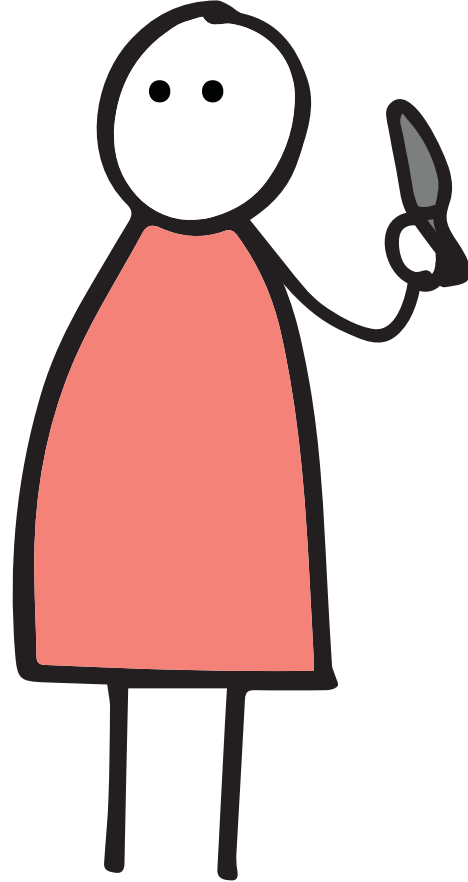
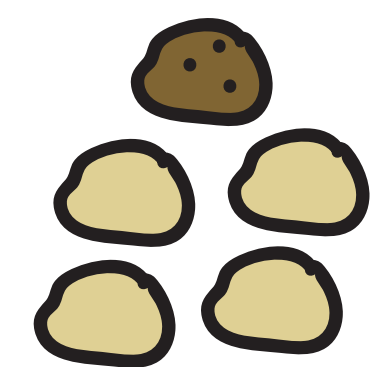


YERAY

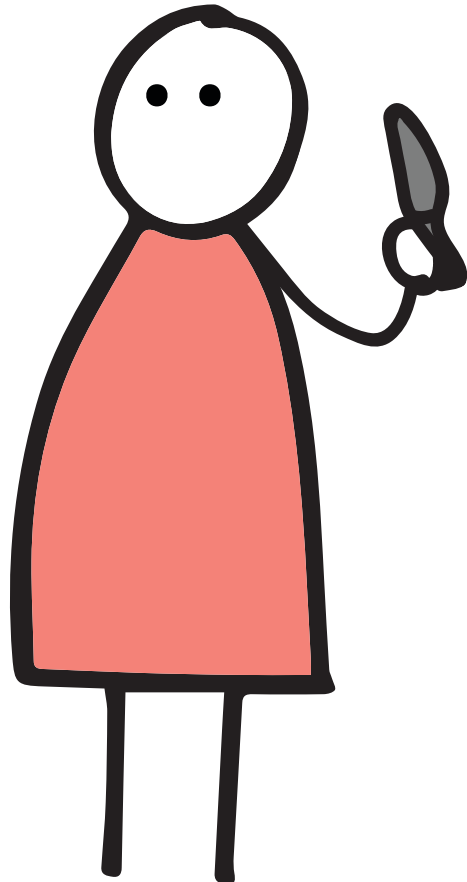


MARCEL

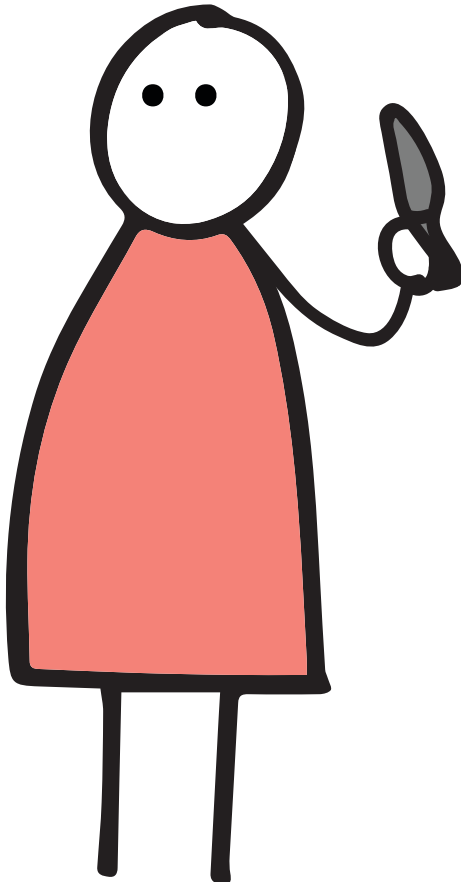
Versió recursiva



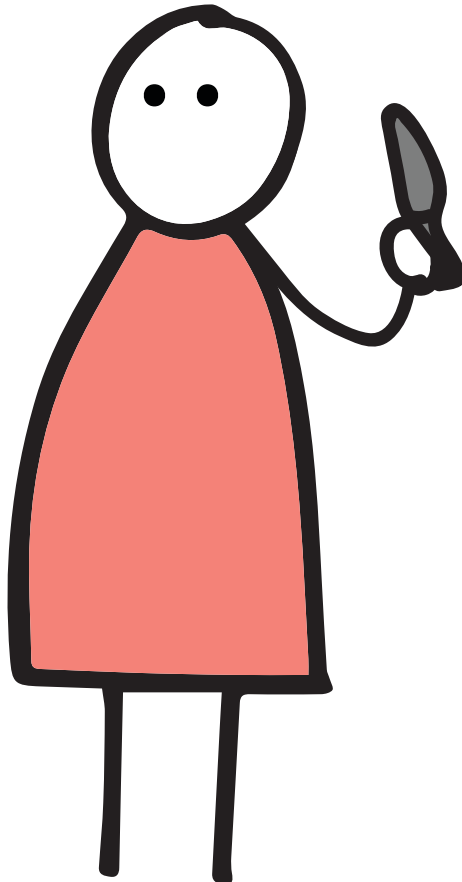
JUAN



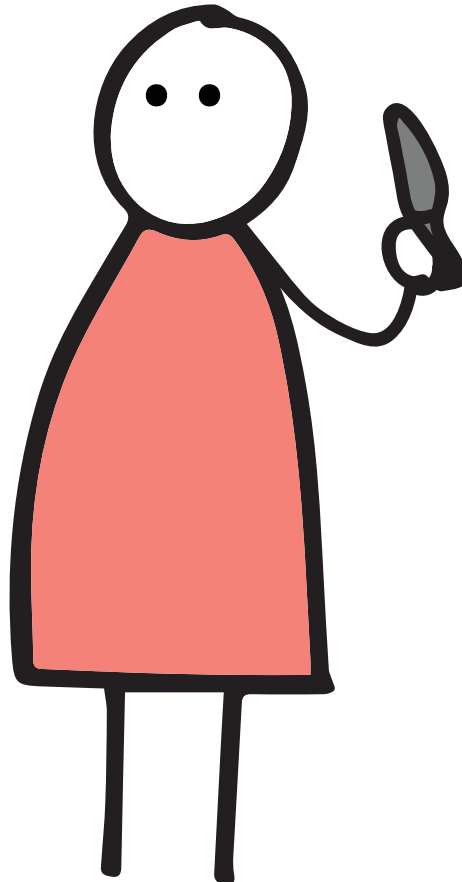
POL



IVET

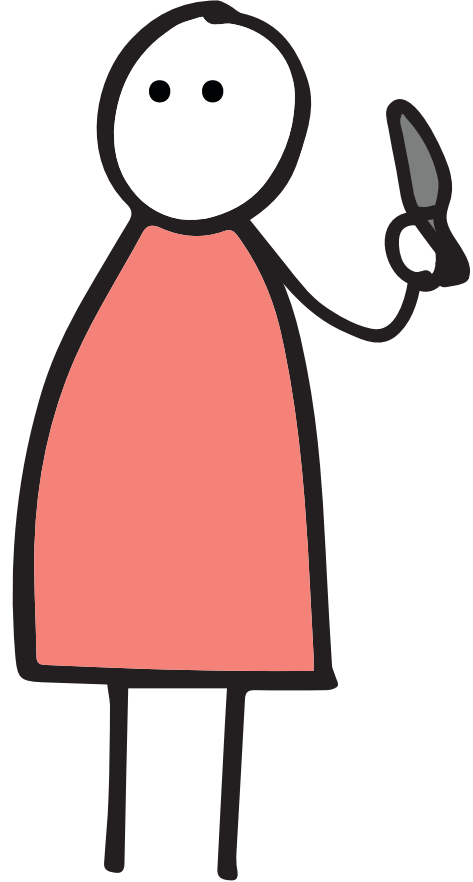
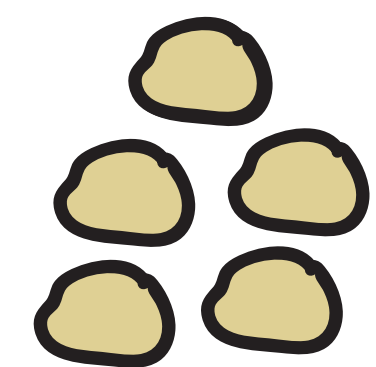


YERAY

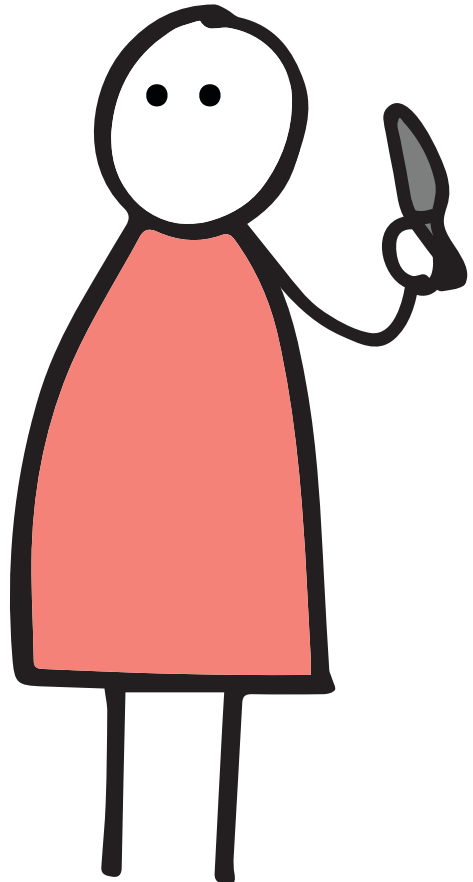


MARCEL

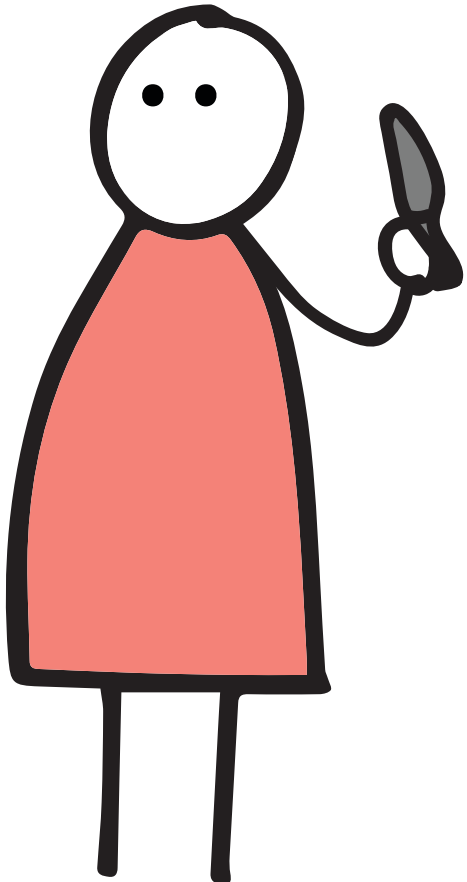
Versió recursiva



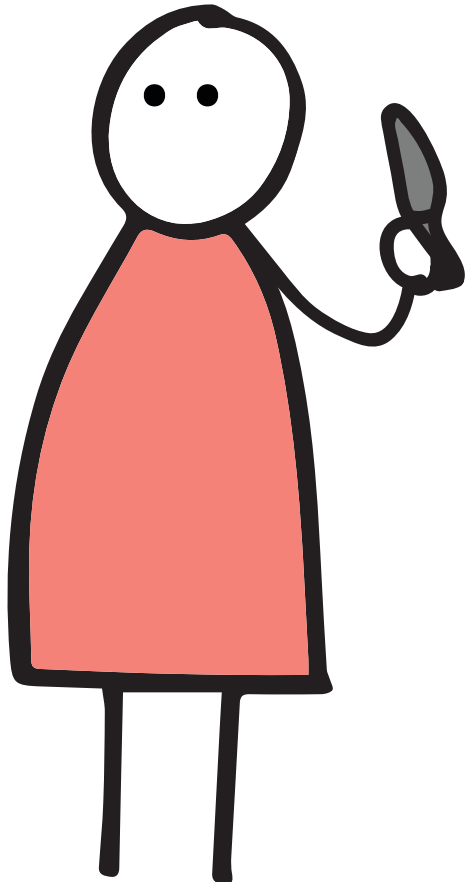
JUAN



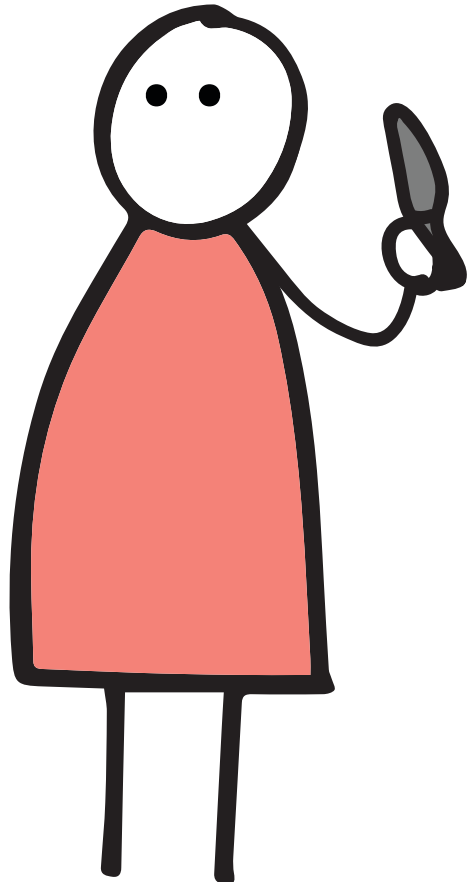
POL



IVET

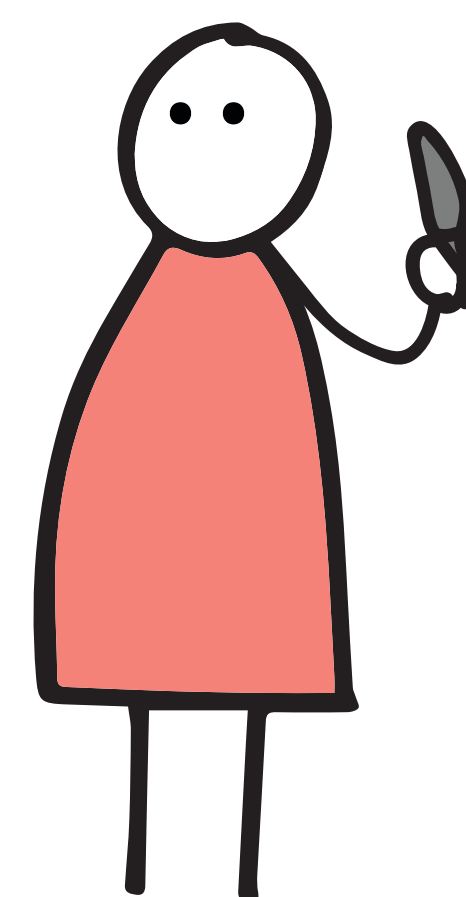
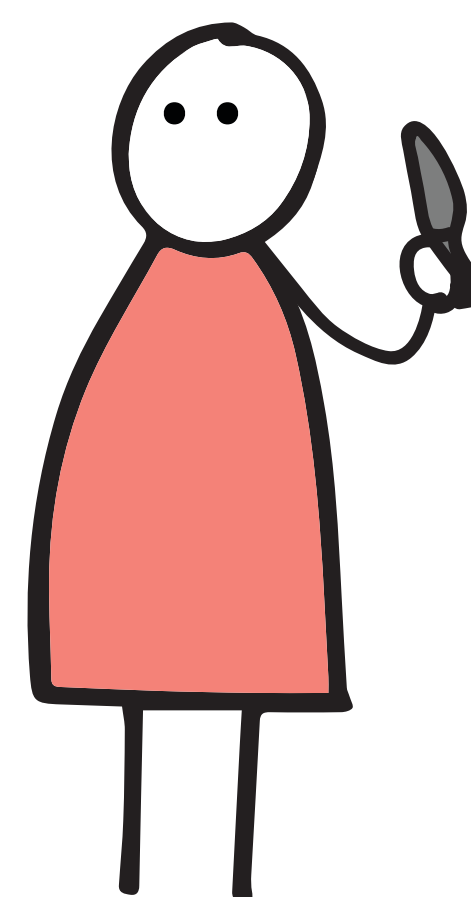
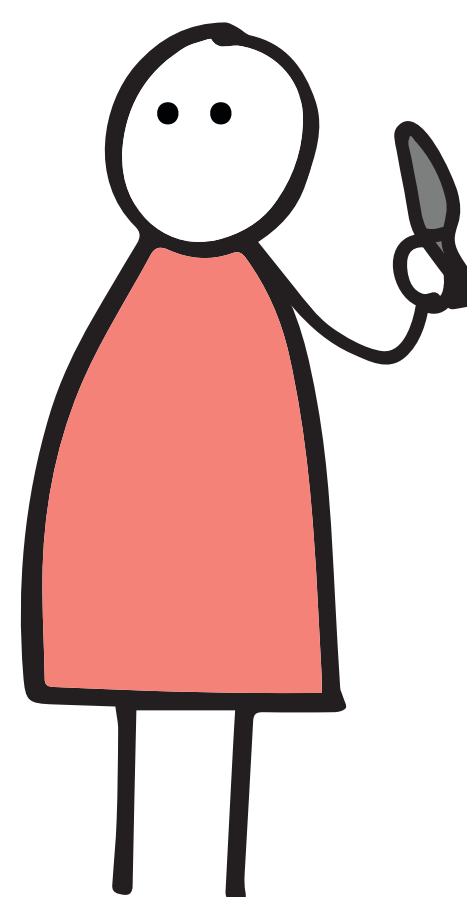
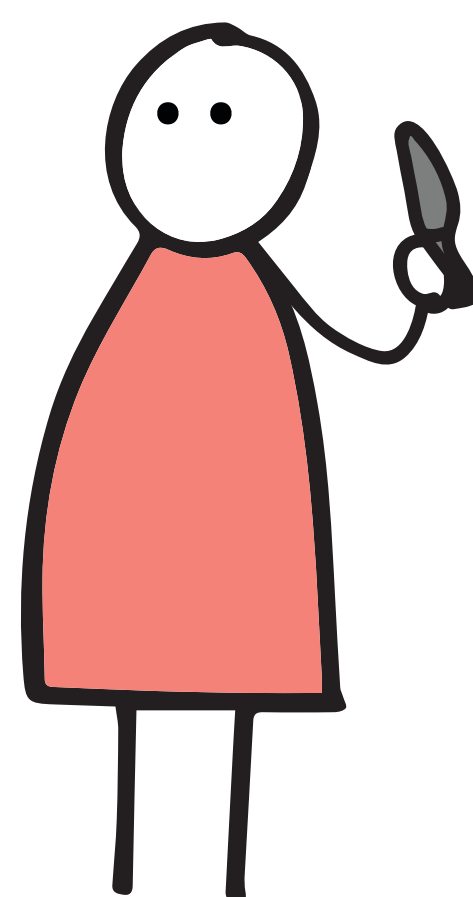
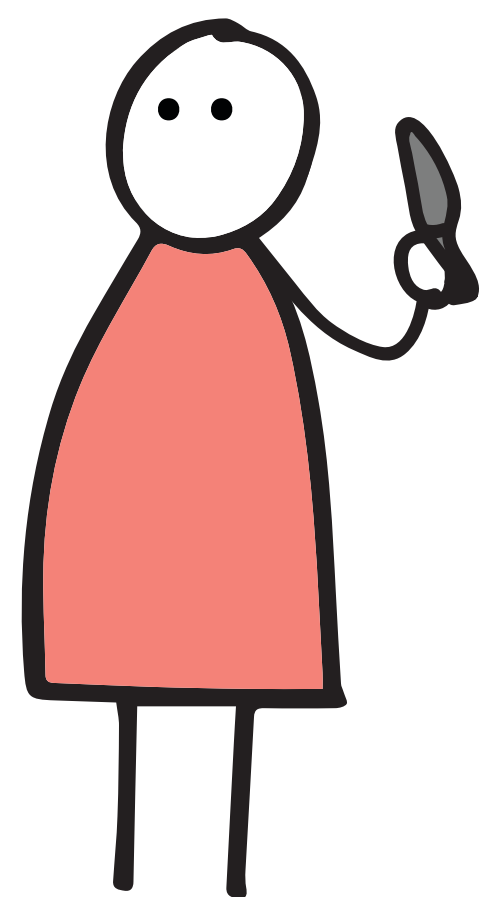
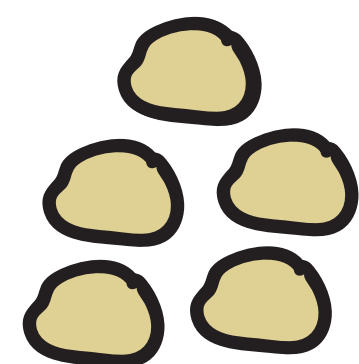


YERAY

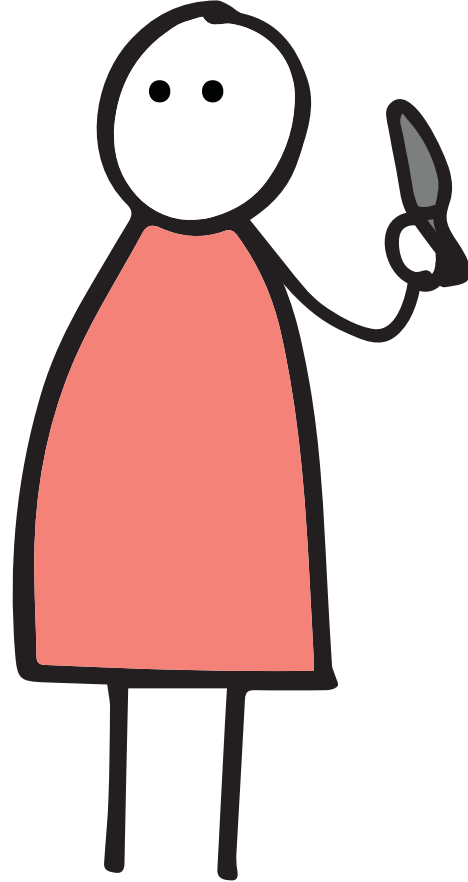
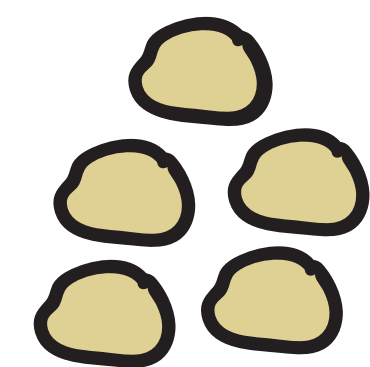


MARCEL

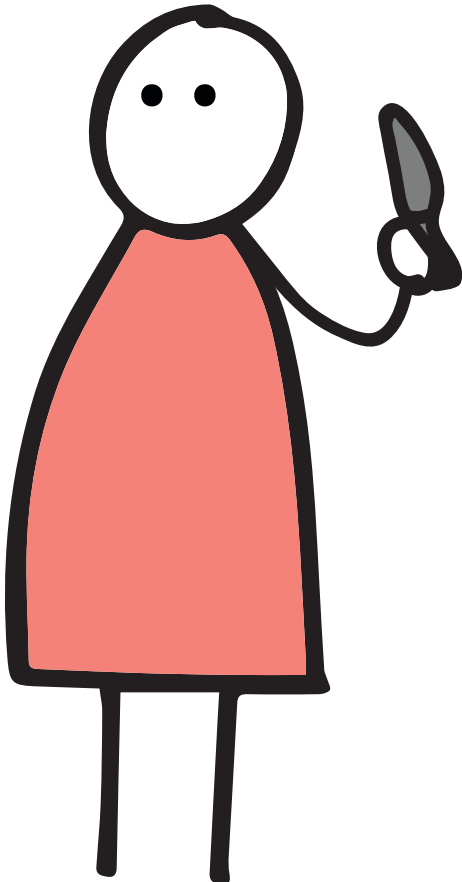
Versió recursiva



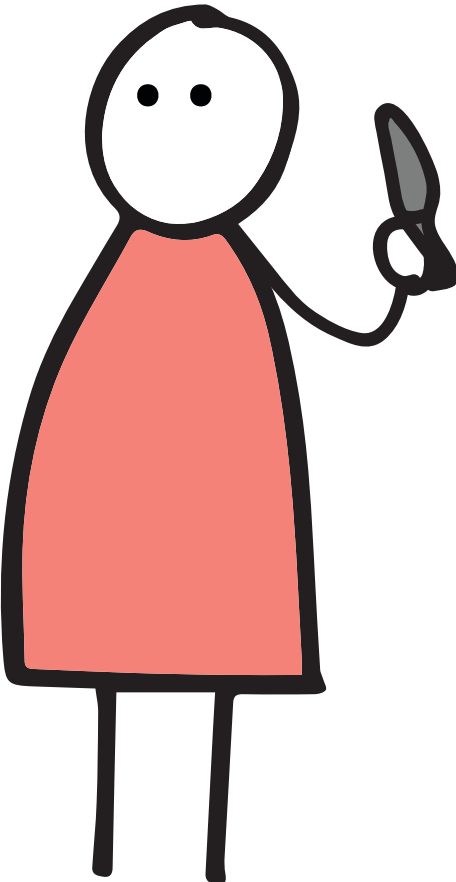
Versió recursiva



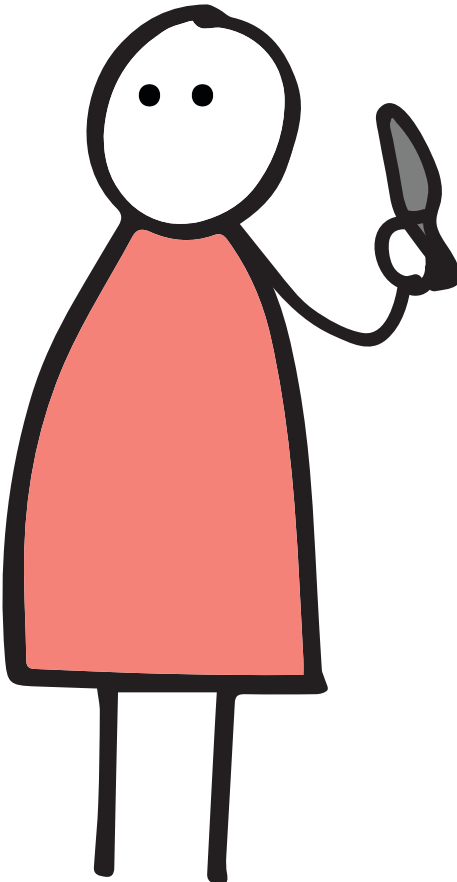
JUAN



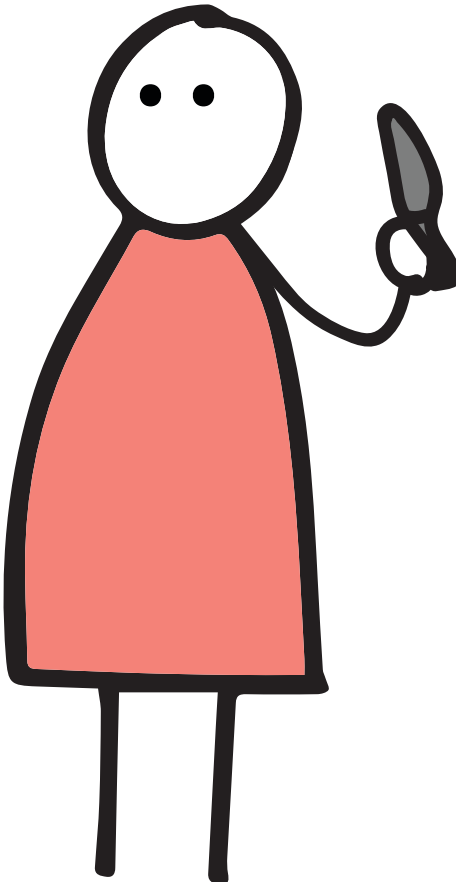
JUAN



JUAN



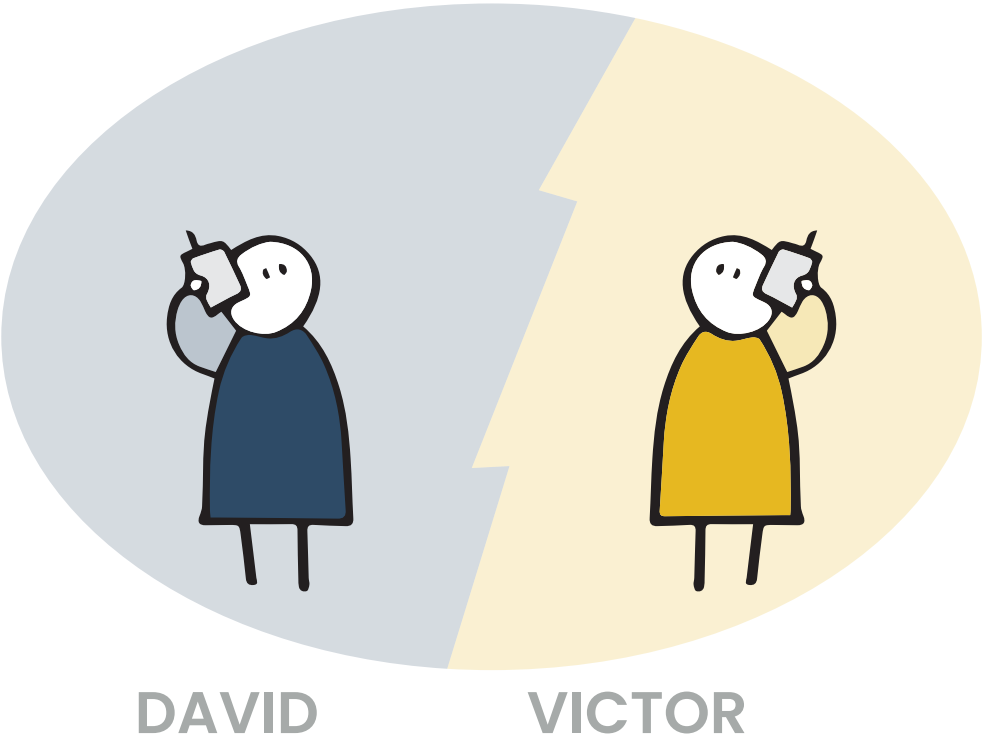
JUAN

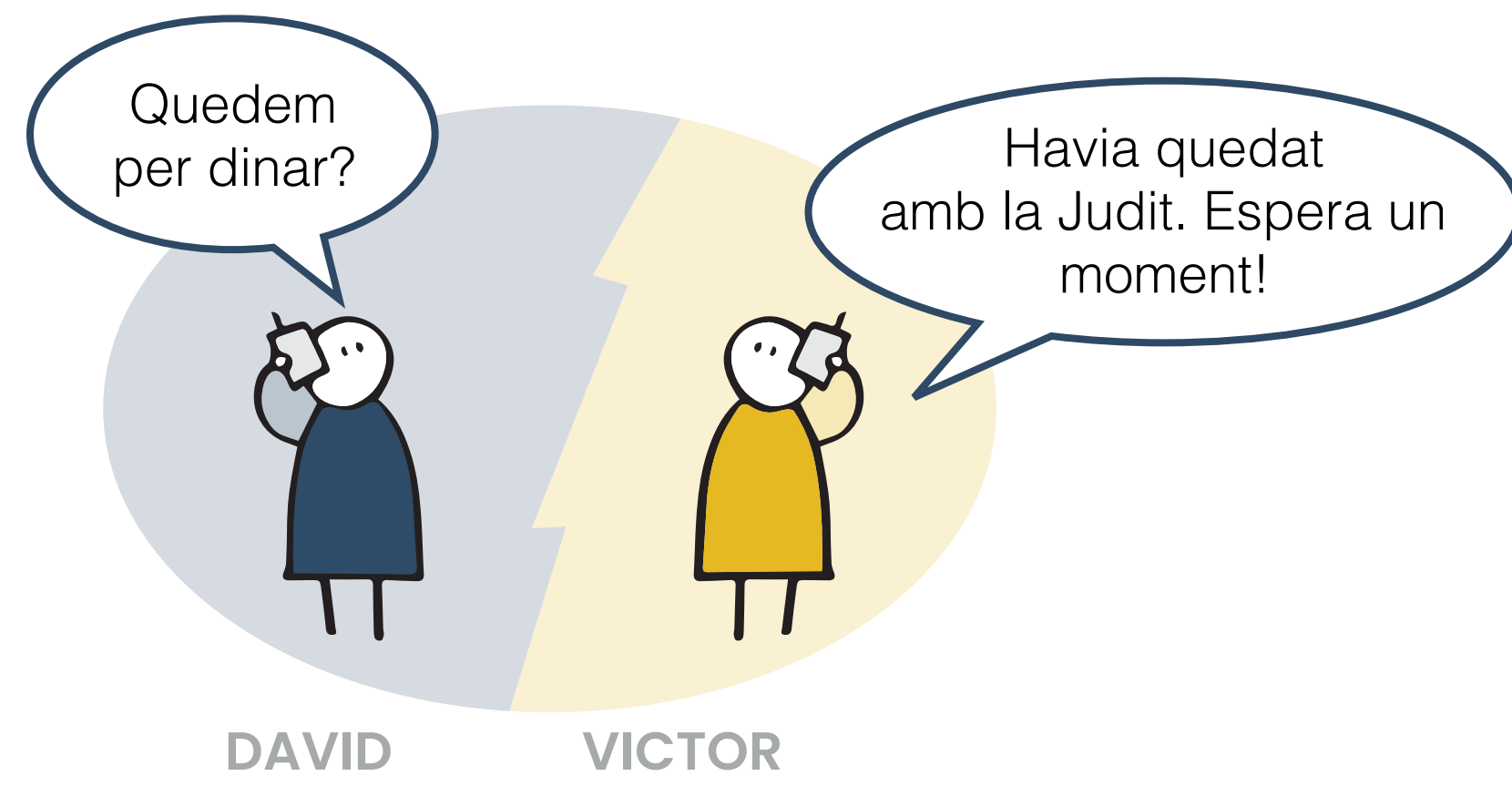


JUAN

Exemplificació gràfica de com funciona una funció recursiva.

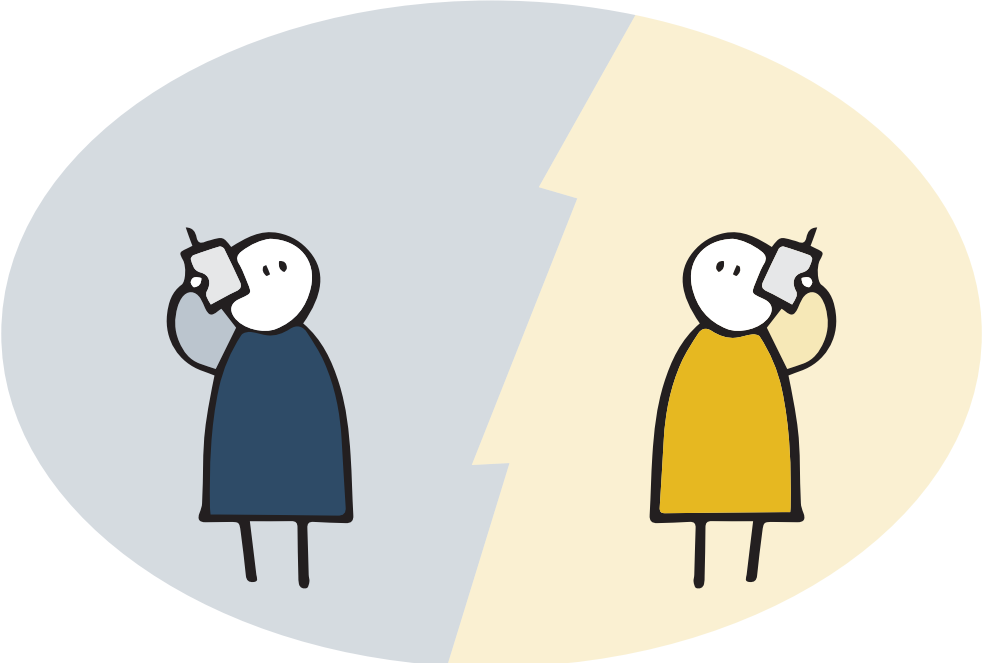
Exemplificació gràfica de com funciona una funció recursiva.





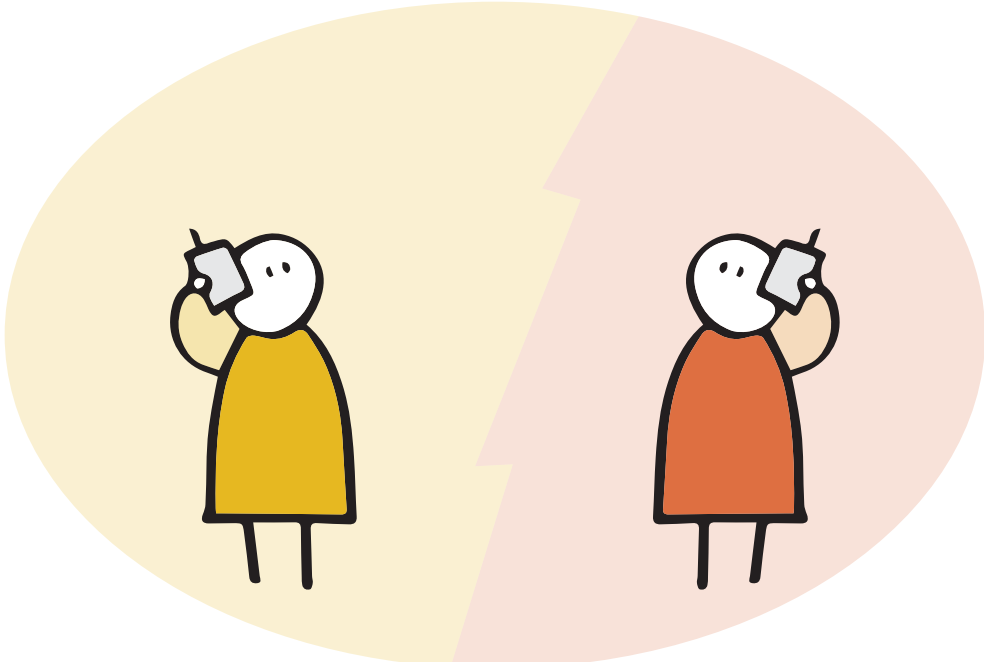
Exemplificació gràfica de com funciona una funció recursiva.

Exemplificació gràfica de com funciona una funció recursiva.



DAVID

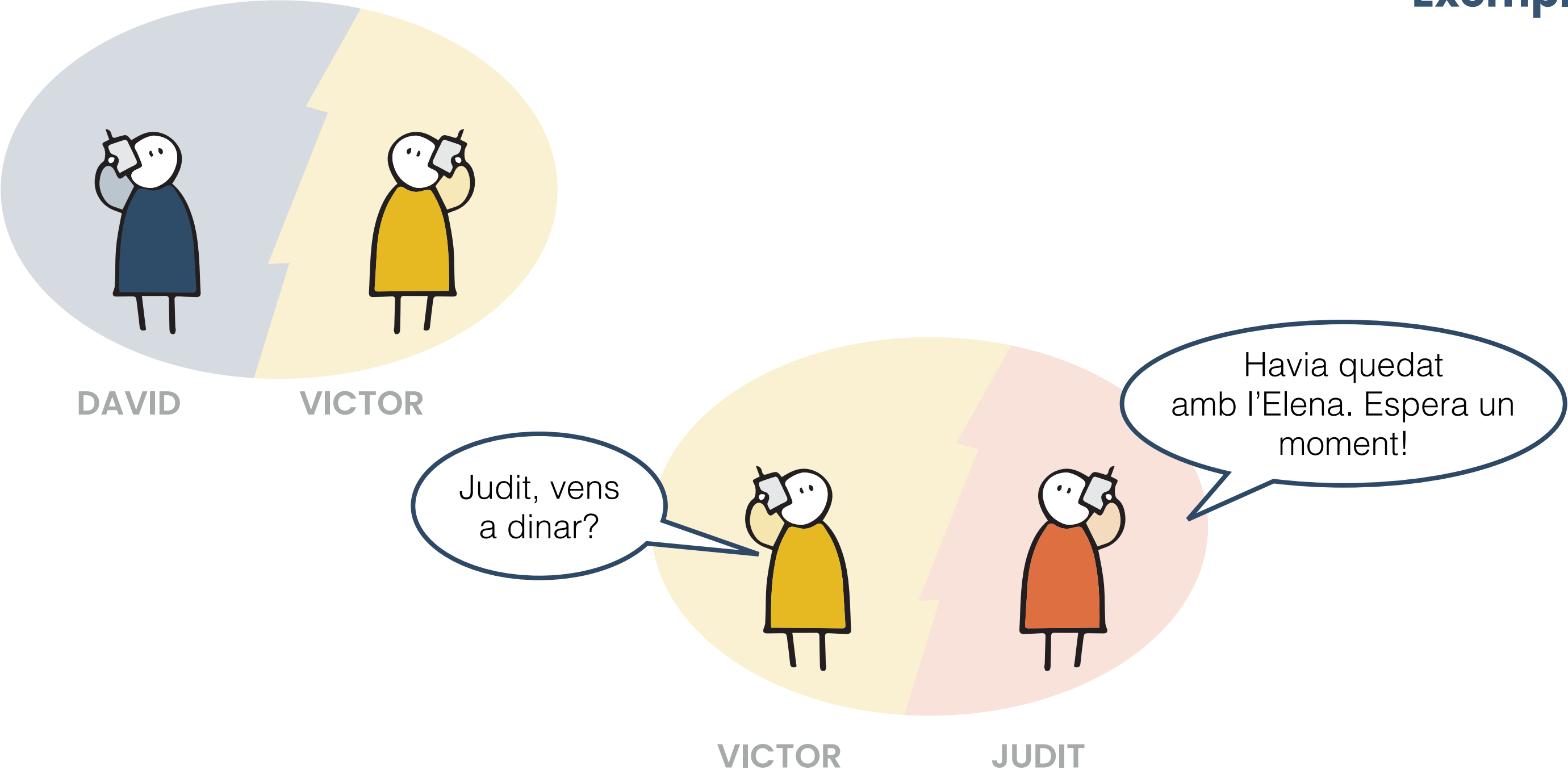
VICTOR



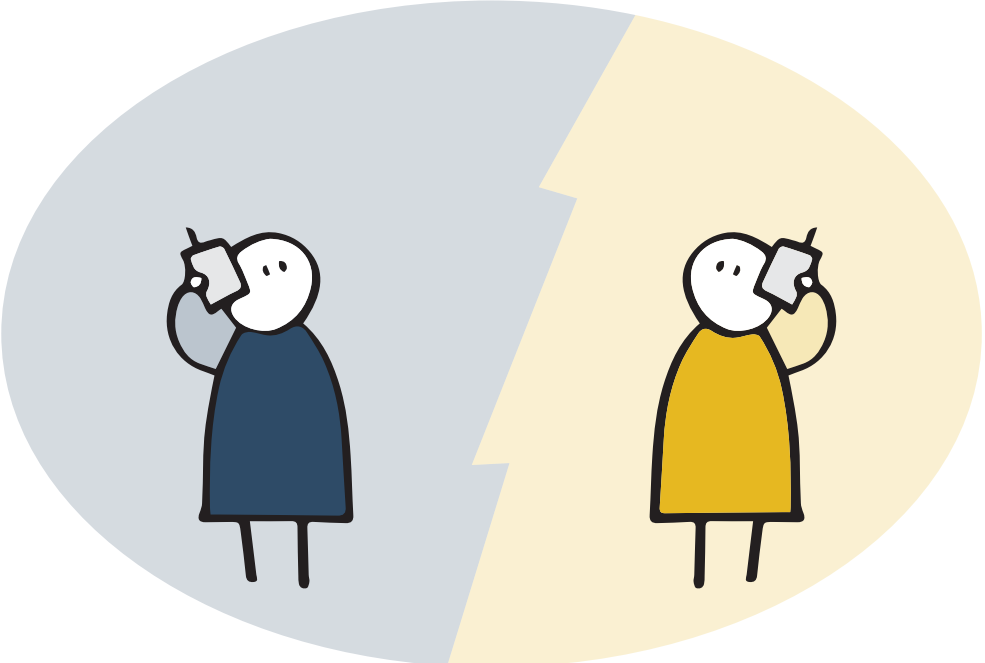
VICTOR

JUDIT

Exemplificació gràfica de com funciona una funció recursiva.

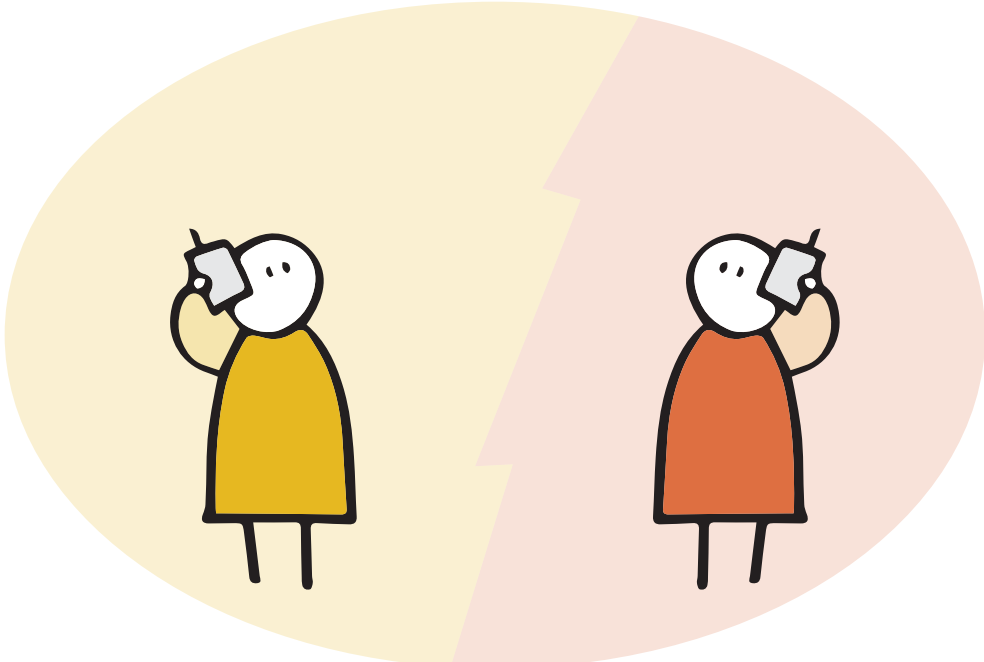


Exemplificació gràfica de com funciona una funció recursiva.



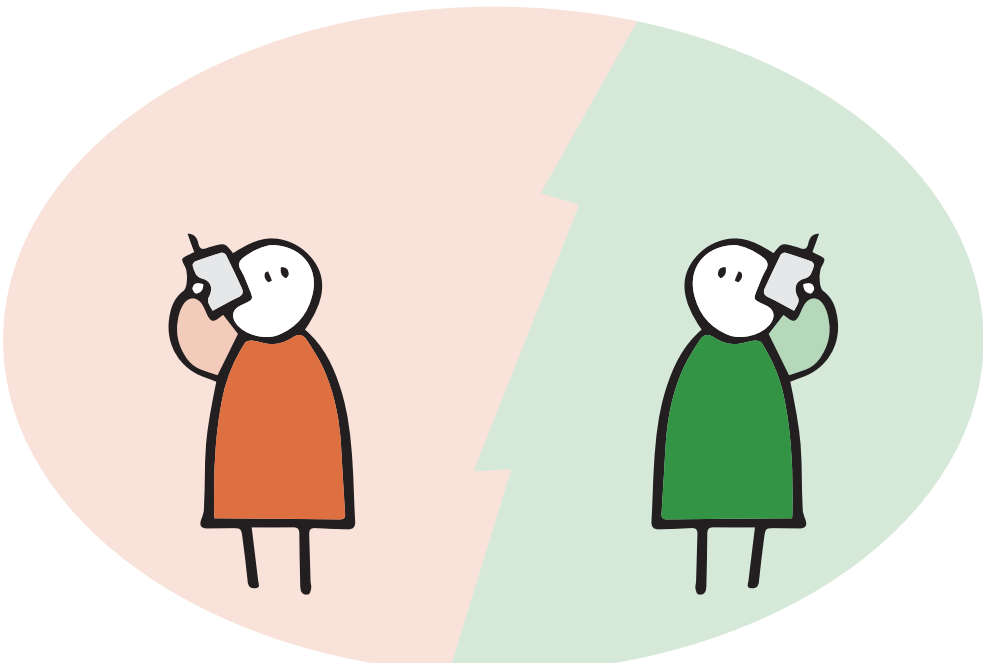
DAVID

VICTOR



VICTOR

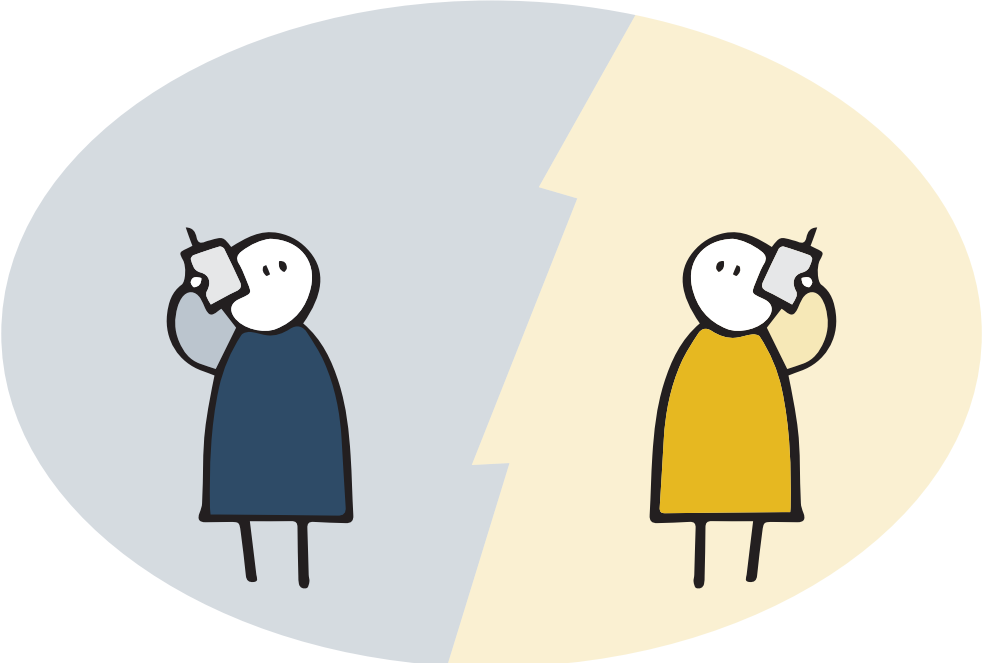
JUDIT



JUDIT

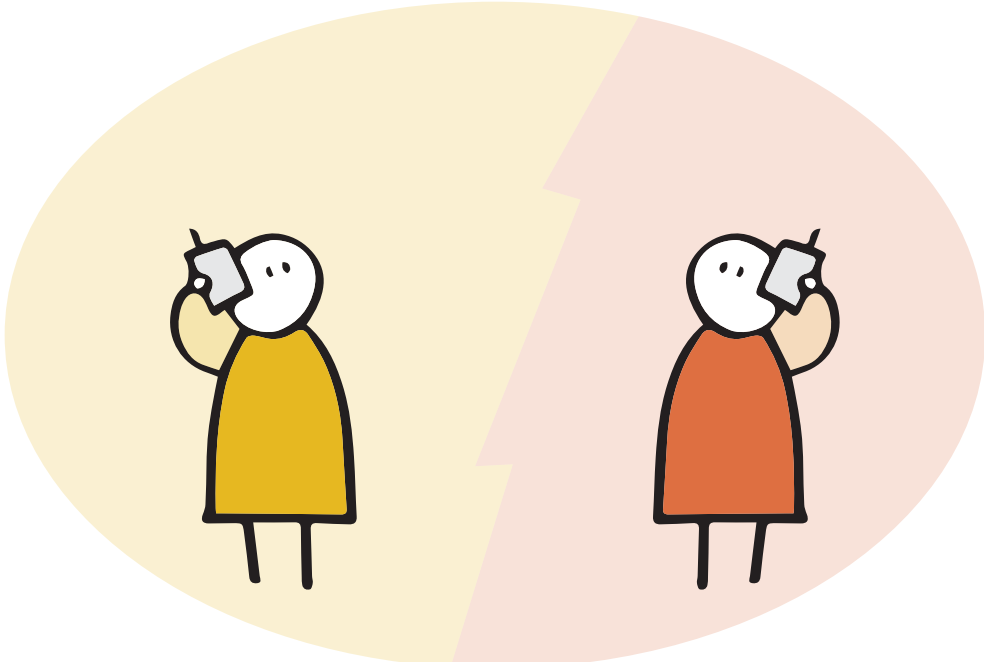
ELENA

Exemplificació gràfica de com funciona una funció recursiva.



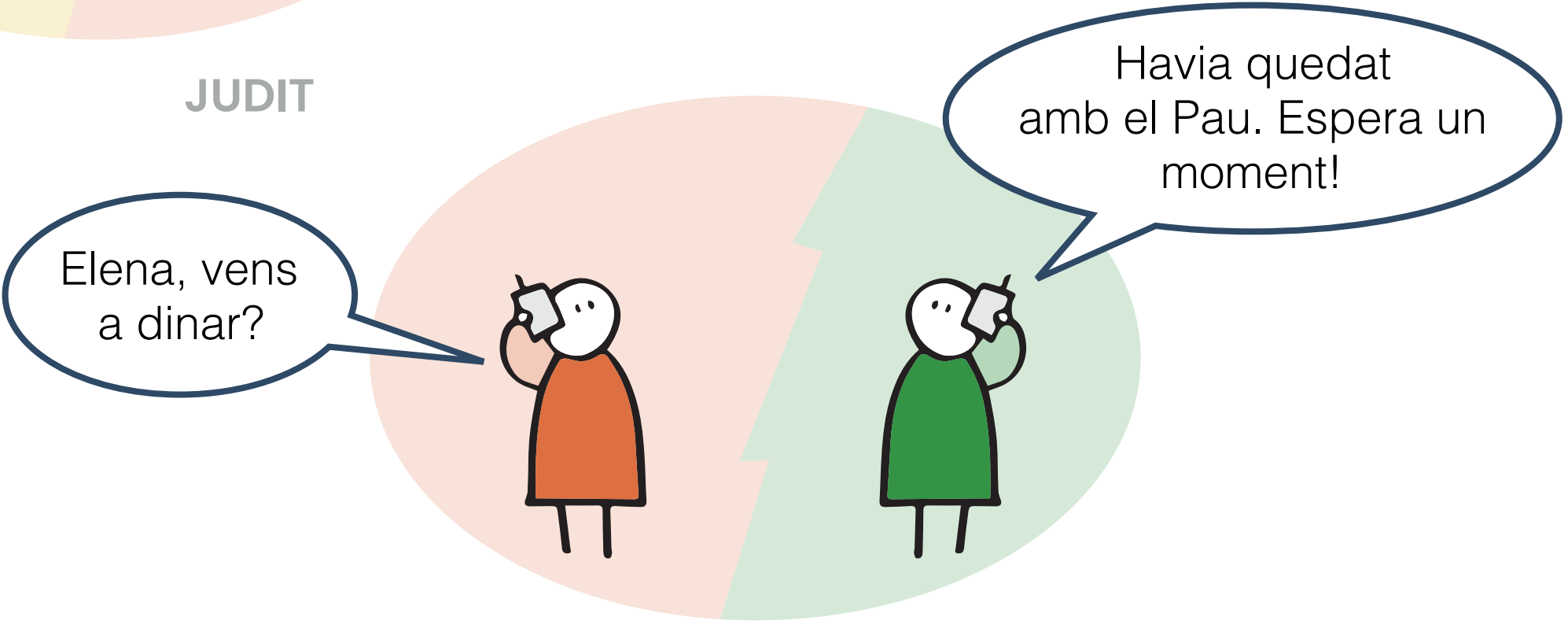
DAVID

VICTOR



VICTOR

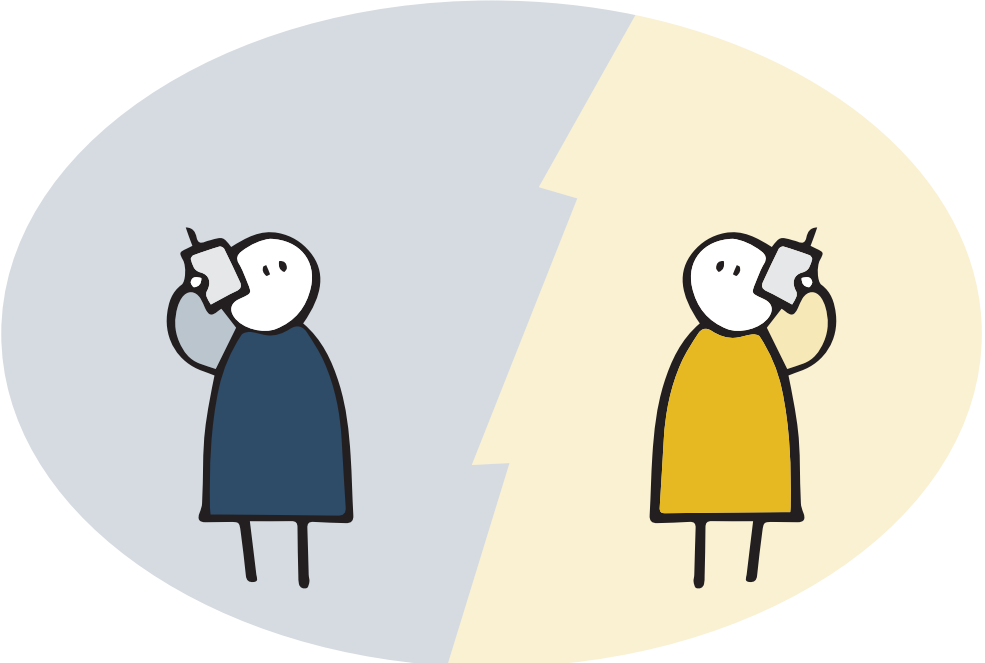
JUDIT



JUDIT

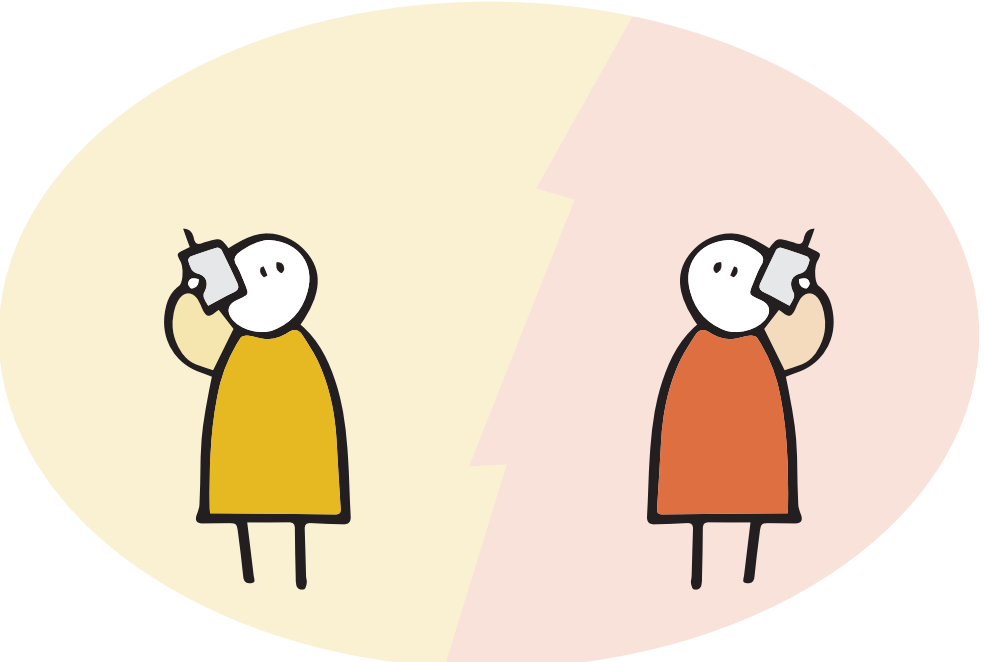
ELENA

Exemplificació gràfica de com funciona una funció recursiva.



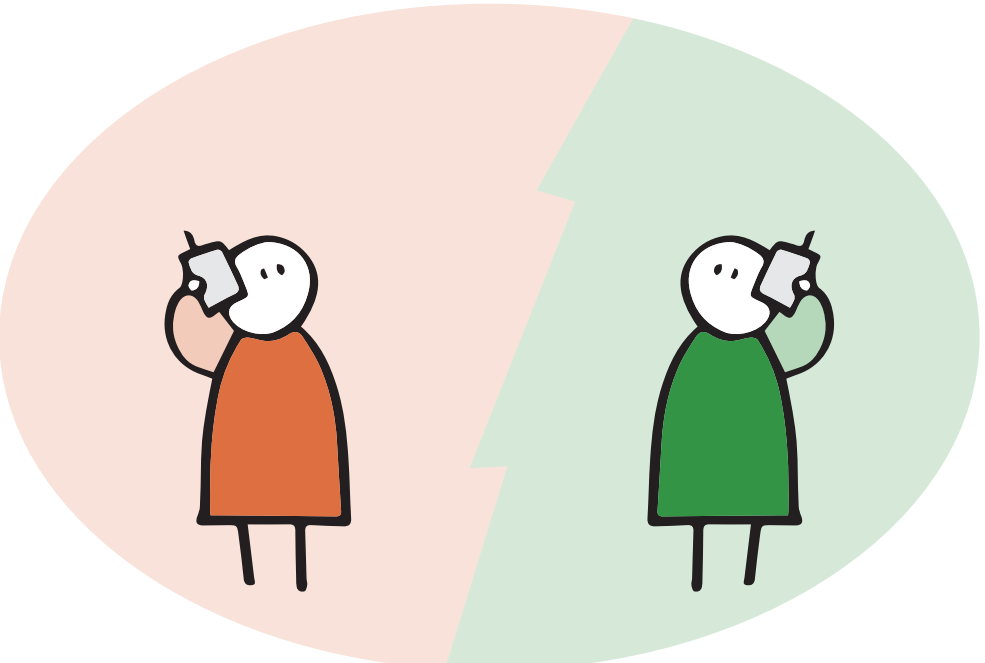
DAVID

VICTOR



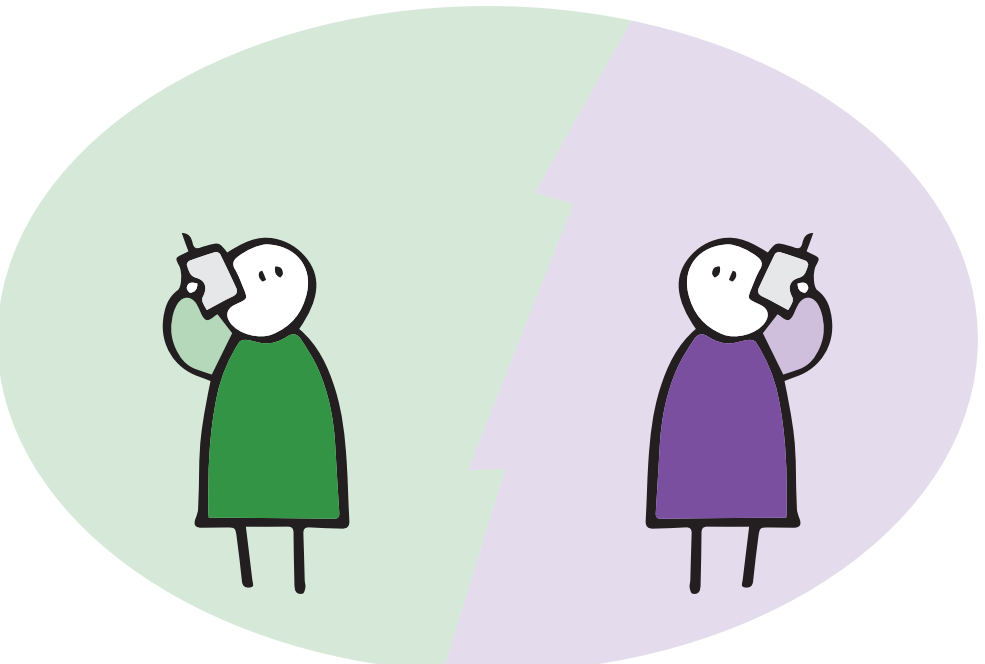
VICTOR

JUDIT



JUDIT

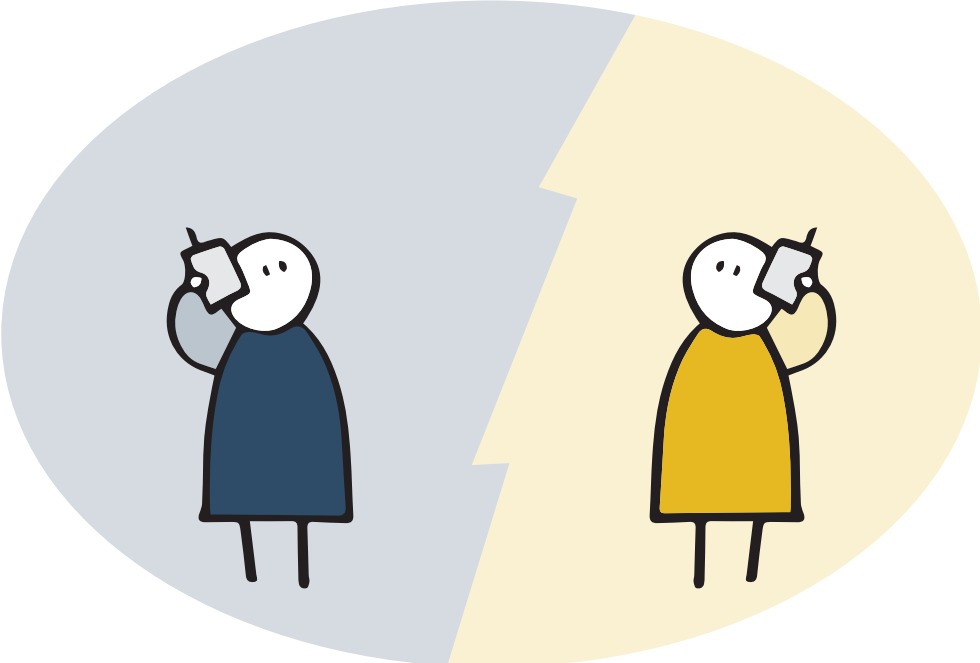
ELENA



ELENA

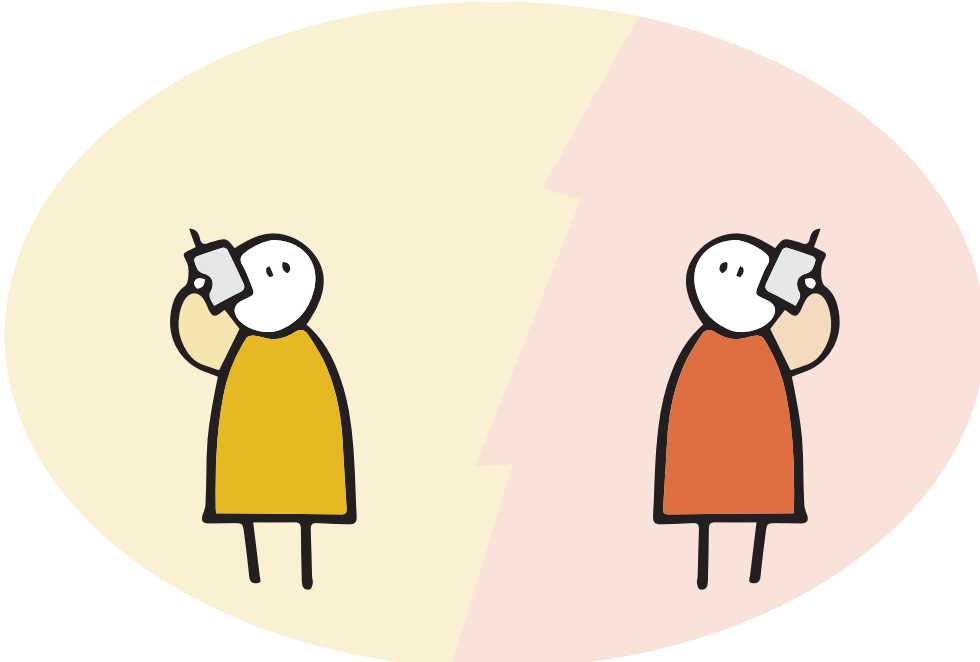
PAU

Exemplificació gràfica de com funciona una funció recursiva.



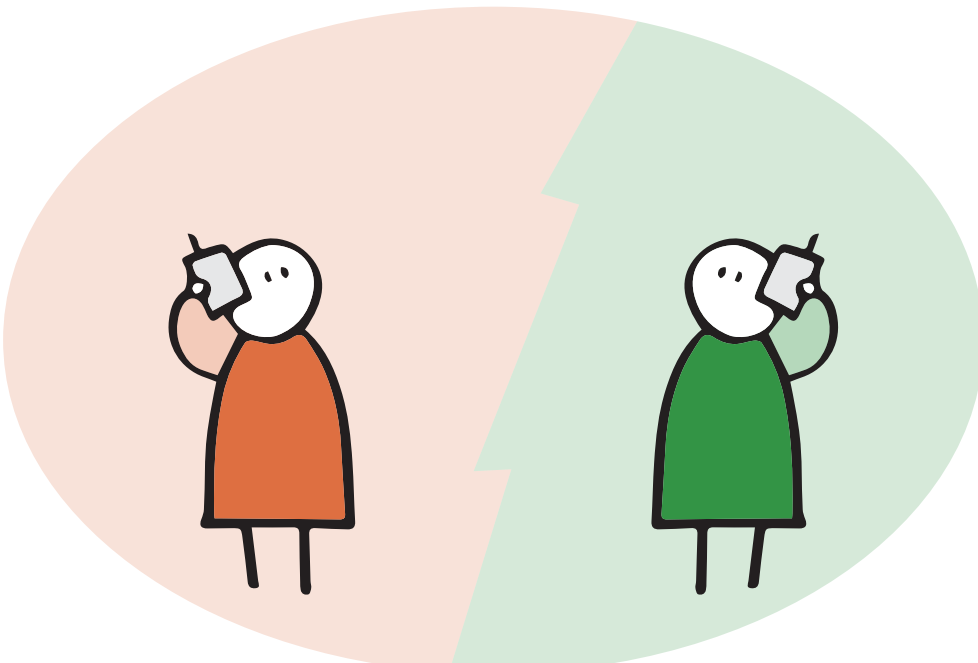
DAVID

VICTOR



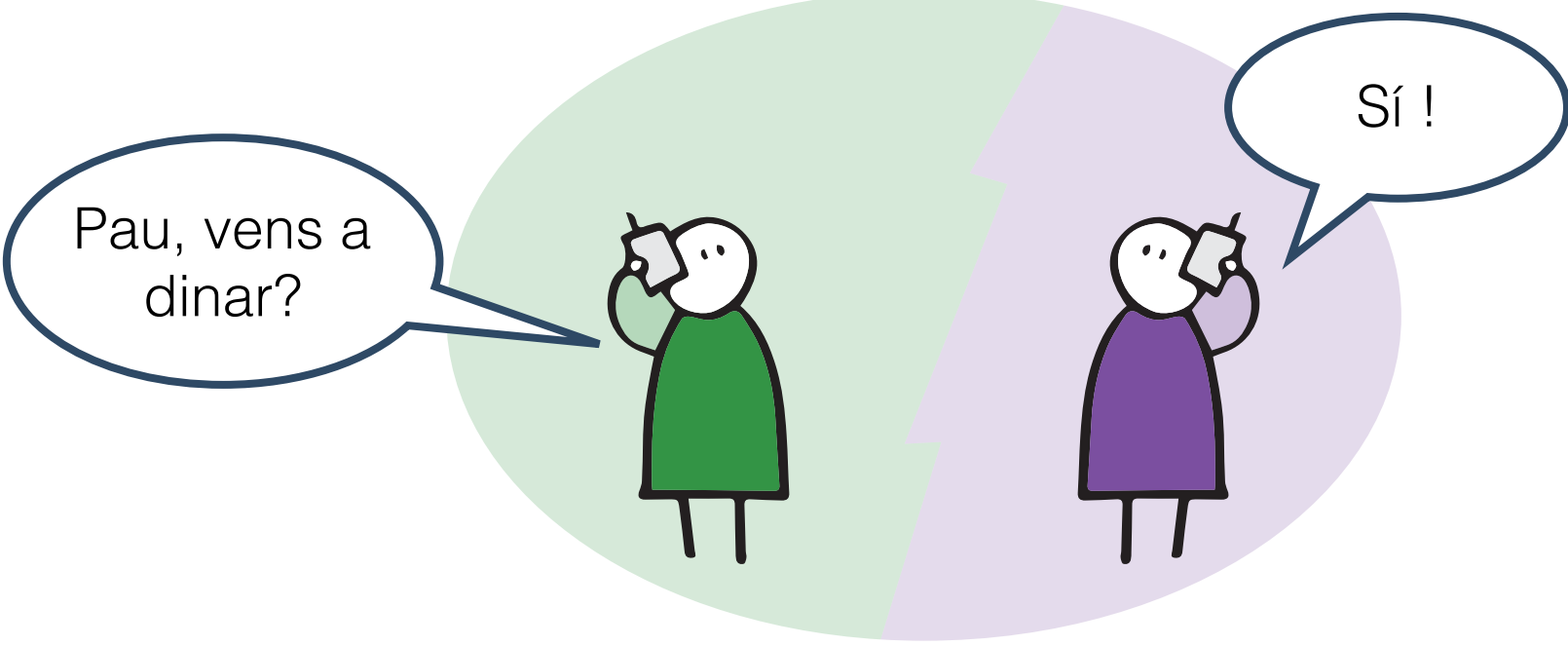
VICTOR

JUDIT



JUDIT

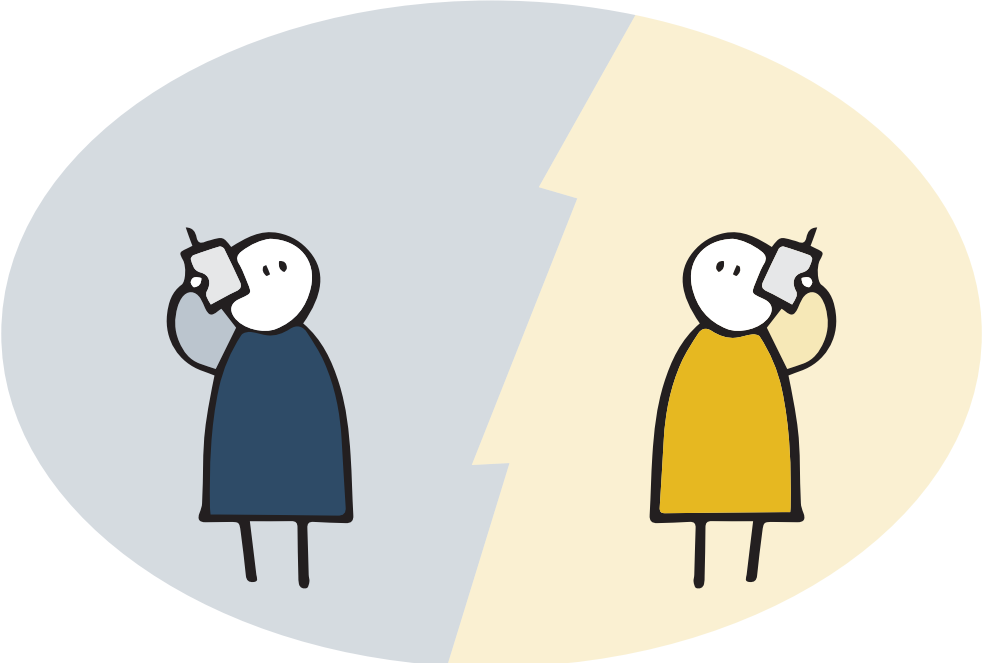
ELENA



ELENA

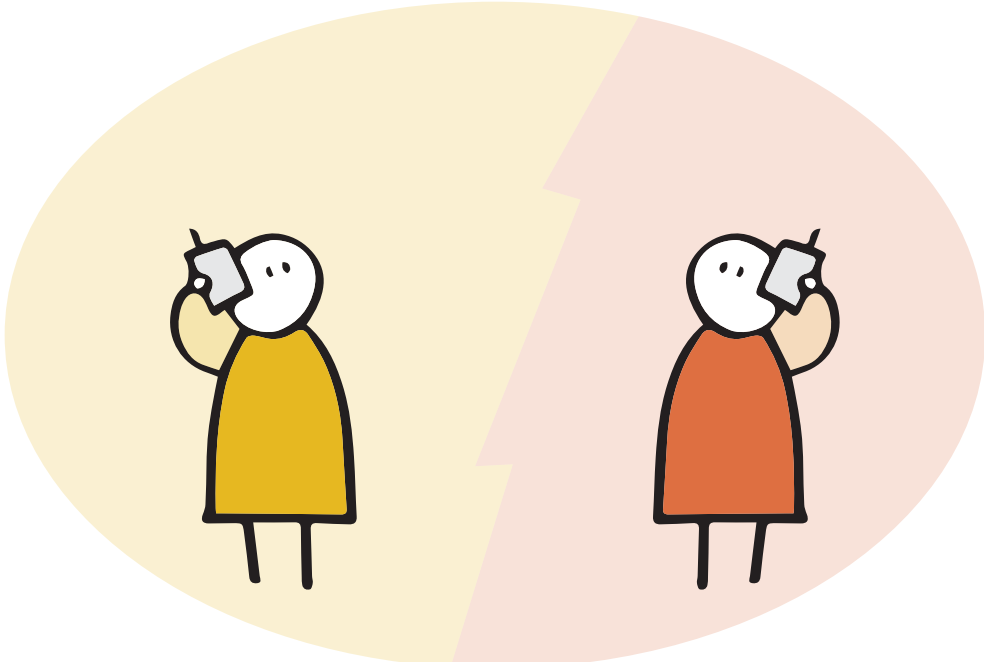
PAU

Exemplificació gràfica de com funciona una funció recursiva.



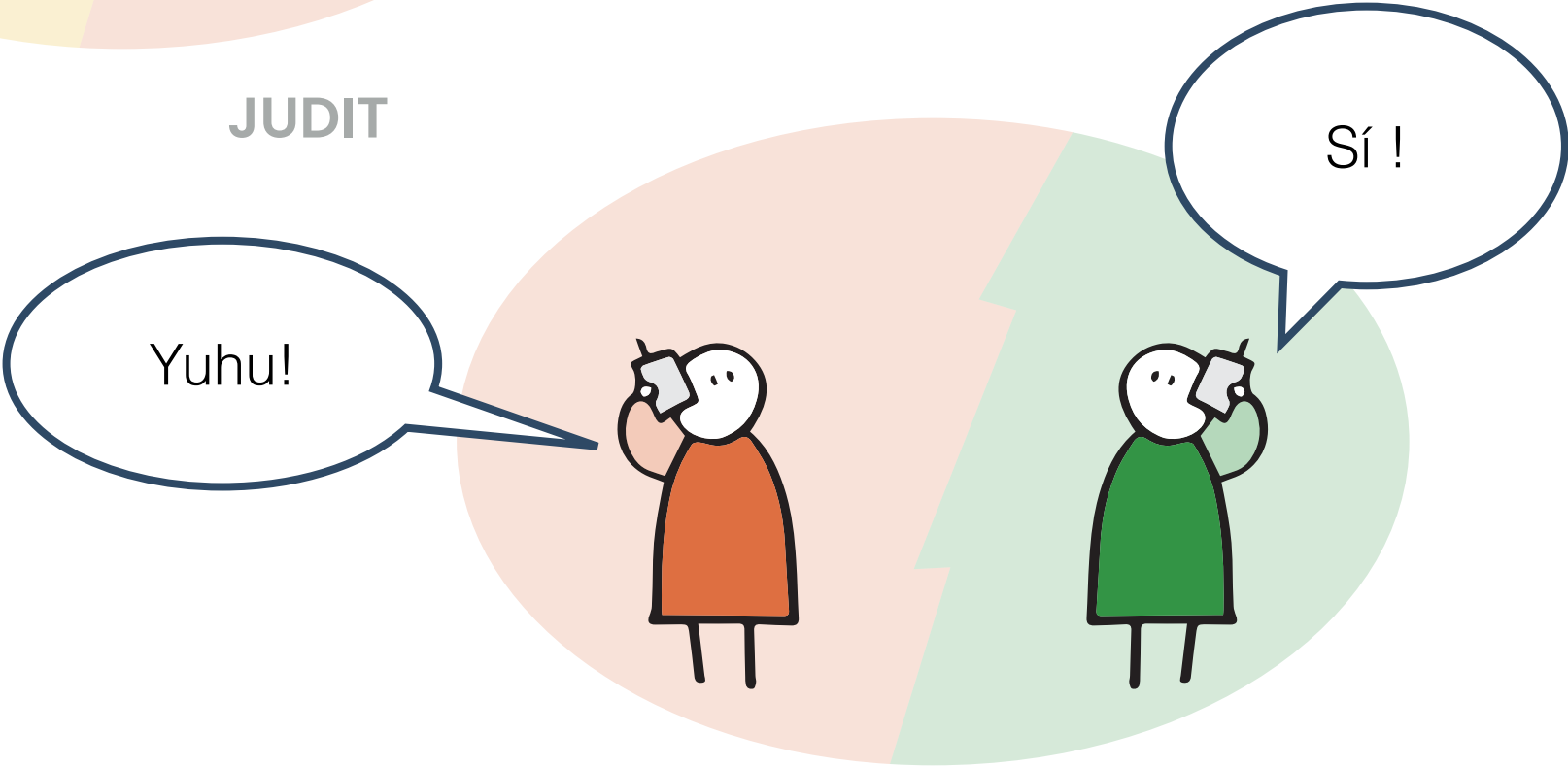
DAVID

VICTOR



VICTOR

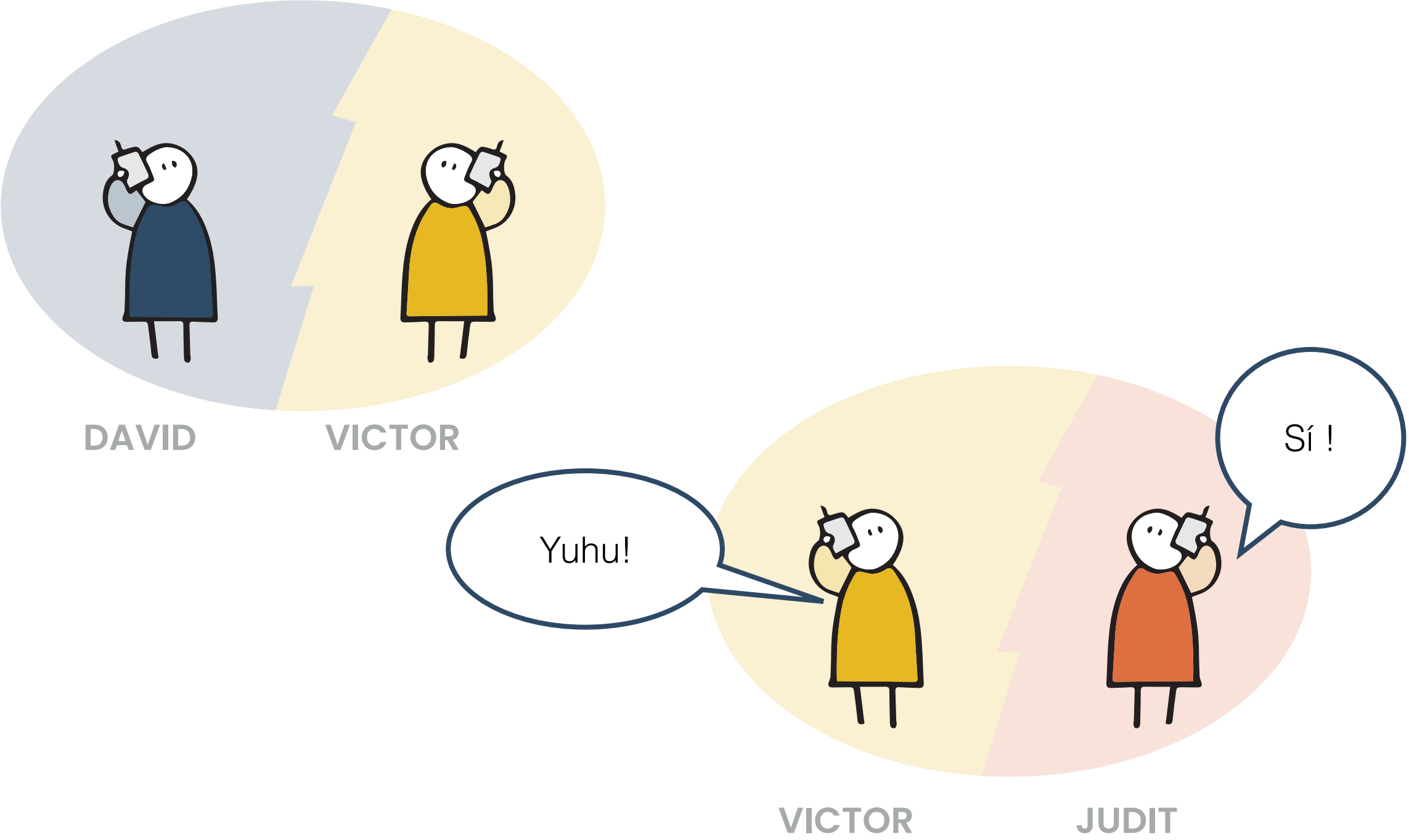
JUDIT

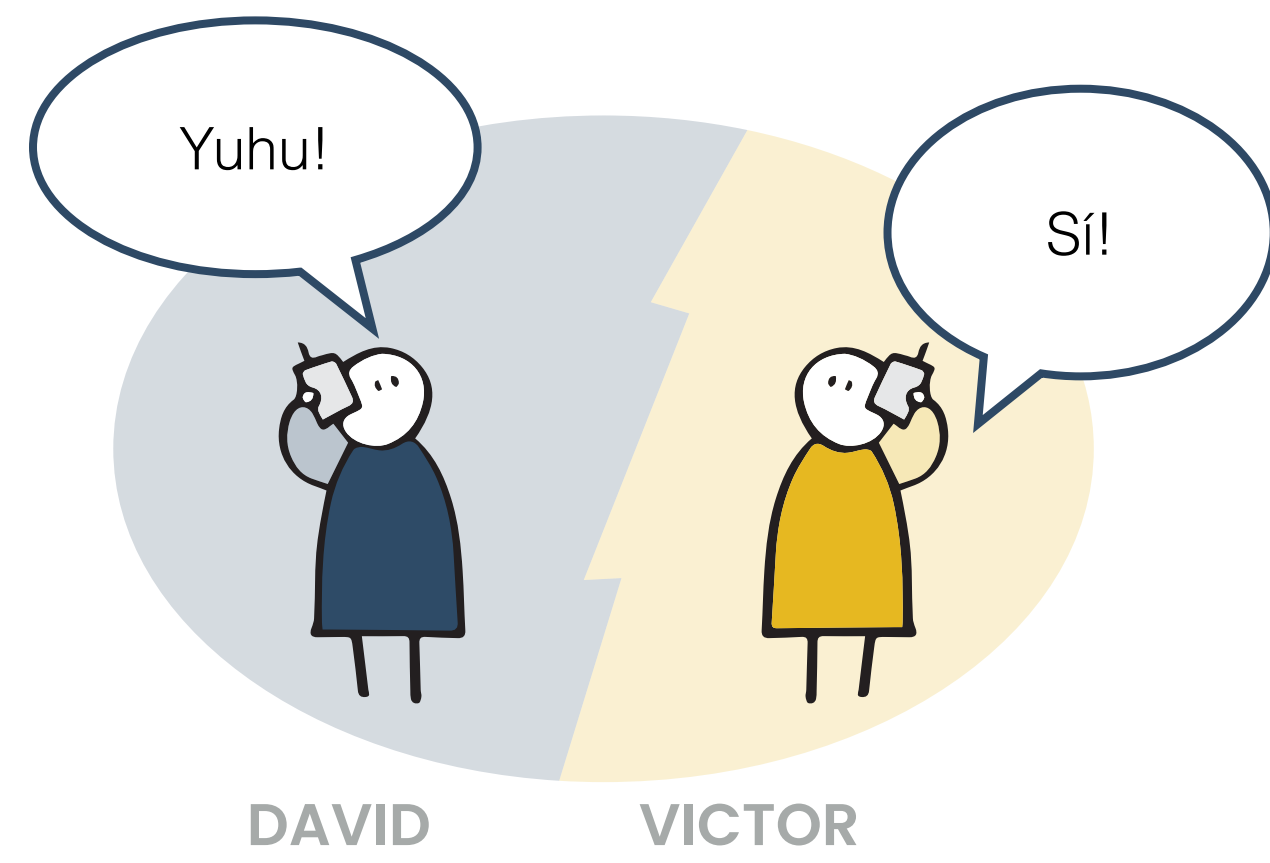


JUDIT

ELENA

Exemplificació gràfica de com funciona una funció recursiva.





Exemplificació gràfica de com funciona una funció recursiva.

Exemplificació gràfica de com funciona una funció recursiva.

Exemple. Càlcul del factorial.

Versió iterativa

```

#include <stdio.h>

/* FACTORIAL FENT SERVIR ITERACIÓ */

int factorial(int n)
{
    int valor = 1;

    for(int i = 2; i <= n; i++){
        valor = valor * i;
    }

    return valor;
}

int main()
{
    int num = 5;
    printf("Factorial = %d\n", factorial(num));
    return 0;
}
```

Exemple. Càlcul del factorial.

Versió iterativa

```
#include <stdio.h>

/* FACTORIAL FENT SERVIR ITERACIÓ */

int factorial(int n)
{
    int valor = 1;

    for(int i = 2; i <= n; i++){
        valor = valor * i;
    }

    return valor;
}

int main()
{
    int num = 5;
    printf("Factorial = %d\n", factorial(num));
    return 0;
}
```

Versió recursiva

```
#include <stdio.h>

/* FACTORIAL VERSIO RECURSIVA */

int factorial(int n)
{
    int valor;
    if (n == 0){
        valor = 1;
    }
    else {
        valor = n * factorial(n-1);
    }
    return valor;
}

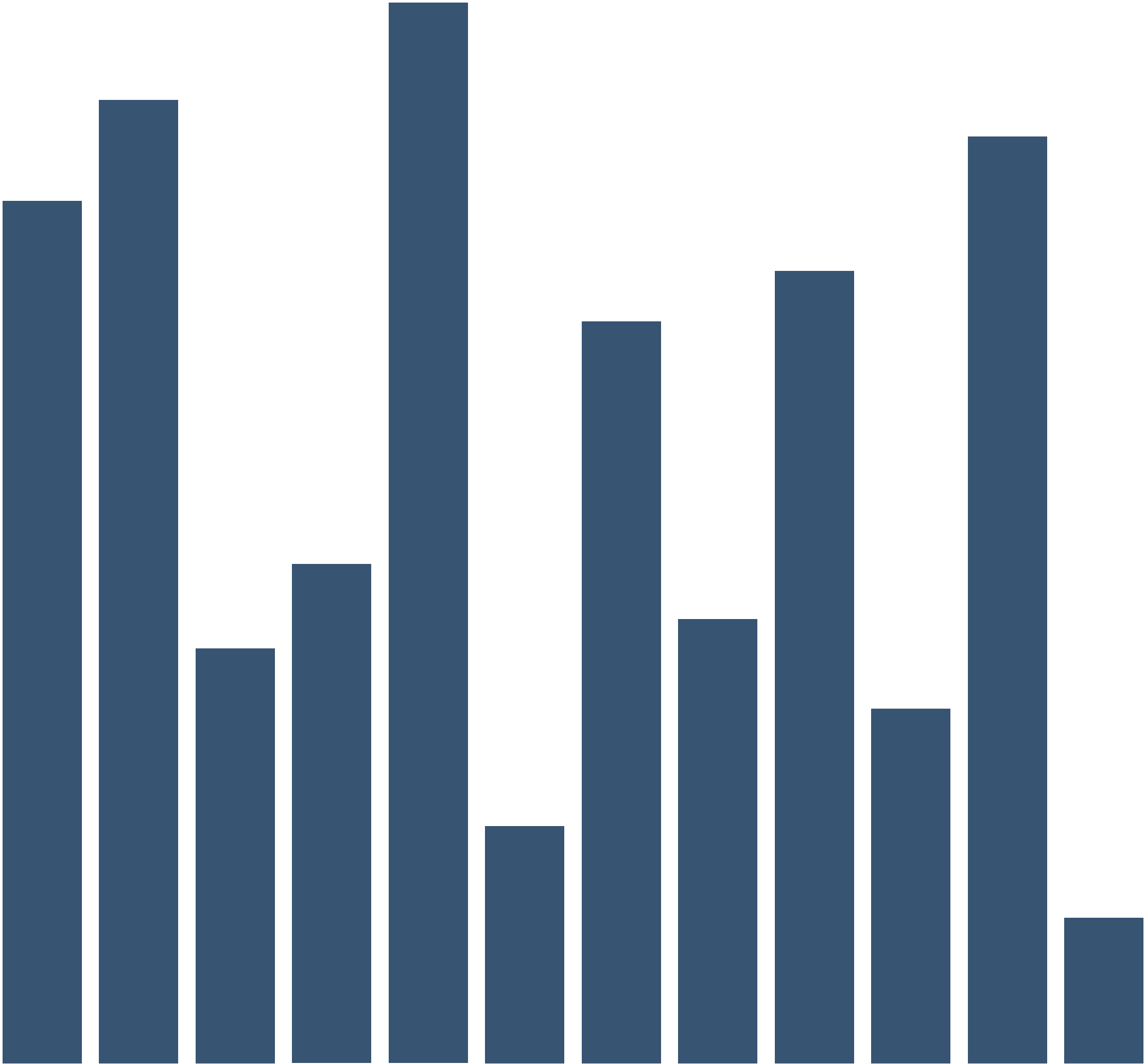
int main()
{
    int num = 5;
    printf("Factorial = %d\n", factorial(num));
    return 0;
}
```

Quicksort

- Quicksort és un algorisme d'ordenació creat per Tony Hoare al 1959 i publicat el 1961 que, a dia d'avui, és un dels algorismes d'ordenació més utilitzats.
- És un algorisme de tipus “divideix i venceràs” (aquesta filosofia es basa en dividir el problema en trossos més petits que es puguin solucionar) i és un algorisme **recursiu**.
- **Funcionament:**
 - Es selecciona un element de la taula i s'anomena “pivot”. Aquesta tria es pot fer aleatòriament o triant el valor que sabem que és el “central”.
 - Llavors, la resta d'elements de la taula, es divideixen entre aquells que són més grans que el pivot, i aquells que són més petits que el pivot.
 - Per cadascun d'aquests dos trossos, es torna a fer el mateix procés (recursivament).
 - Les implementacions eficients de Quicksort no proporcionen ordenació estable. (L'ordre dels elements iguals no es preserva).
- **Costos:**
 - Pitjor cas: $O(n^2)$
 - Cas average: $O(n \log n)$

Quicksort

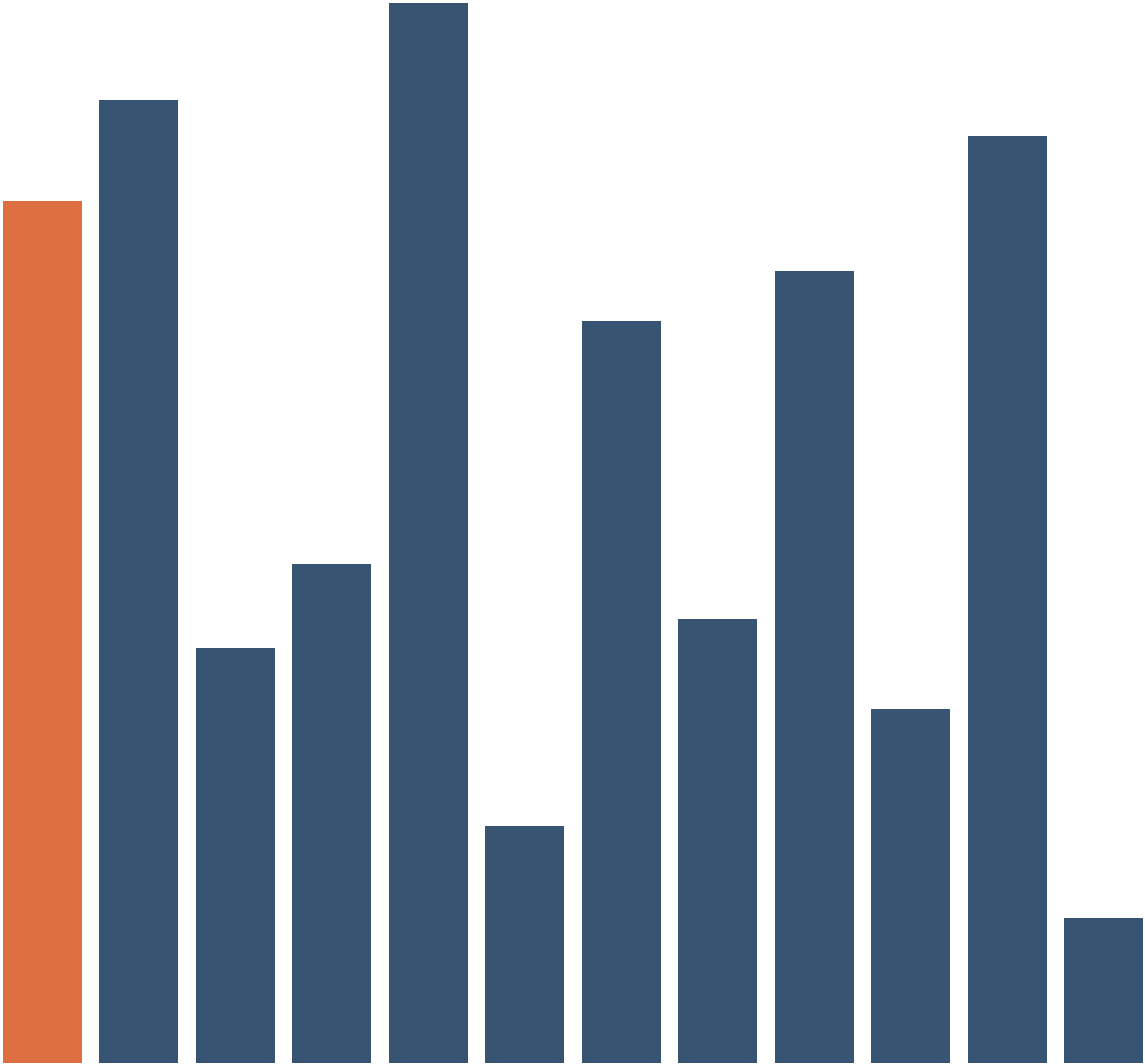
Funcionament



Quicksort

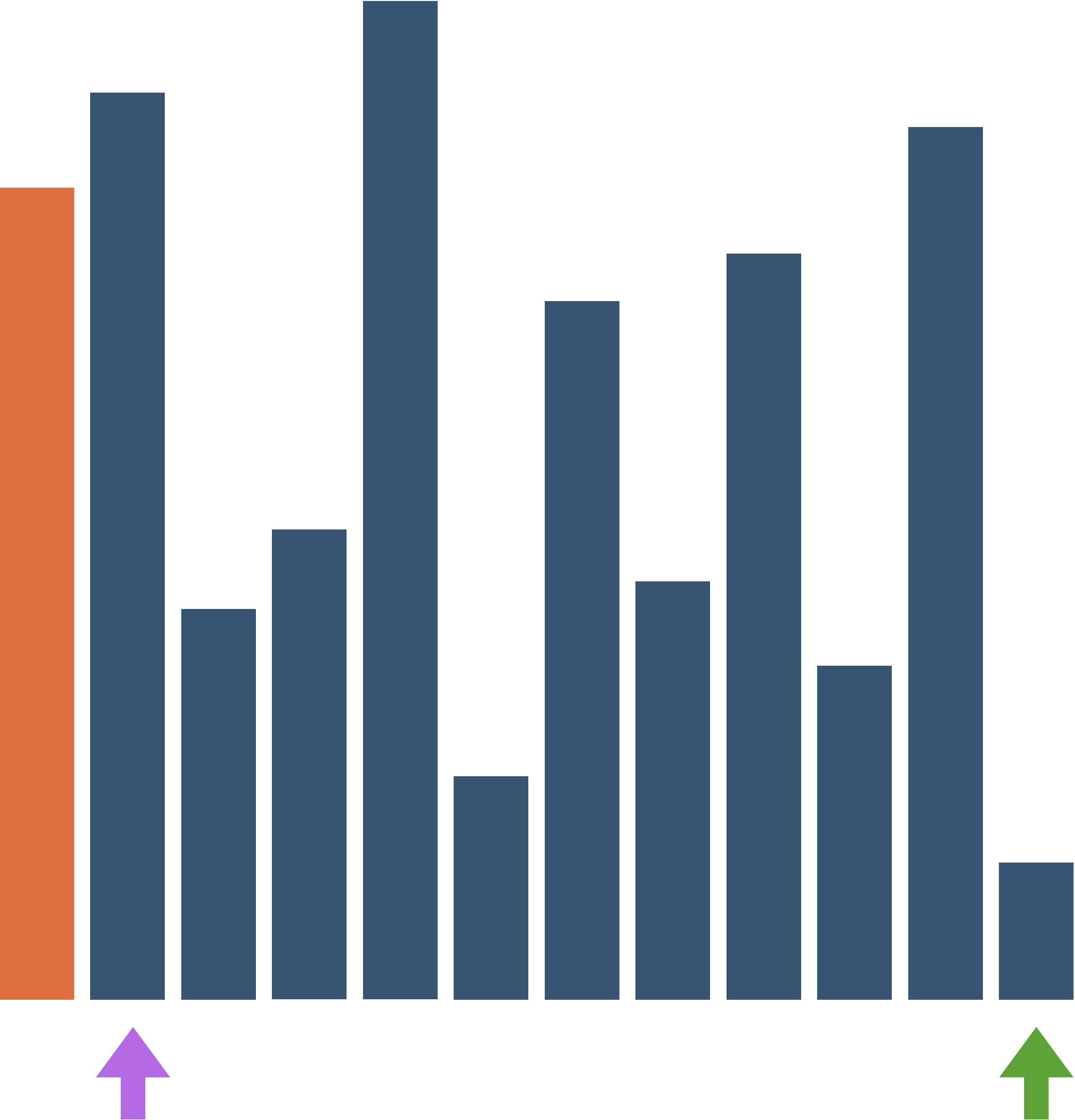
Funcionament

OBJECTIU: Els que són més petits que el pivot, a l'esquerra.
Els que són més grans que el pivot, a la dreta.



Quicksort

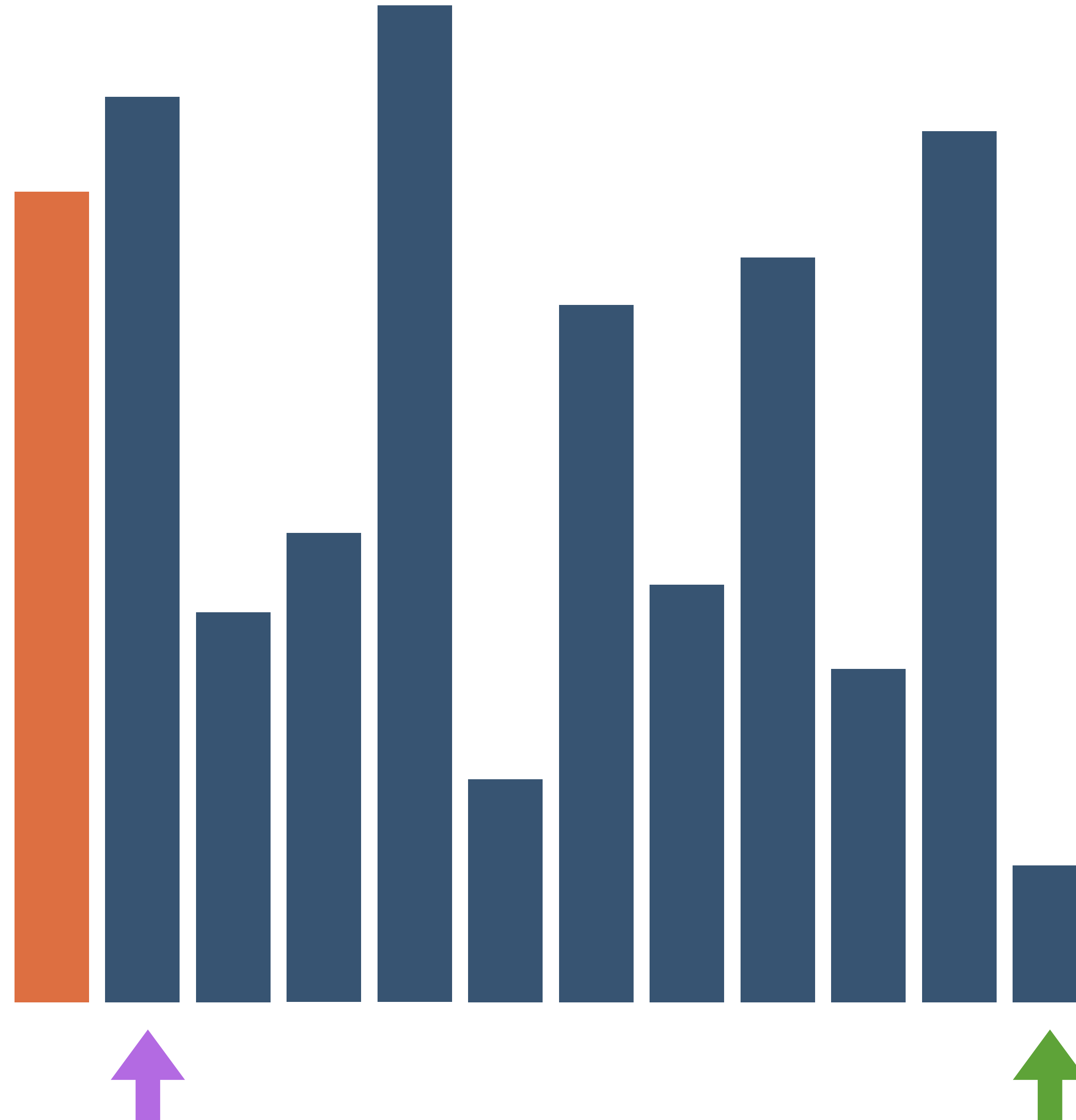
Funcionament



Quicksort

Funcionament

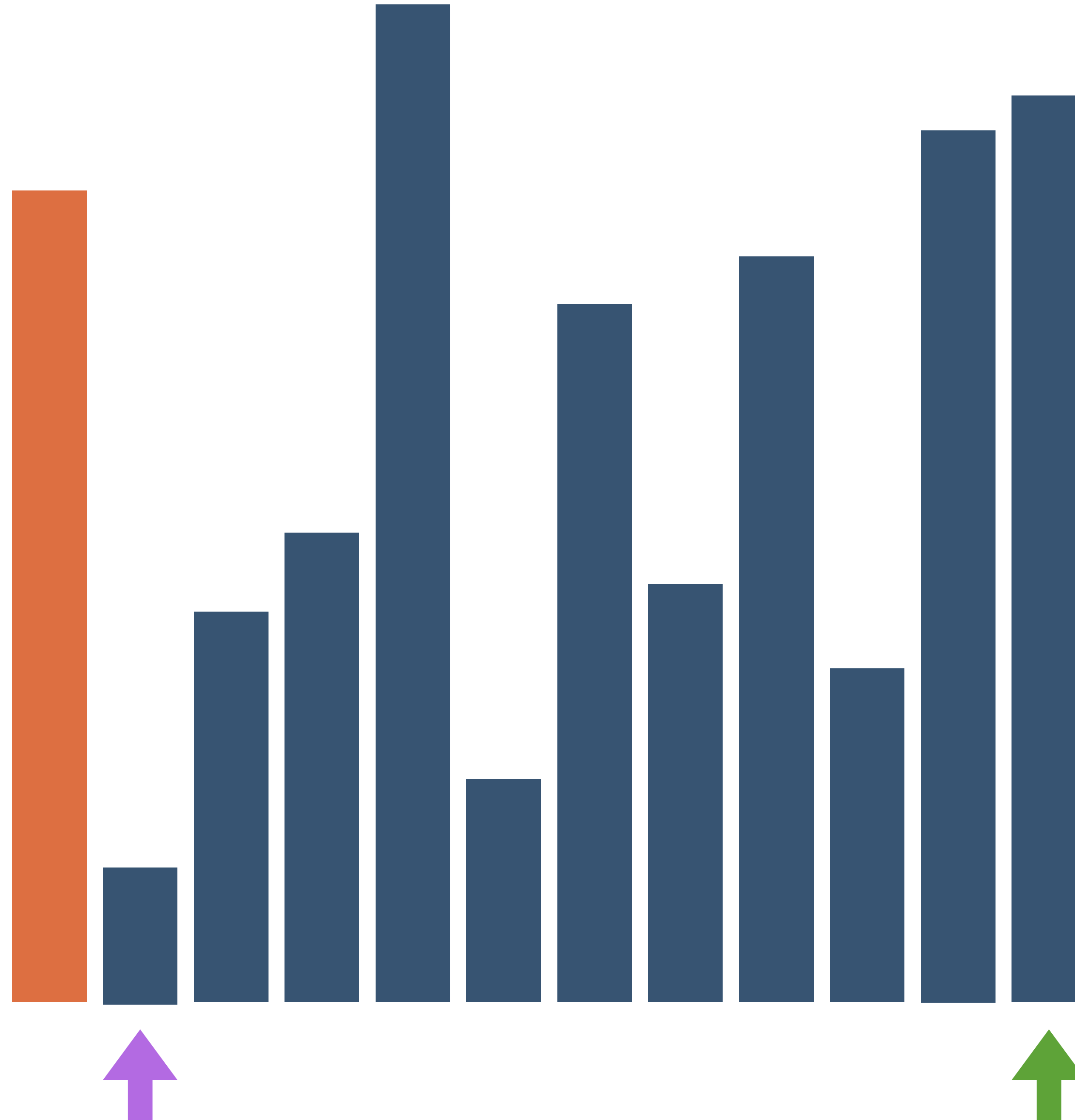
Si el de l'esquerra és més gran que el pivot AND el de la dreta és més petit que el pivot, intercanviem posicions.



Quicksort

Funcionament

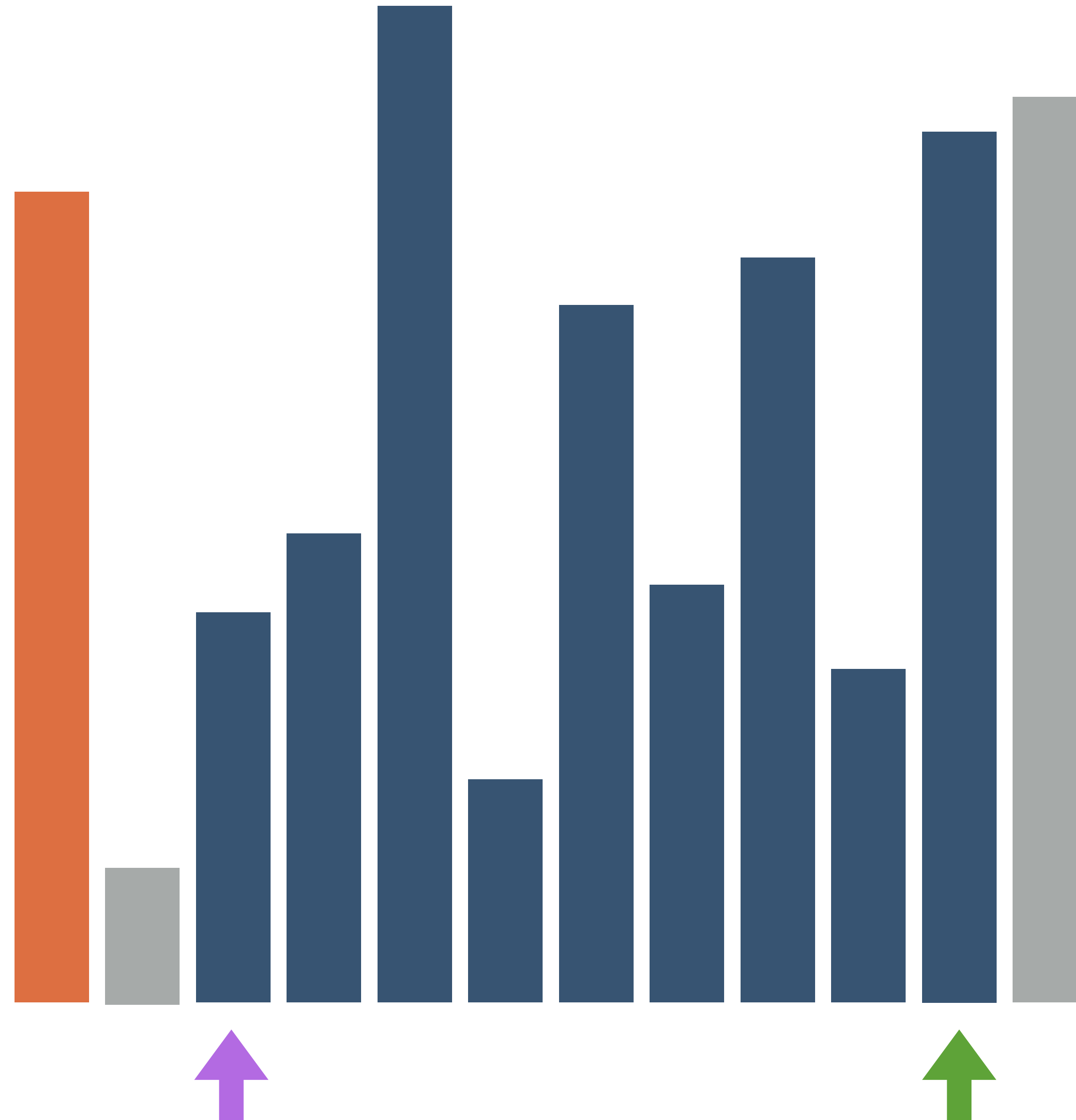
Si el de l'esquerra és més gran que el pivot AND el de la dreta és més petit que el pivot, intercanviem posicions.



Quicksort

Funcionament

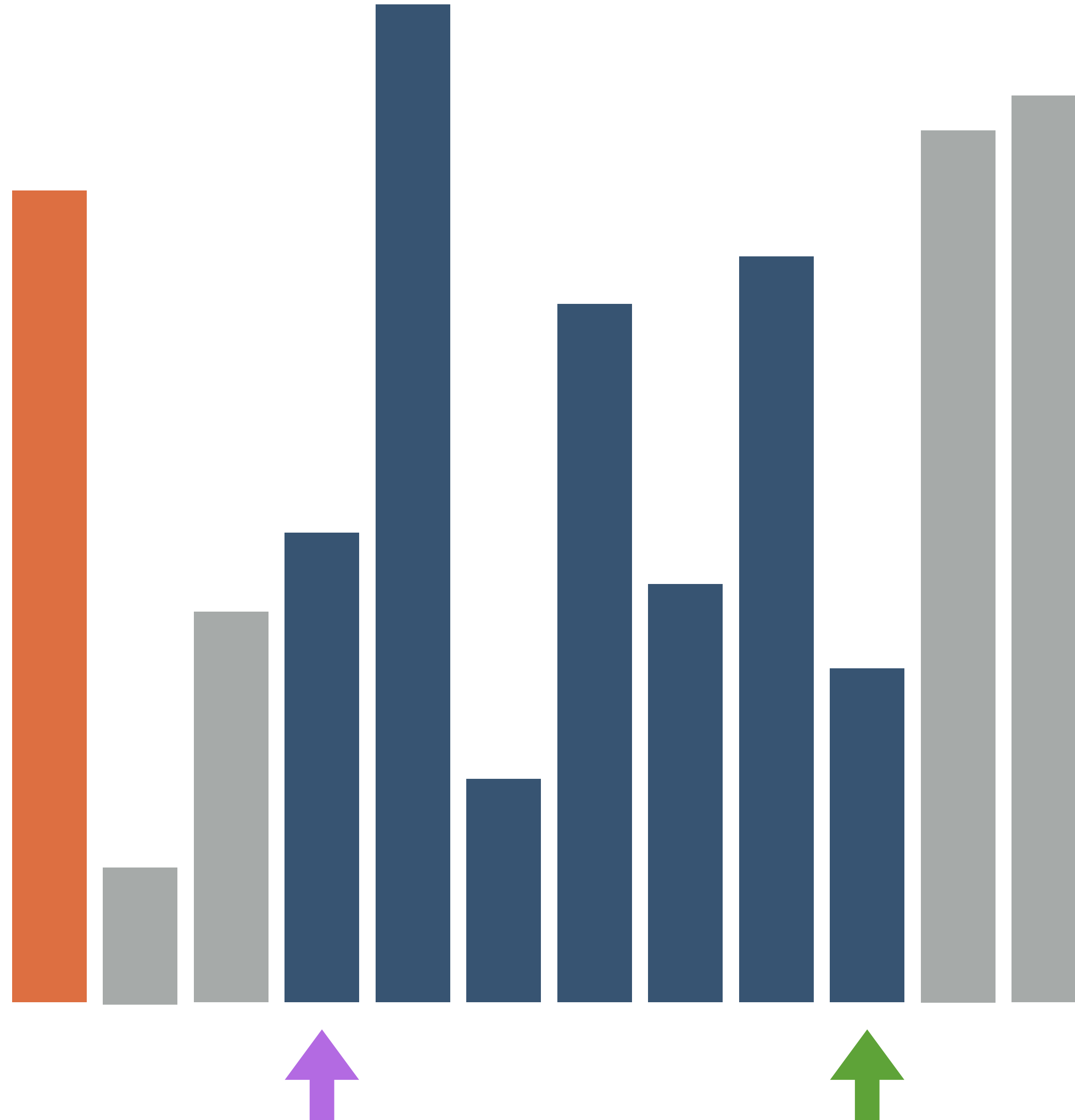
Si el de l'esquerra és més petit que el pivot, i el de la dreta és més gran, tots dos estan bé. Deixem com estan i avancem.



Quicksort

Funcionament

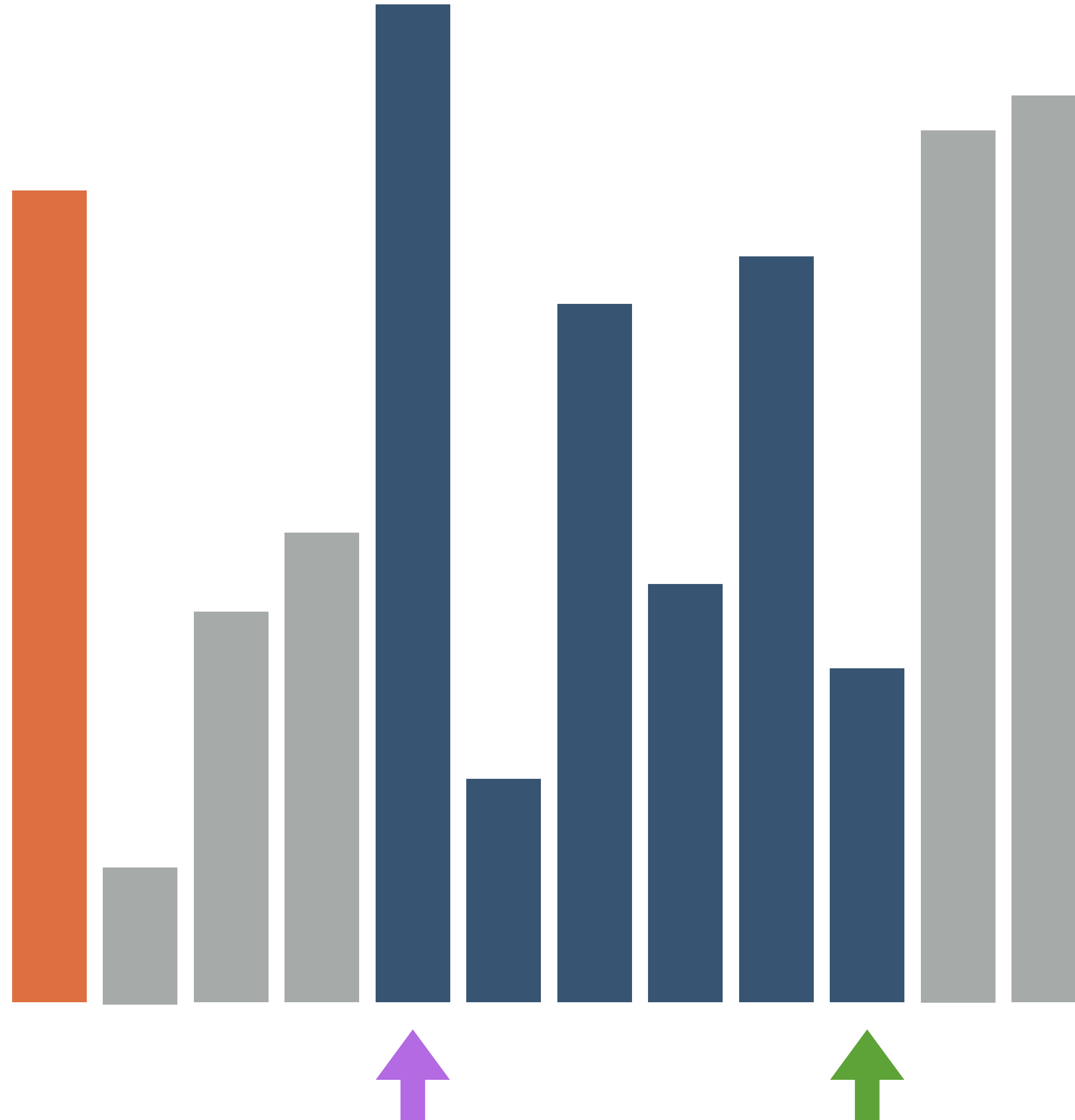
Si el de l'esquerra és més petit que el pivot, el deixem com està, està ben col·locat.
Si el de la dreta també és més petit, mantenim el punter fins que trobem algú amb qui intercanviar.



Quicksort

Funcionament

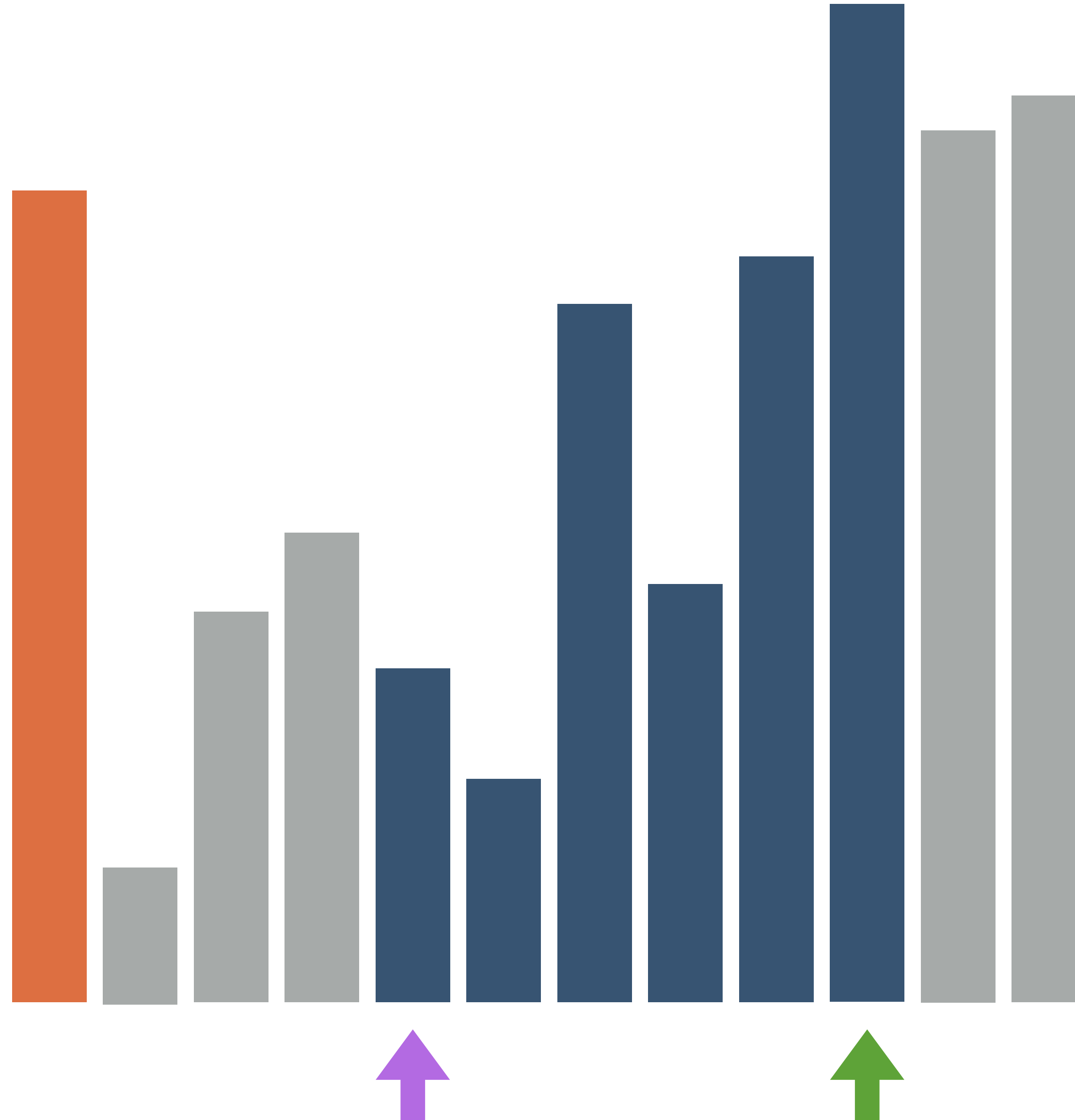
Si el de l'esquerra és més petit que el pivot, el deixem com està, està ben col·locat.
Si el de la dreta també és més petit, mantenim el punter fins que trobem algú amb qui intercanviar.



Quicksort

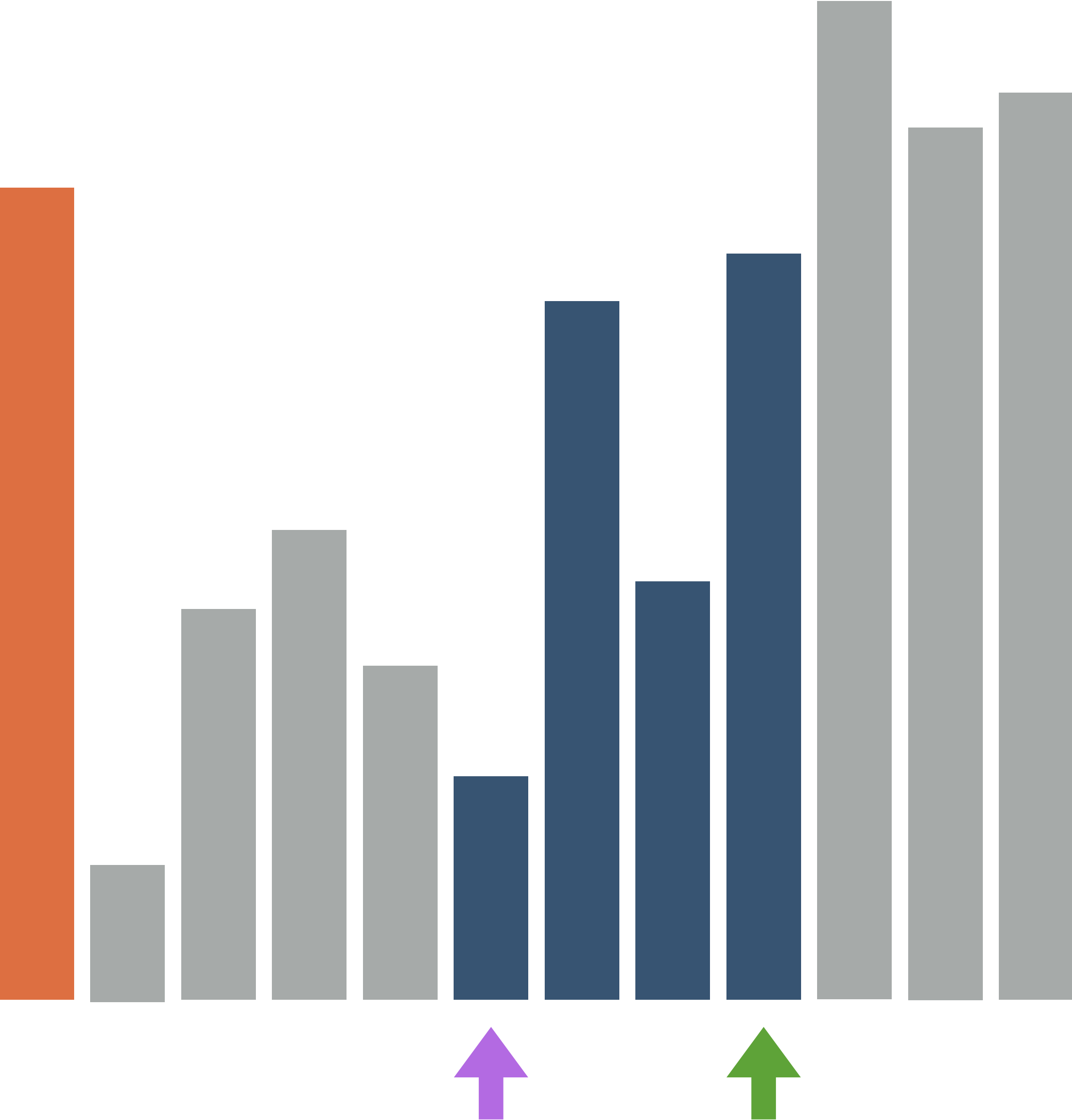
Funcionament

Si el de l'esquerra és més petit que el pivot, el deixem com està, està ben col·locat.
Si el de la dreta també és més petit, mantenim el punter fins que trobem algú amb qui intercanviar.



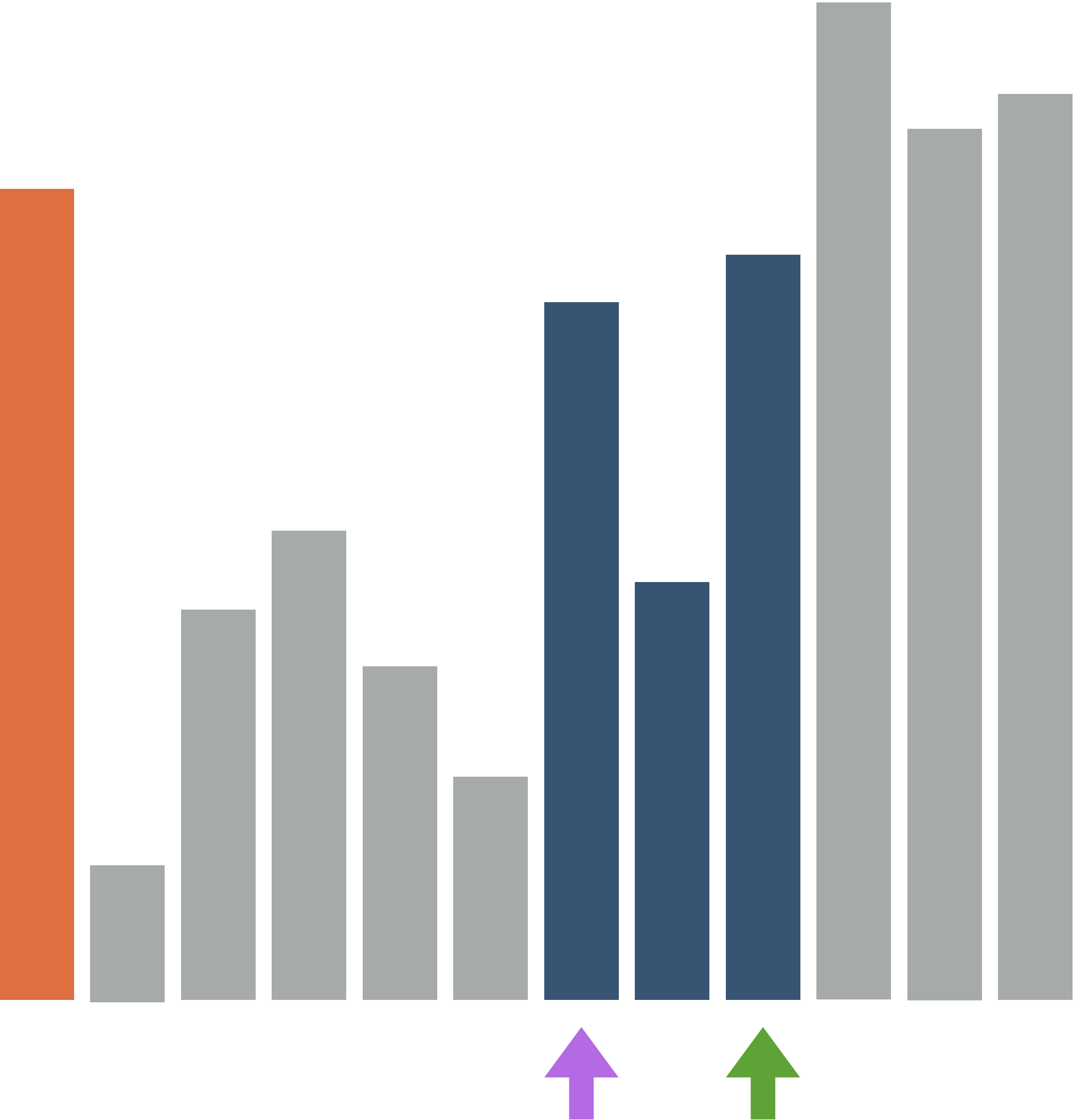
Quicksort

Funcionament



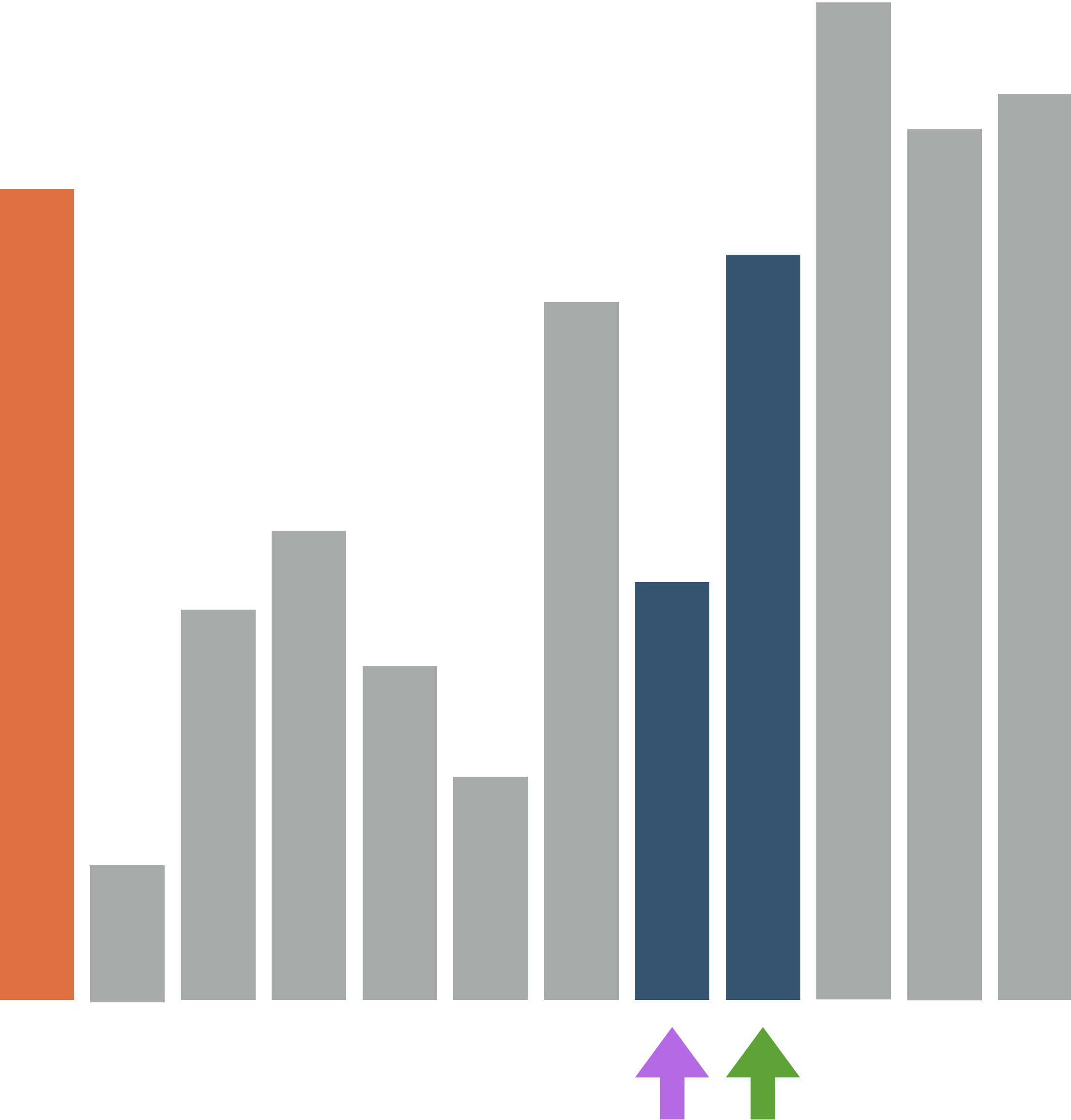
Quicksort

Funcionament



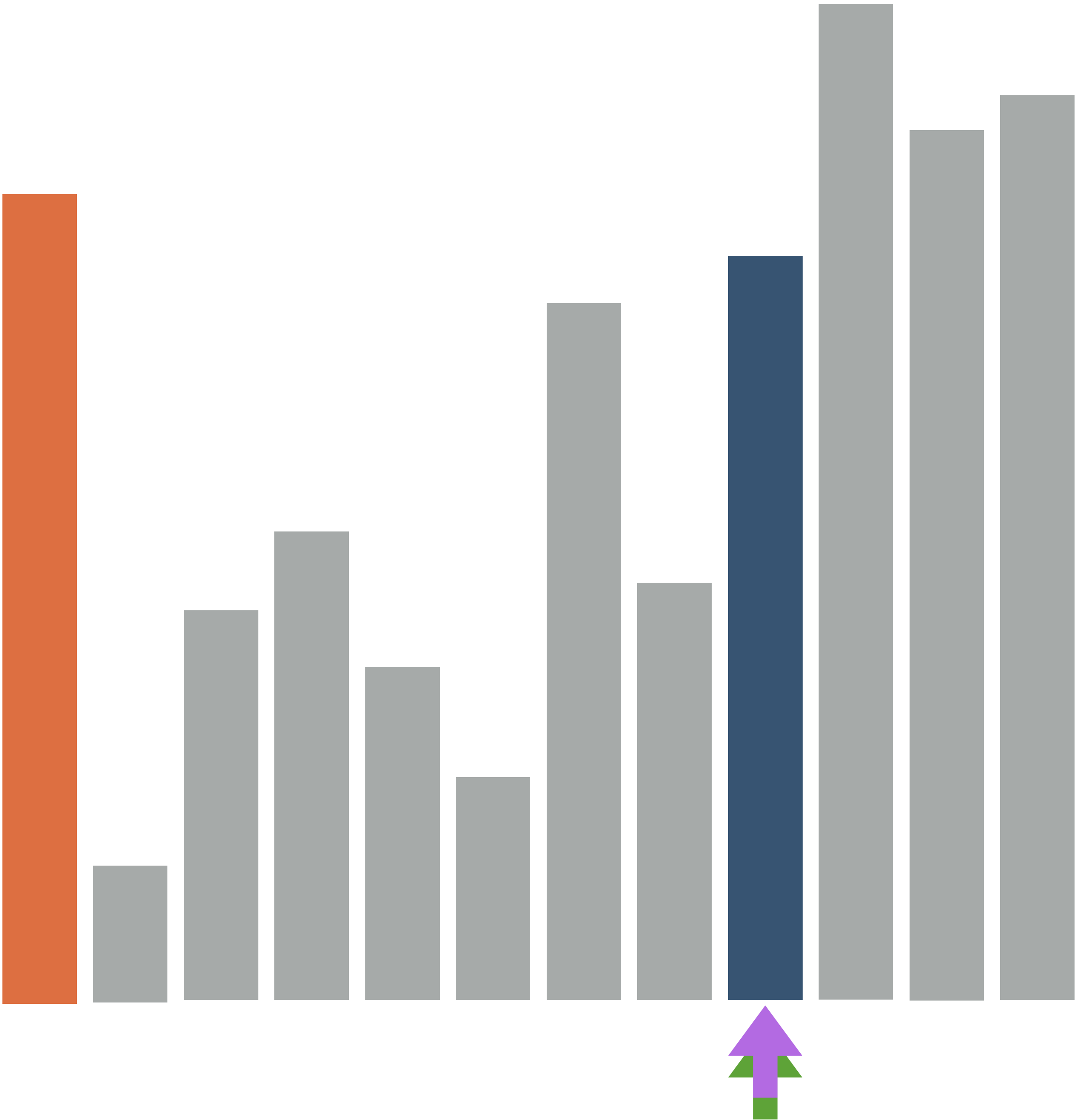
Quicksort

Funcionament



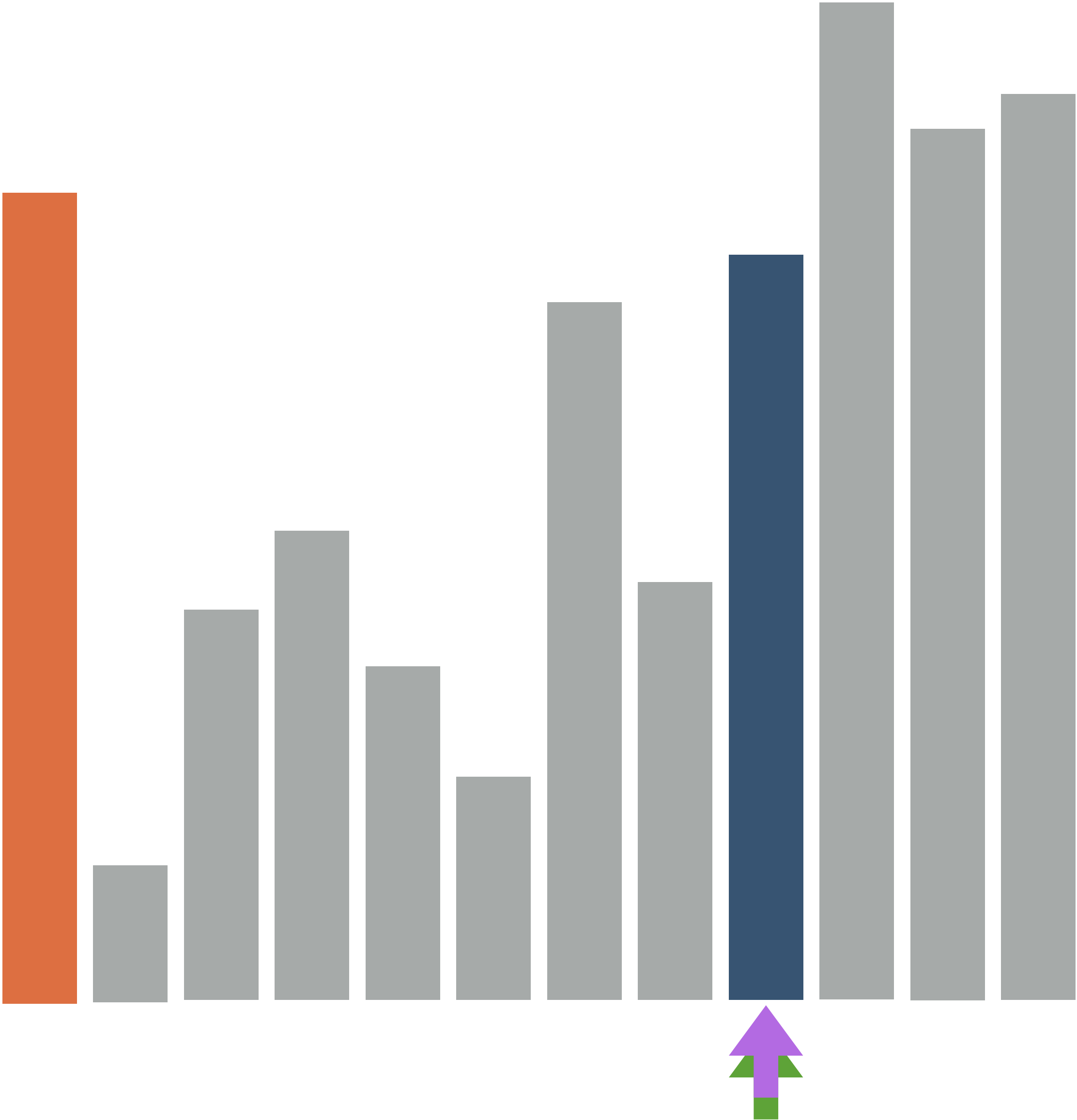
Quicksort

Funcionament



Quicksort

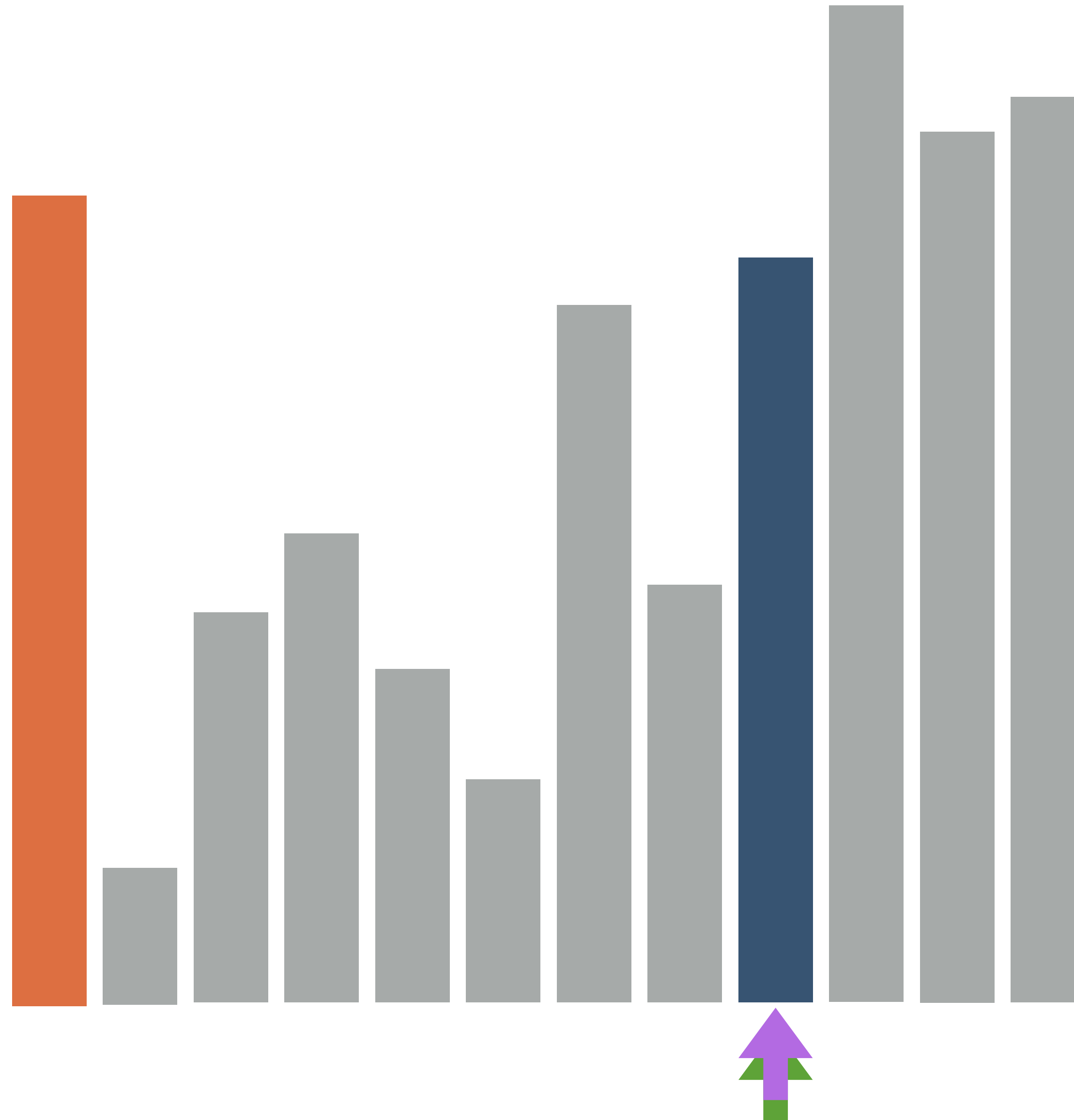
Funcionament



Quicksort

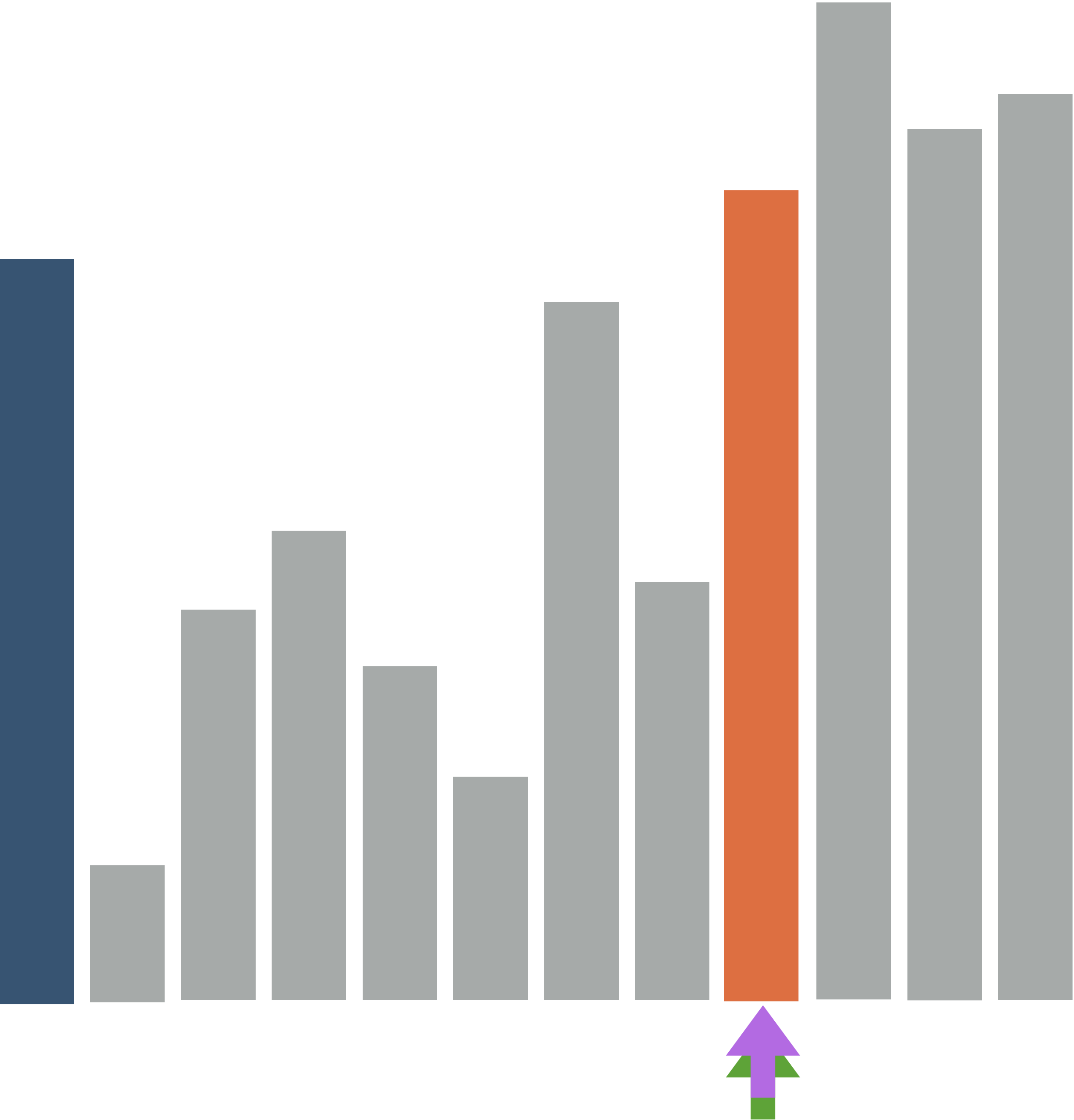
Funcionament

Si el valor al punt de creuament és més petit que el pivot, els intercanvies.
Si no ho és, ho deixes com està.



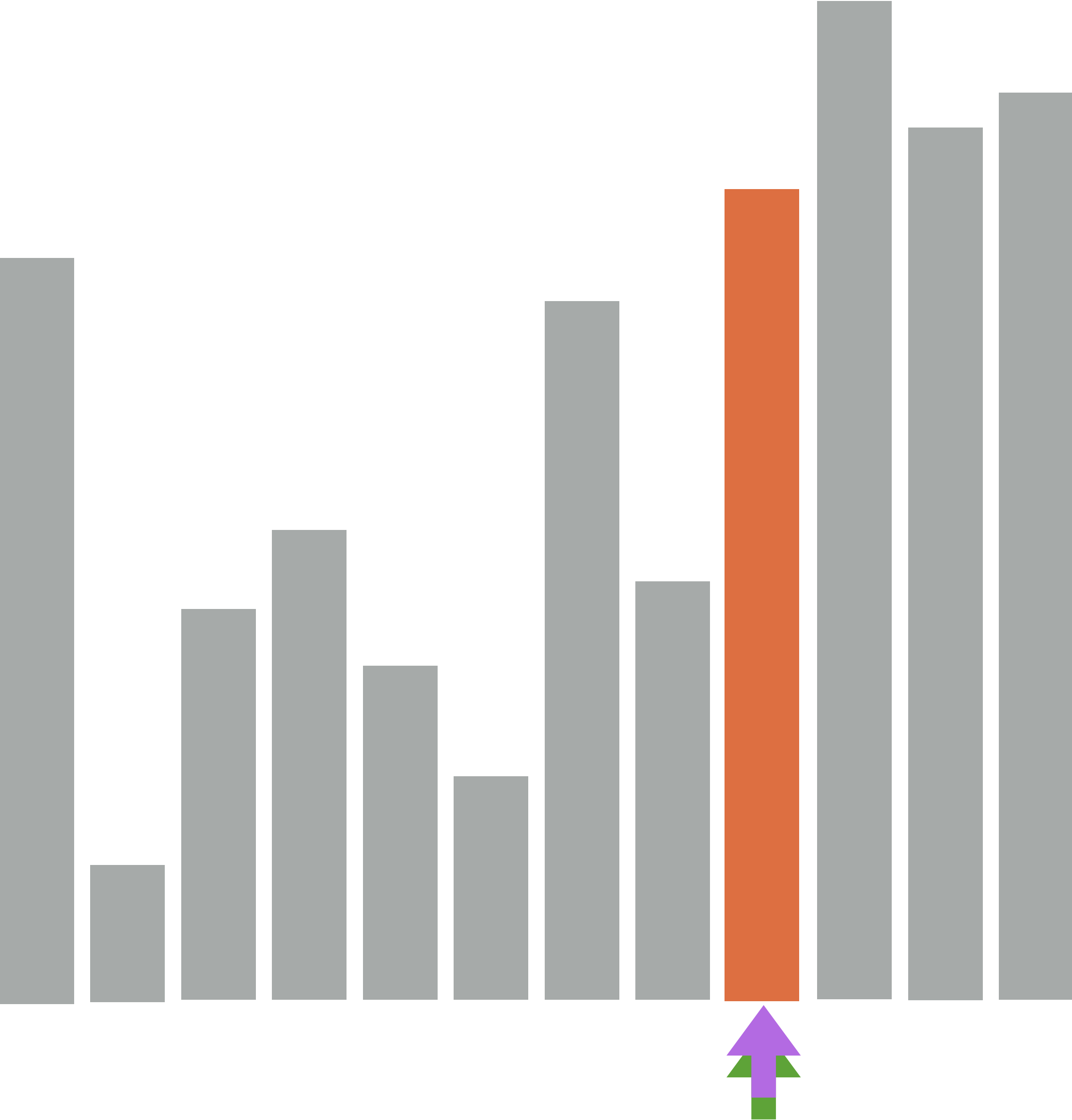
Quicksort

Funcionament



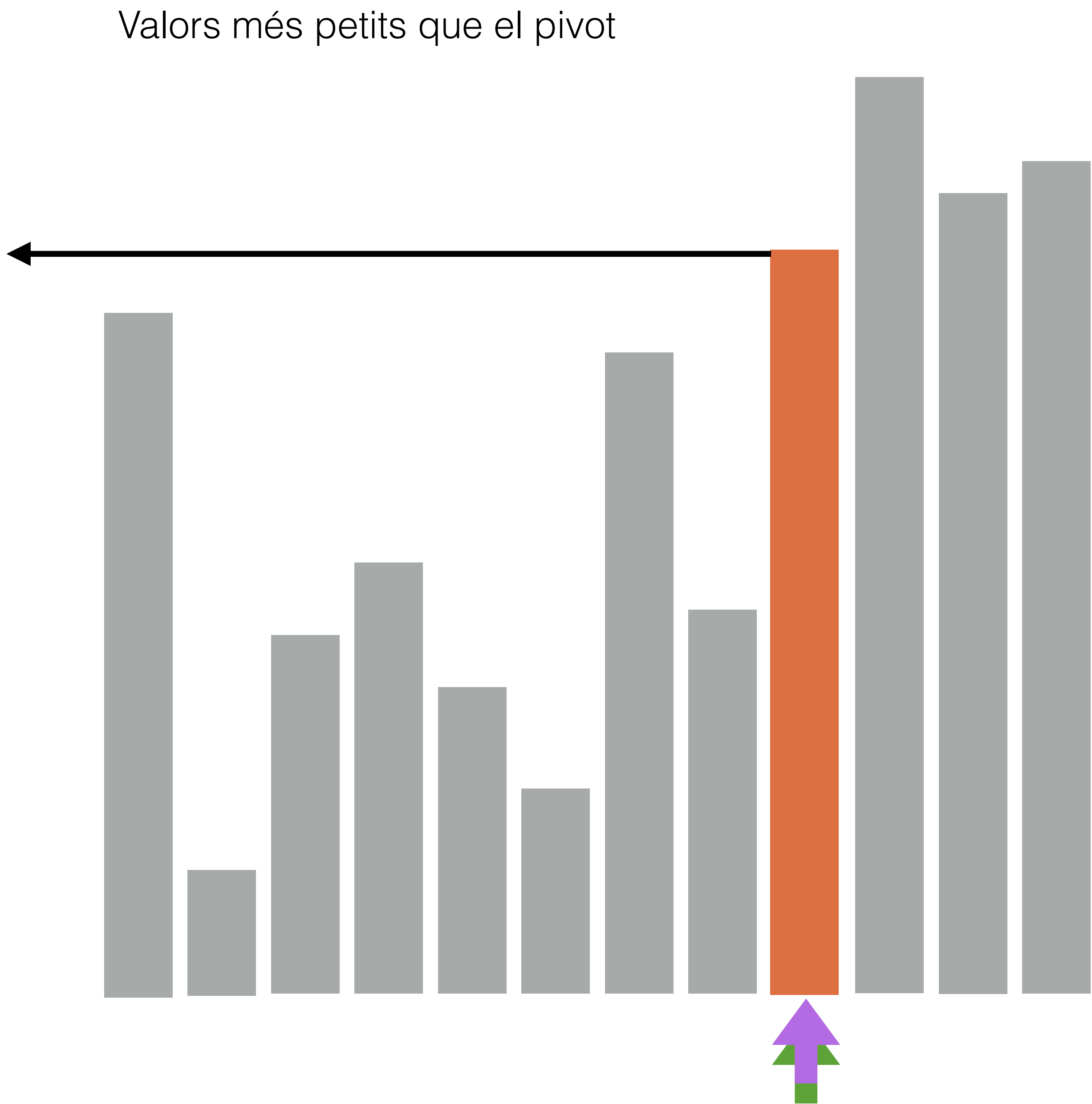
Quicksort

Funcionament



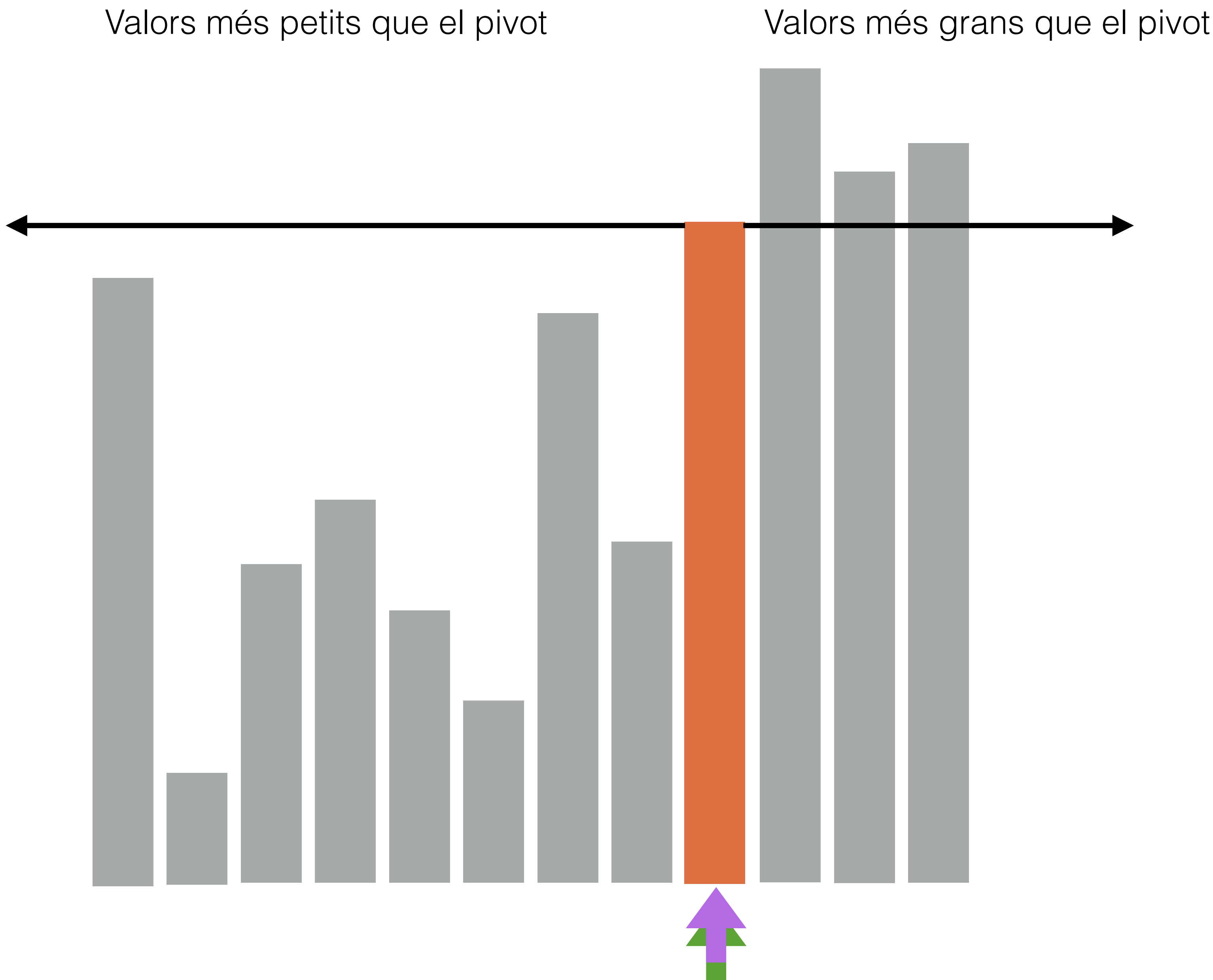
Quicksort

Funcionament



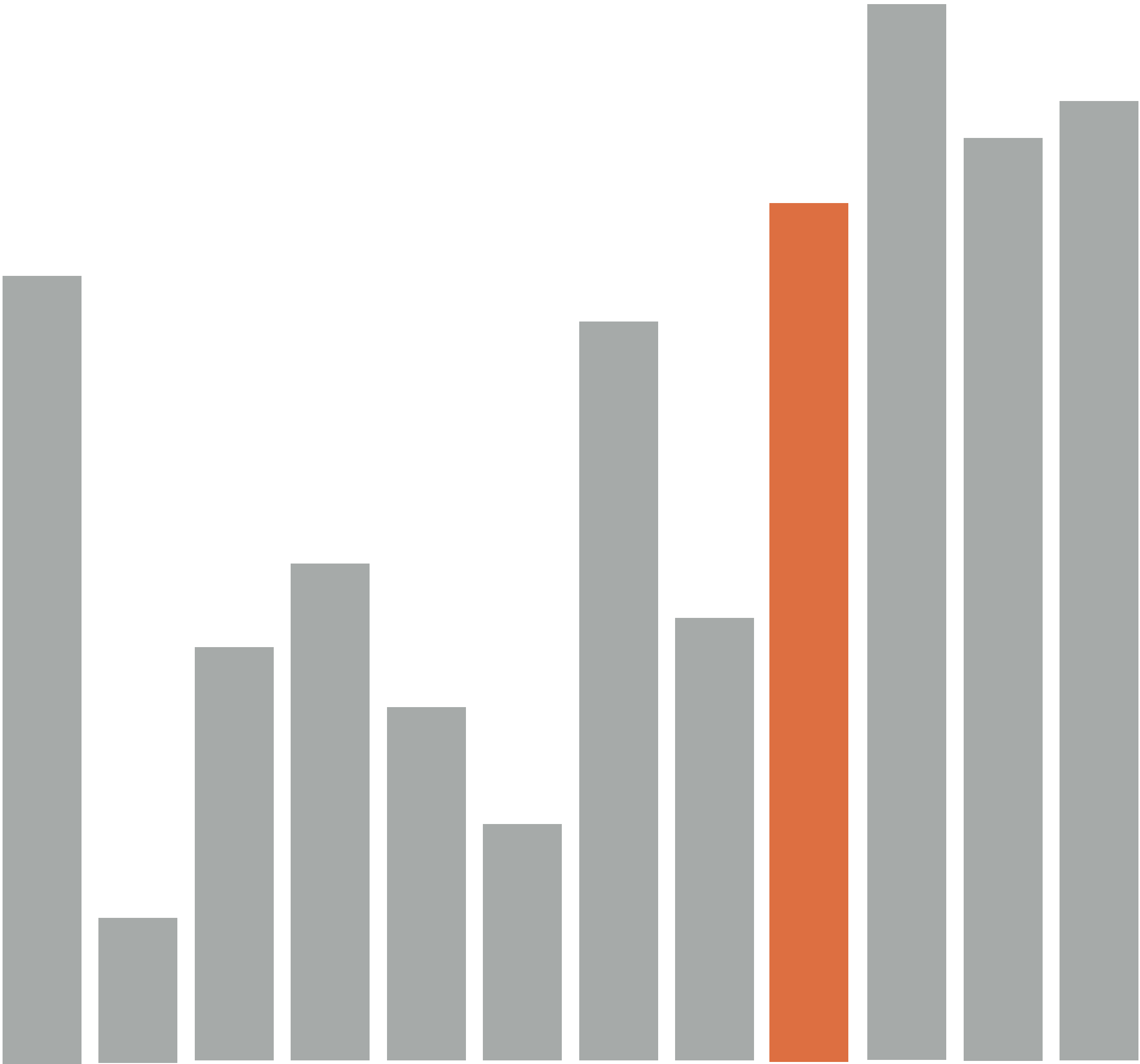
Quicksort

Funcionament



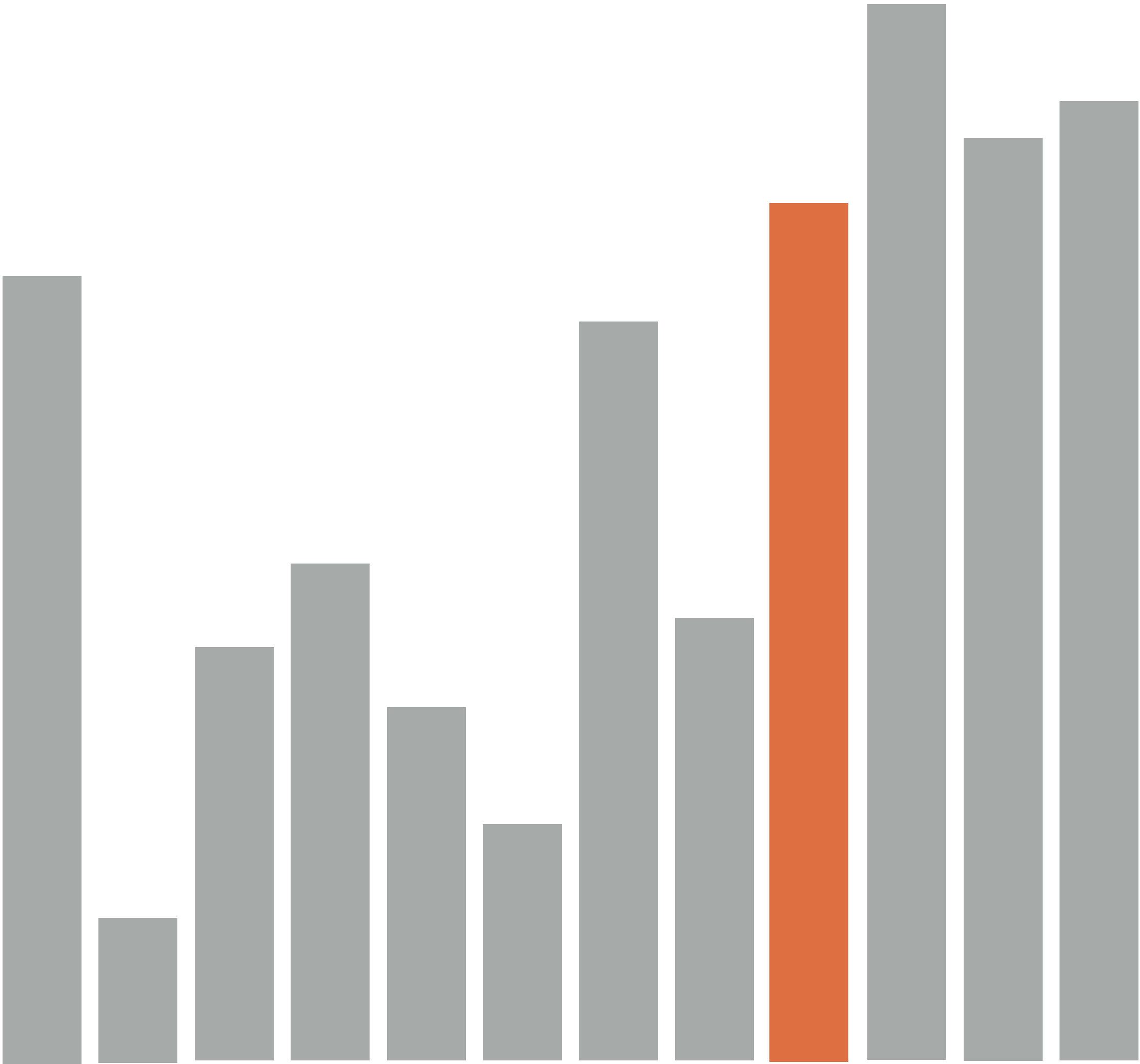
Quicksort

Funcionament



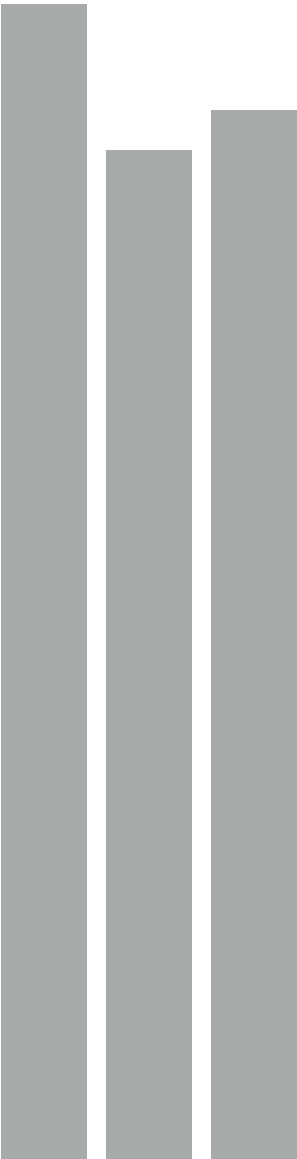
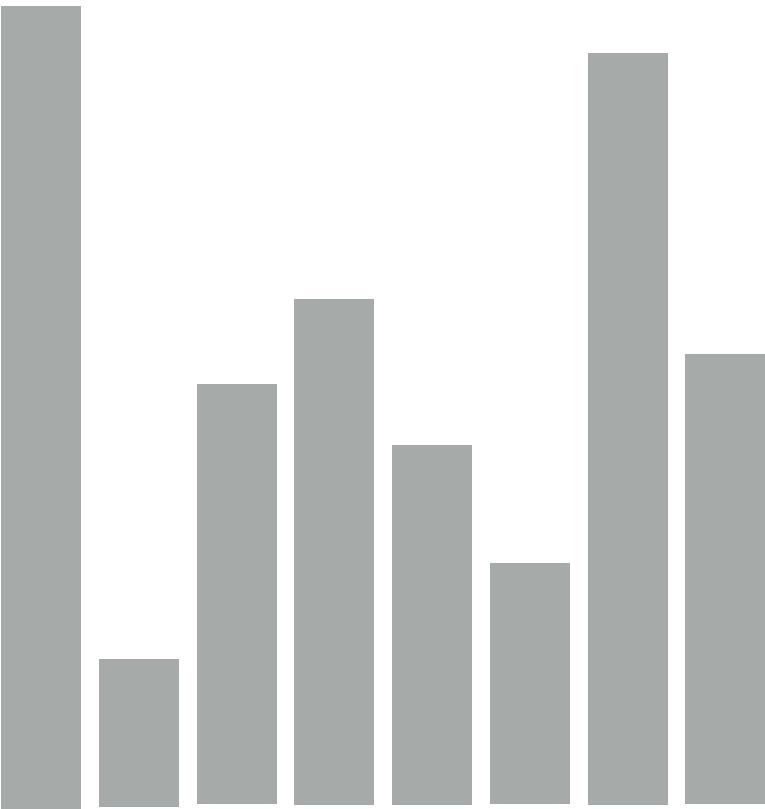
Quicksort

Funcionament



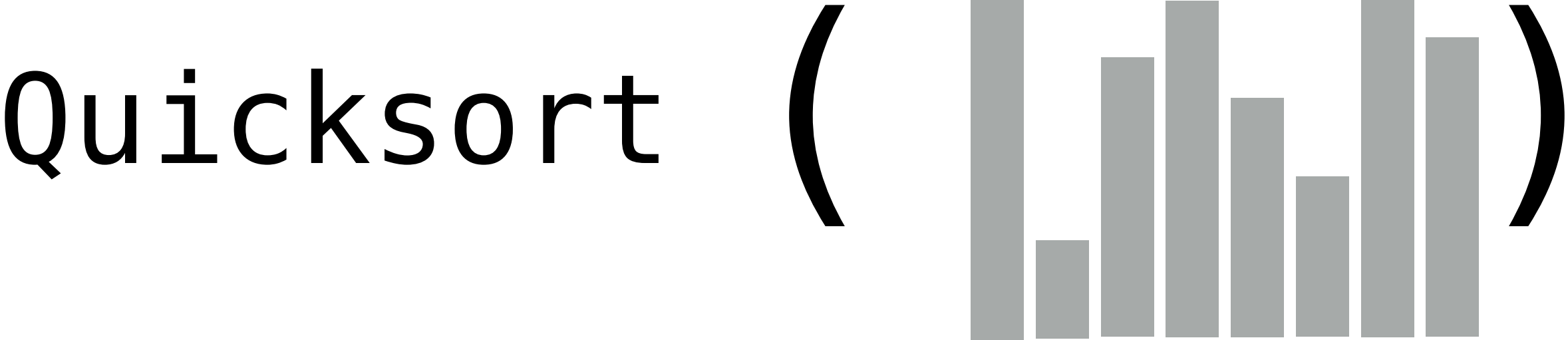
Quicksort

Funcionament



Quicksort

Funcionament



Quicksort

Funcionament

Quicksort



ini

fin

Quicksort



ini

fin

Quicksort

Funcionament

```
void Quicksort(int v[], int ini, int fin){
    int mig, left, right, pivot;
    if (ini == fin)
    { // Cas directe
        return;
    }
    else // if ini<fin
    { // Cas recursiu
        pivot = v[ini];
        Ordenar fins que left i right es creuin;
        mig = on left i right es creuen;
        Quicksort(v,ini,mig-1);
        Quicksort(v,mig+1,fin);
    }
}
```